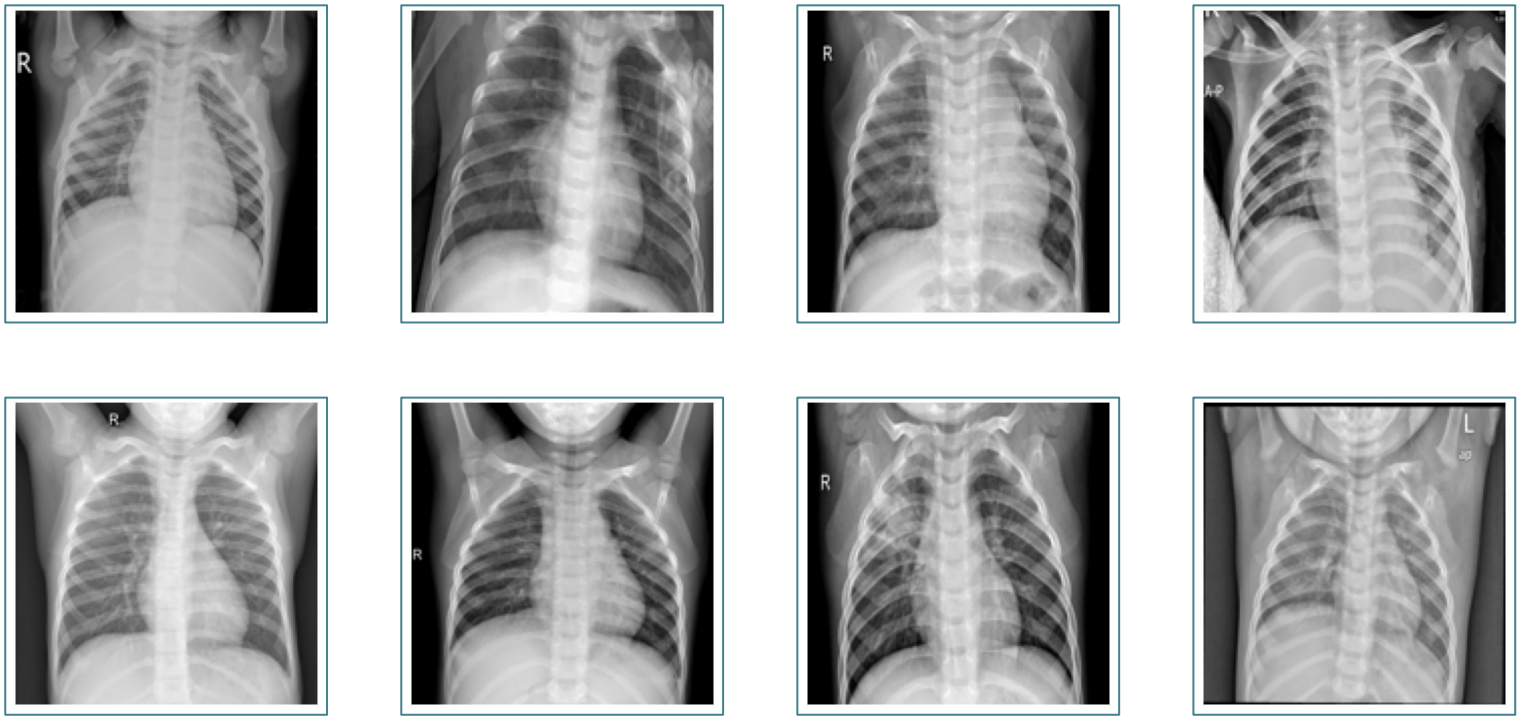
Reconnaissance d'images

TIPE - Mathématiques et informatique

DENIS Mathys - Spécialité S.I en MP - Candidat 48701



Introduction

Comment développer un algorithme capable de classifier des radiographies ?

01 Réseau linéaire basique

02 A) Implémentation avec des images

B) Couche intermédiaire

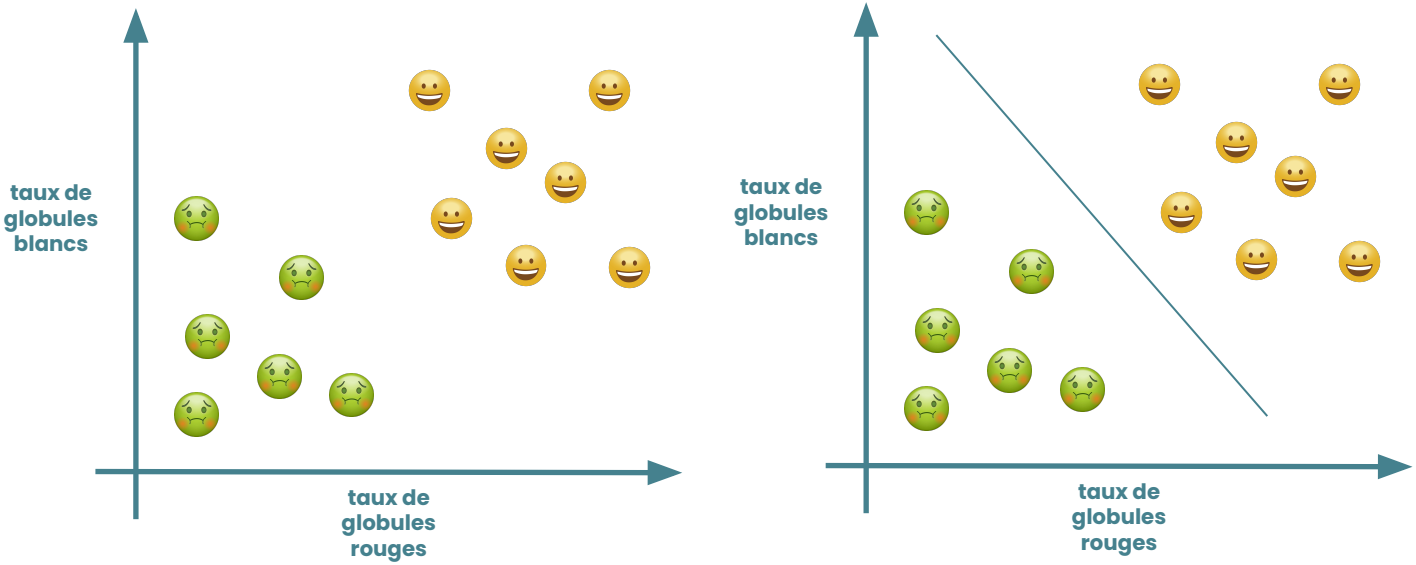
03 Convolution - Bilan

01

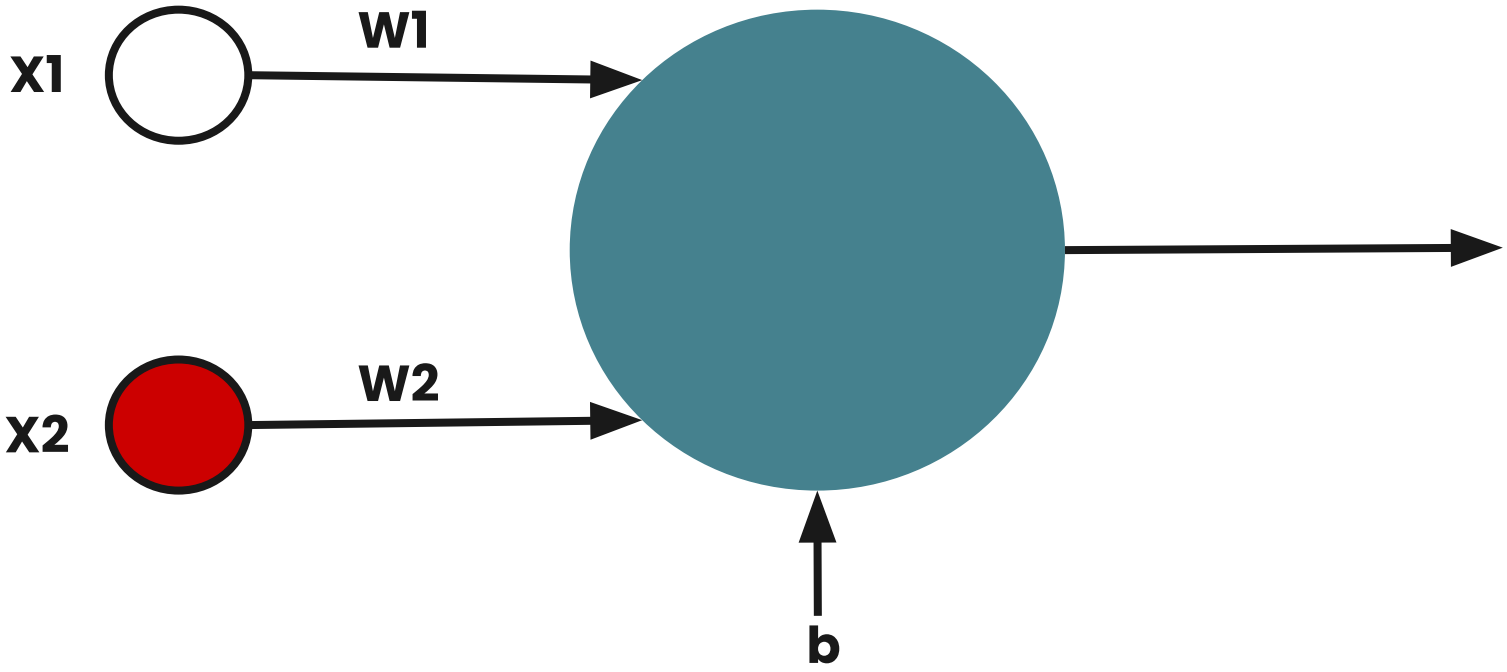
Réseau linéaire basique -

Patients sains / malades

I. Réseau linéaire basique -



I. Réseau linéaire basique -



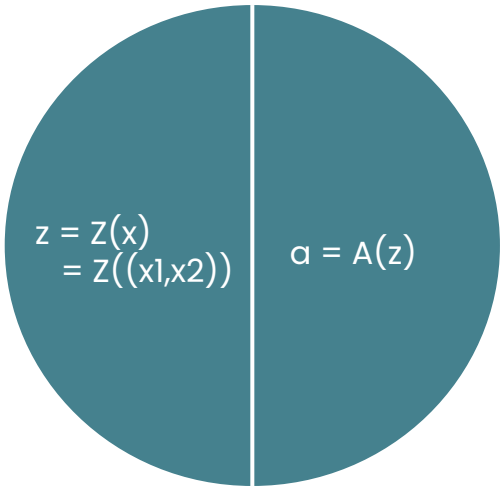
I. Réseau linéaire basique -

Z(x) = w1 x1 + w2 x2 +b

$$X = \begin{pmatrix} x_1 & x_2 \end{pmatrix}$$

$$W = \begin{pmatrix} w_1 \\ w_2 \end{pmatrix}$$

$$Z = X \cdot W + b$$

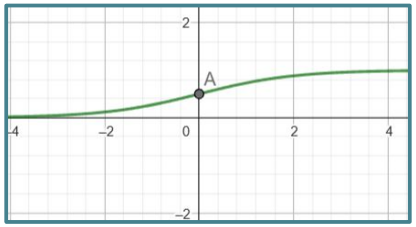


- $\sigma : \mathbb{R} \to \mathbb{R}$

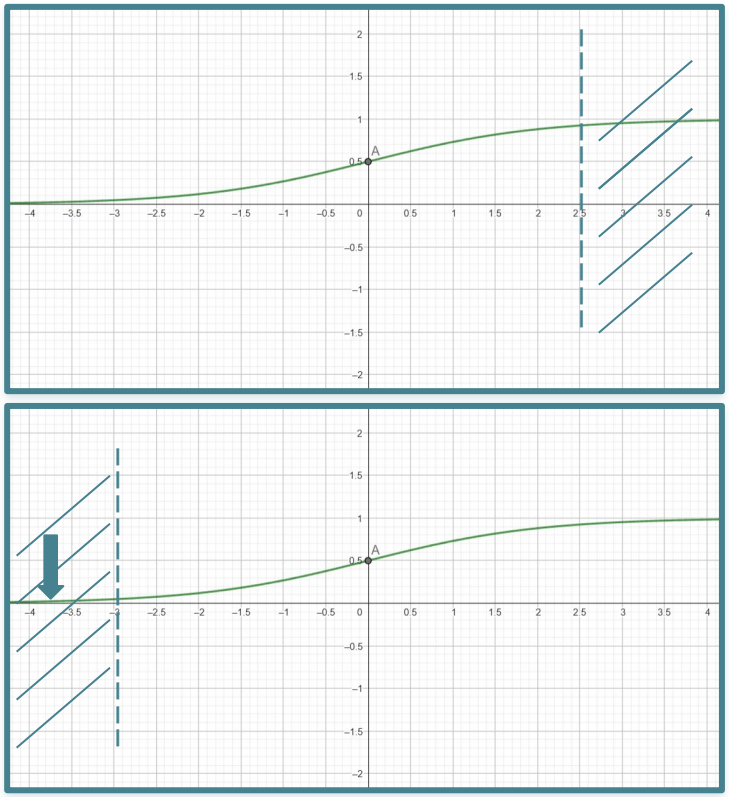
$$x \mapsto \frac{1}{1+e^{-x}}$$

- $\sigma \in \mathcal{C}^1(\mathbb{R}; \mathbb{R})$
- $\forall x \in \mathbb{R},$

$$\sigma'(x) = \sigma(x) \cdot (1 - \sigma(x))$$



I. Réseau linéaire basique -



$$\sigma : \mathbb{R} \to \mathbb{R}$$

$$x \mapsto \frac{1}{1+e^{-x}}$$

- $\sigma \in \mathcal{C}^1(\mathbb{R};\mathbb{R})$
- $\forall x \in \mathbb{R},$

$$\sigma'(x) = \sigma(x) \cdot (1 - \sigma(x))$$

I. Réseau linéaire basique -

$$L = \prod_{i=1}^{n} P_{\{X=x_i\}}(Y = y_i)$$

$$= \prod_{i=1}^{n} a_i^{y_i} \left(1 - a_i\right)^{1-y_i}$$

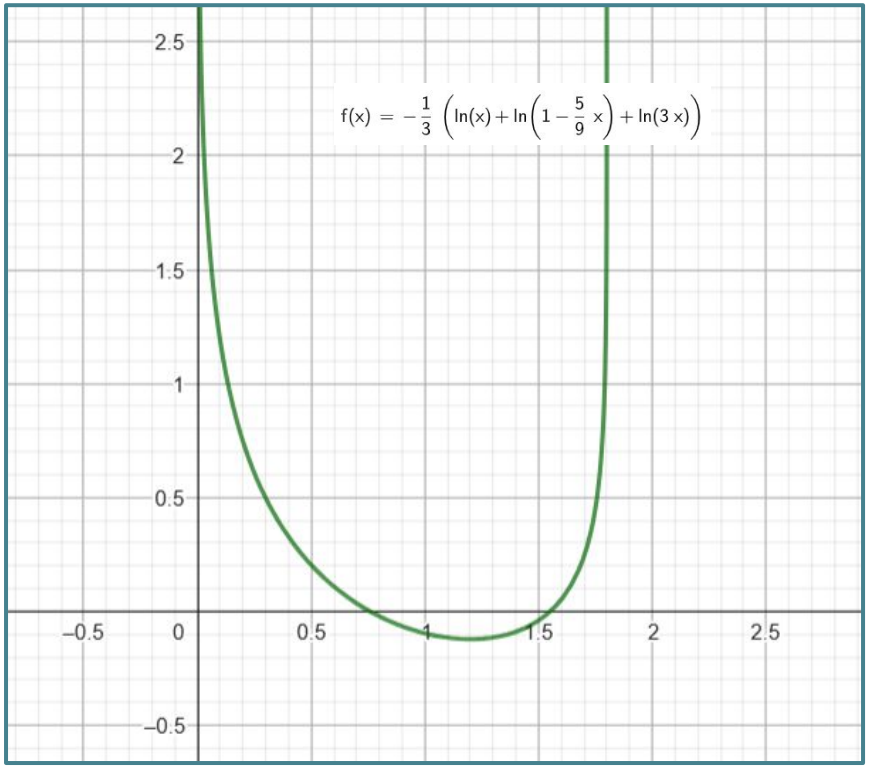
$$Y \sim \mathcal{B}(p) \quad \text{avec} \quad p = P_{\{X=x\}}(Y = 1) = \sigma(z) = a(z)$$

$$\forall y \in Y(\Omega) = \{0, 1\}, \quad P_{\{X=x\}}(Y = y) = a(z)^y \left(1 - a(z)\right)^{1-y}$$

$$\ln(L) = \sum_{i=1}^{n} y_i \ln(a_i) + (1 - y_i) \ln(1 - a_i)$$

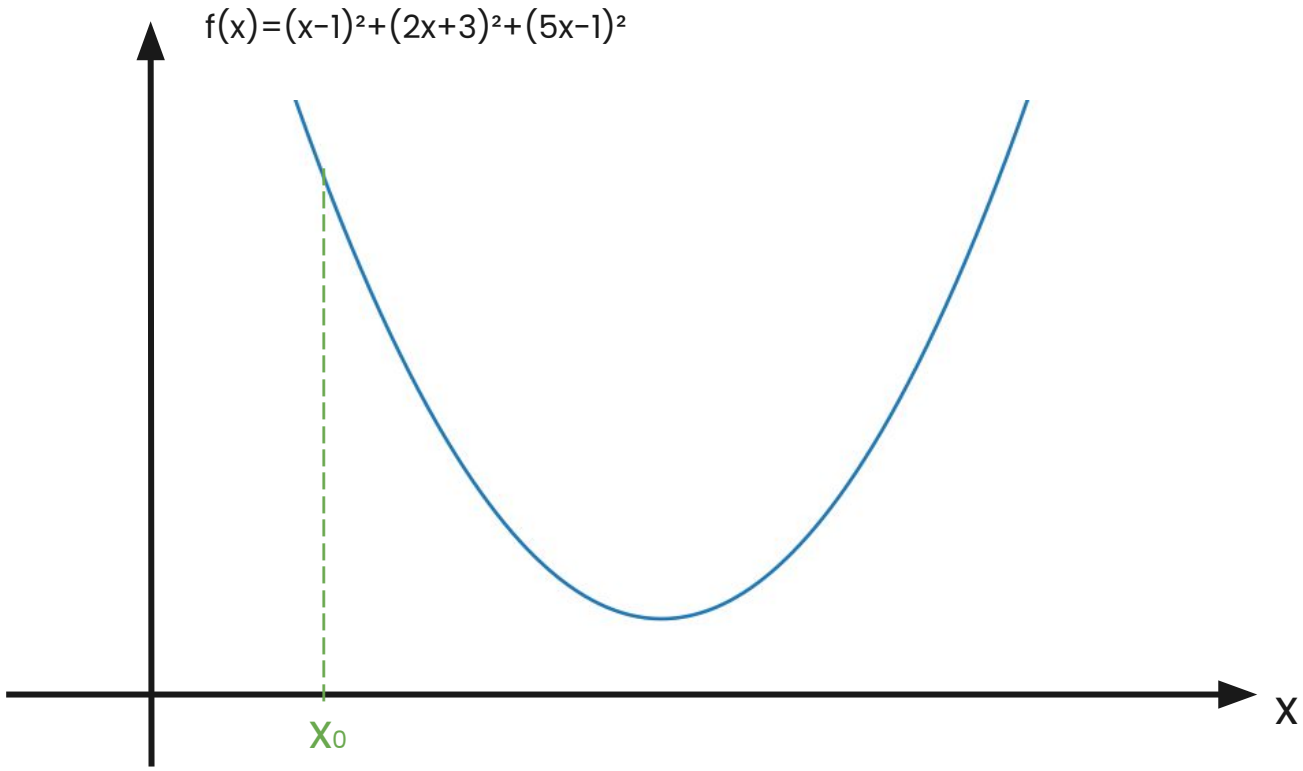
$$E = -\frac{1}{n} \sum_{i=1}^{n} \left[y_i \ln(a_i) + (1 - y_i) \ln(1 - a_i) \right]$$

I. Réseau linéaire basique -

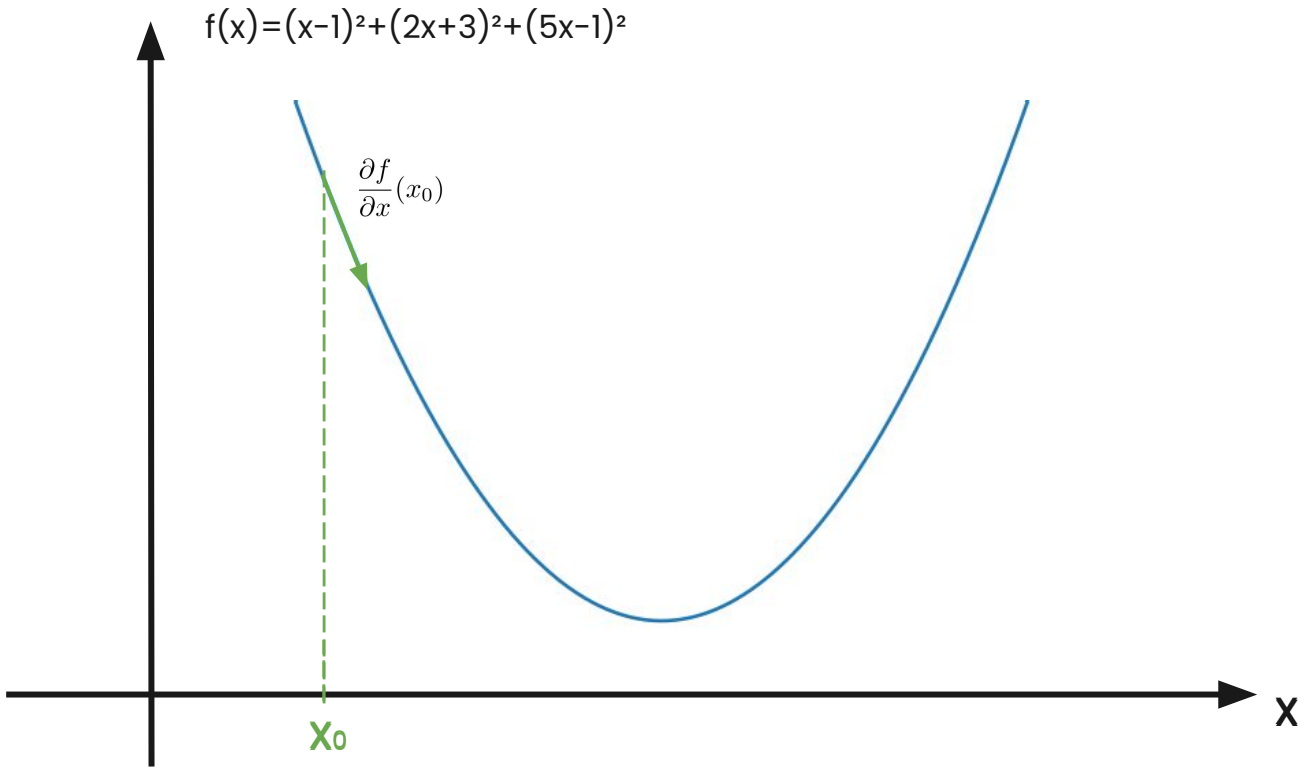


Représentation de la fonction erreur à une dimension

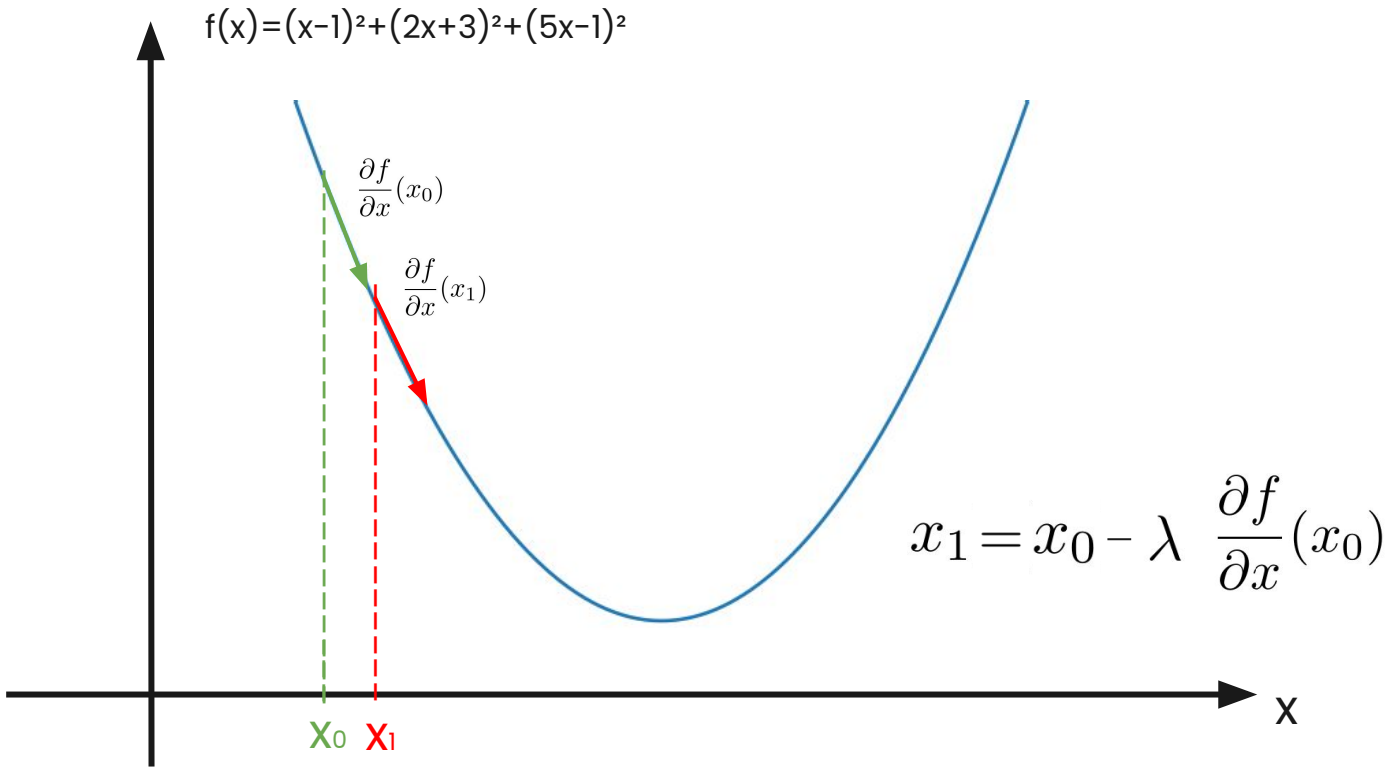
I. Réseau linéaire basique -



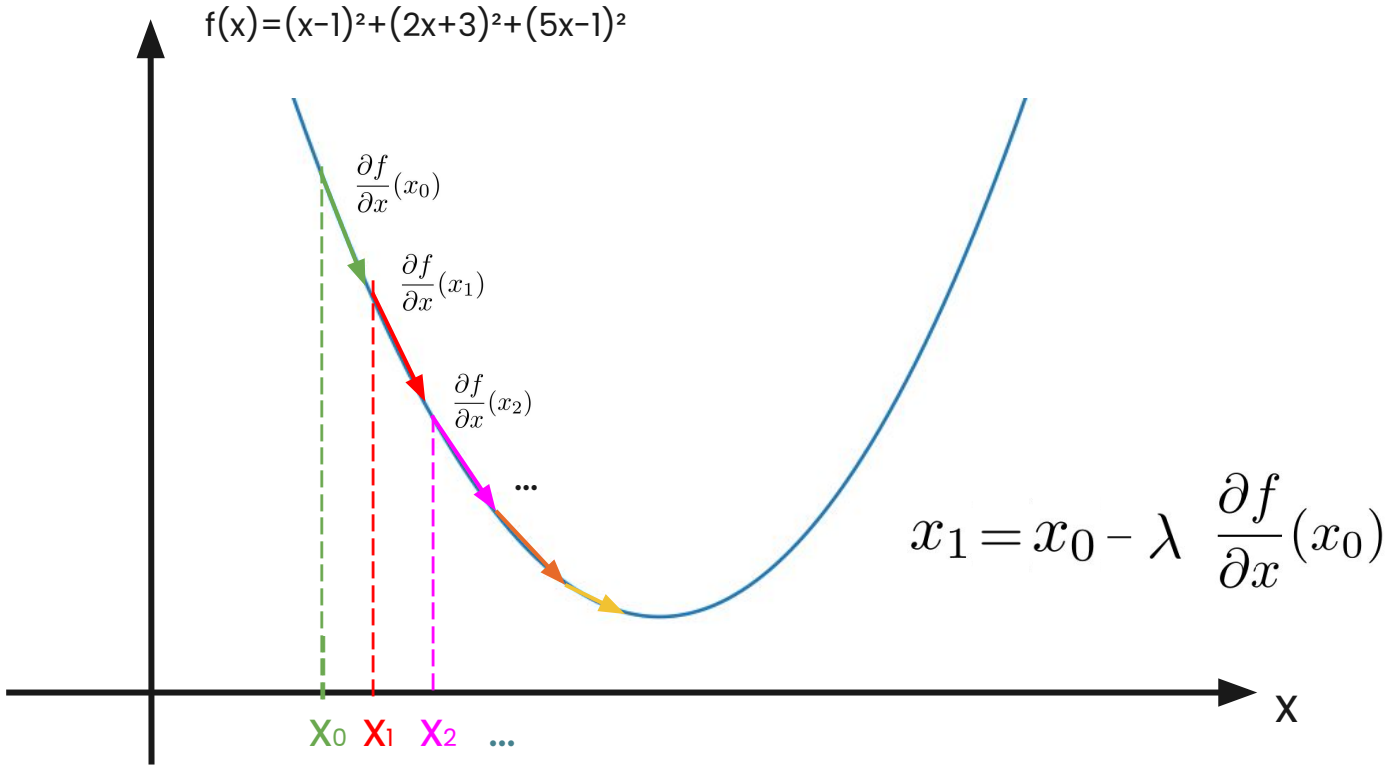
I. Réseau linéaire basique -



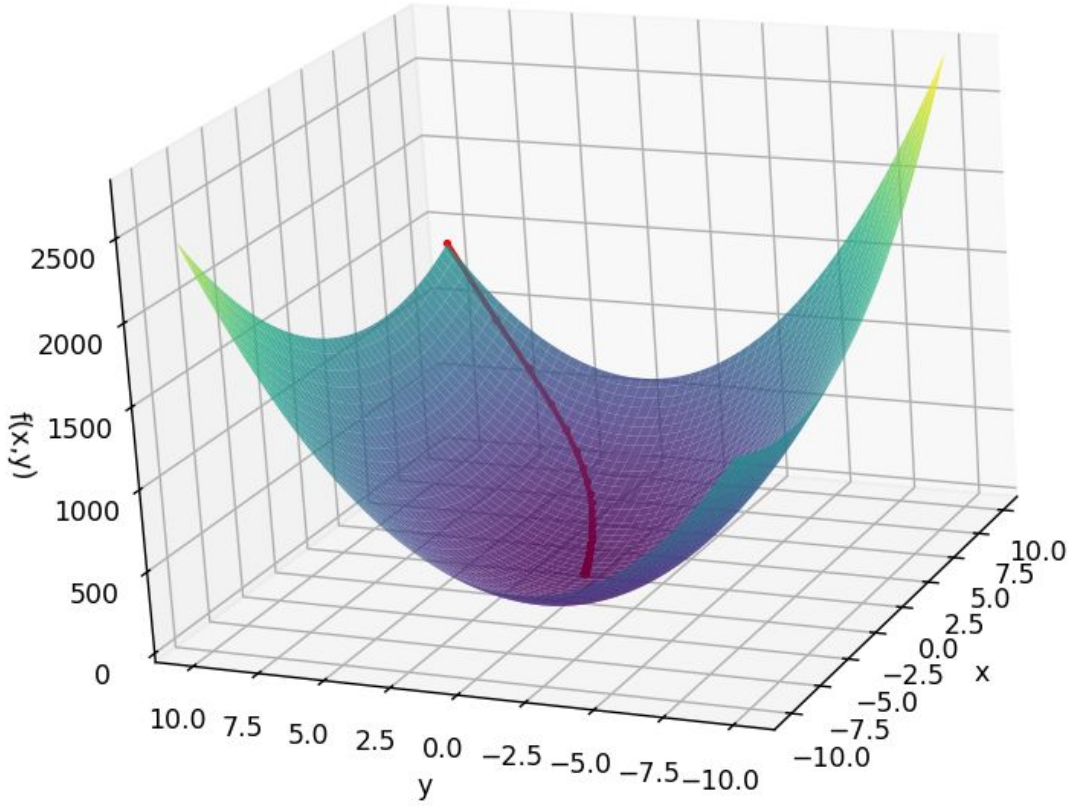
I. Réseau linéaire basique -



I. Réseau linéaire basique -



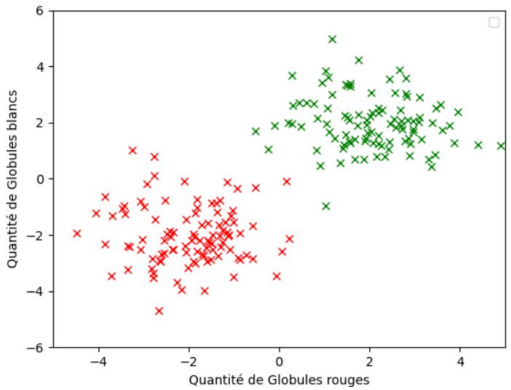
I. Réseau linéaire basique -

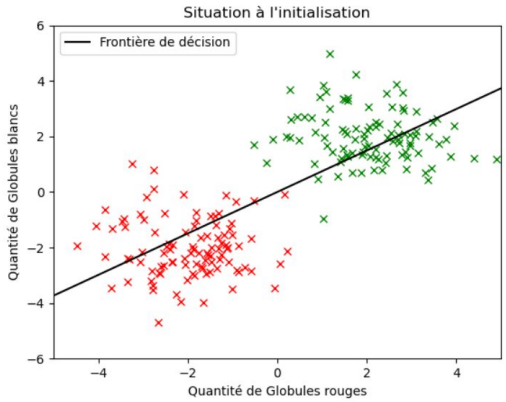


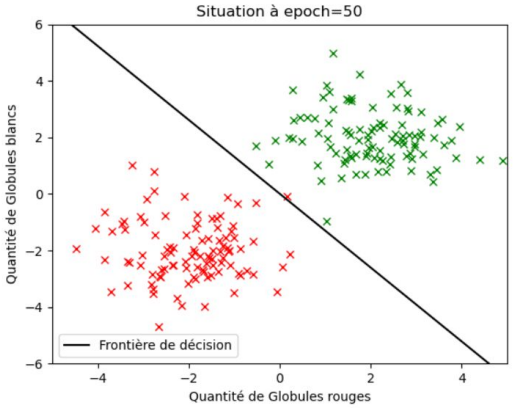
I. Réseau linéaire basique -

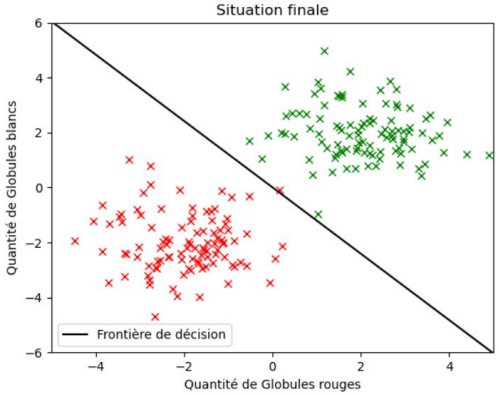
$$\omega_{k+1} = \omega_k - \lambda \nabla \mathcal{E}(\omega_k)$$

I. Réseau linéaire basique -

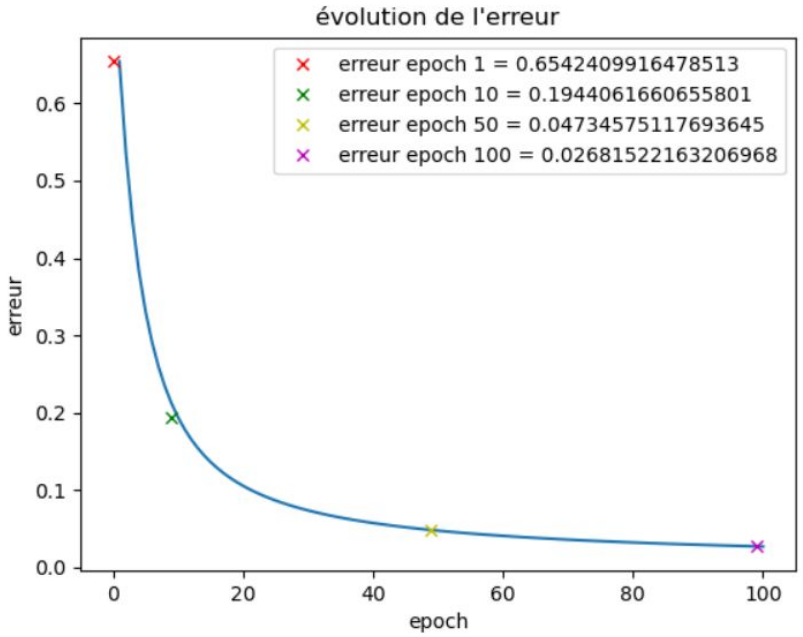


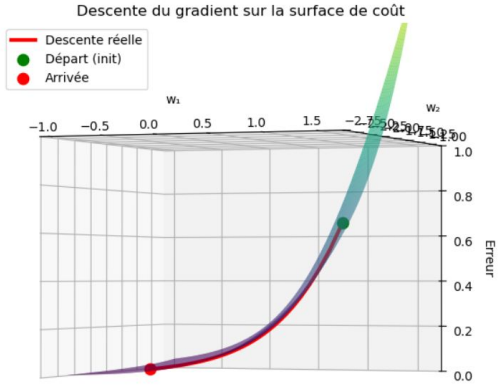


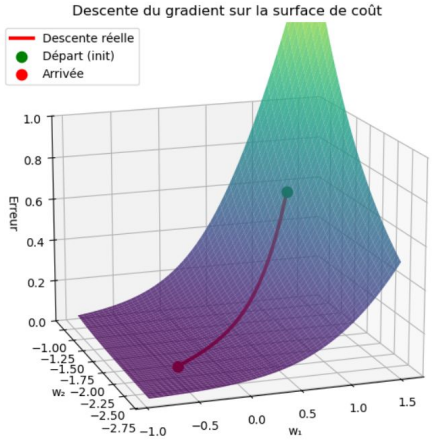




I. Réseau linéaire basique -







I. Réseau linéaire basique -

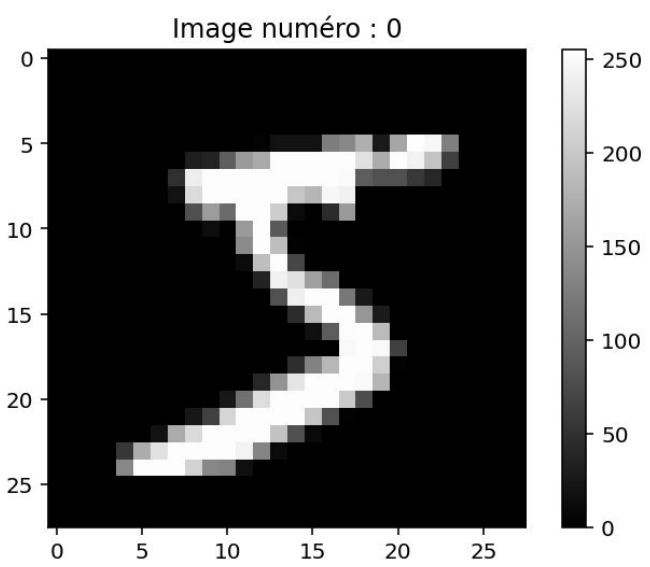
```
In [29]: %runfile 'C:/Users/23mat/OneDrive/Bureau/TIPE/CONCOURS TIPE/Programmes/
globule_rouge_blanc.py' --wdir
c:\users\23mat\onedrive\bureau\tipe\concours tipe\programmes\globule_rouge_blanc.py:73: UserWarning:
No artists with labels found to put in legend.  Note that artists whose label start with an underscore
are ignored when legend() is called with no argument.
  plt.legend()
c:\users\23mat\onedrive\bureau\tipe\concours tipe\programmes\globule_rouge_blanc.py:269: UserWarning:
No artists with labels found to put in legend.  Note that artists whose label start with an underscore
are ignored when legend() is called with no argument.
  plt.legend()
L'algorithme est fiable à 99.11%
```

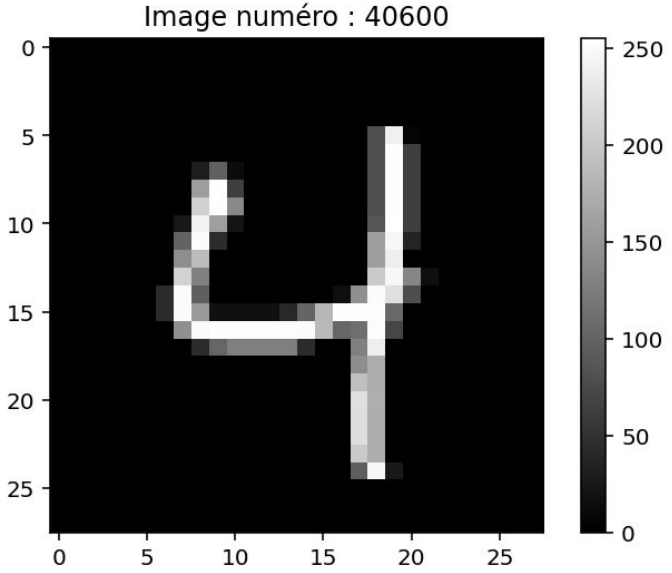
02

Implémentation avec des images -

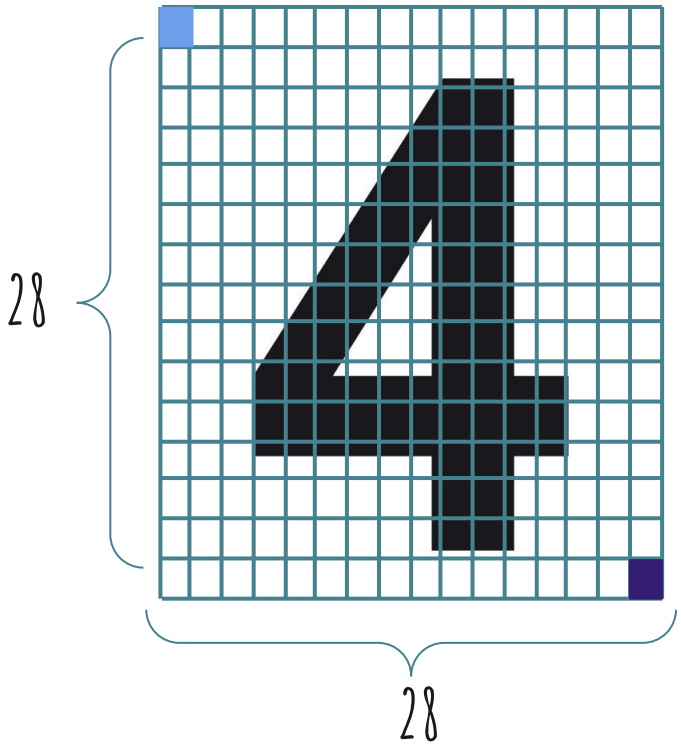
MNIST

II. Implémentation avec des images -



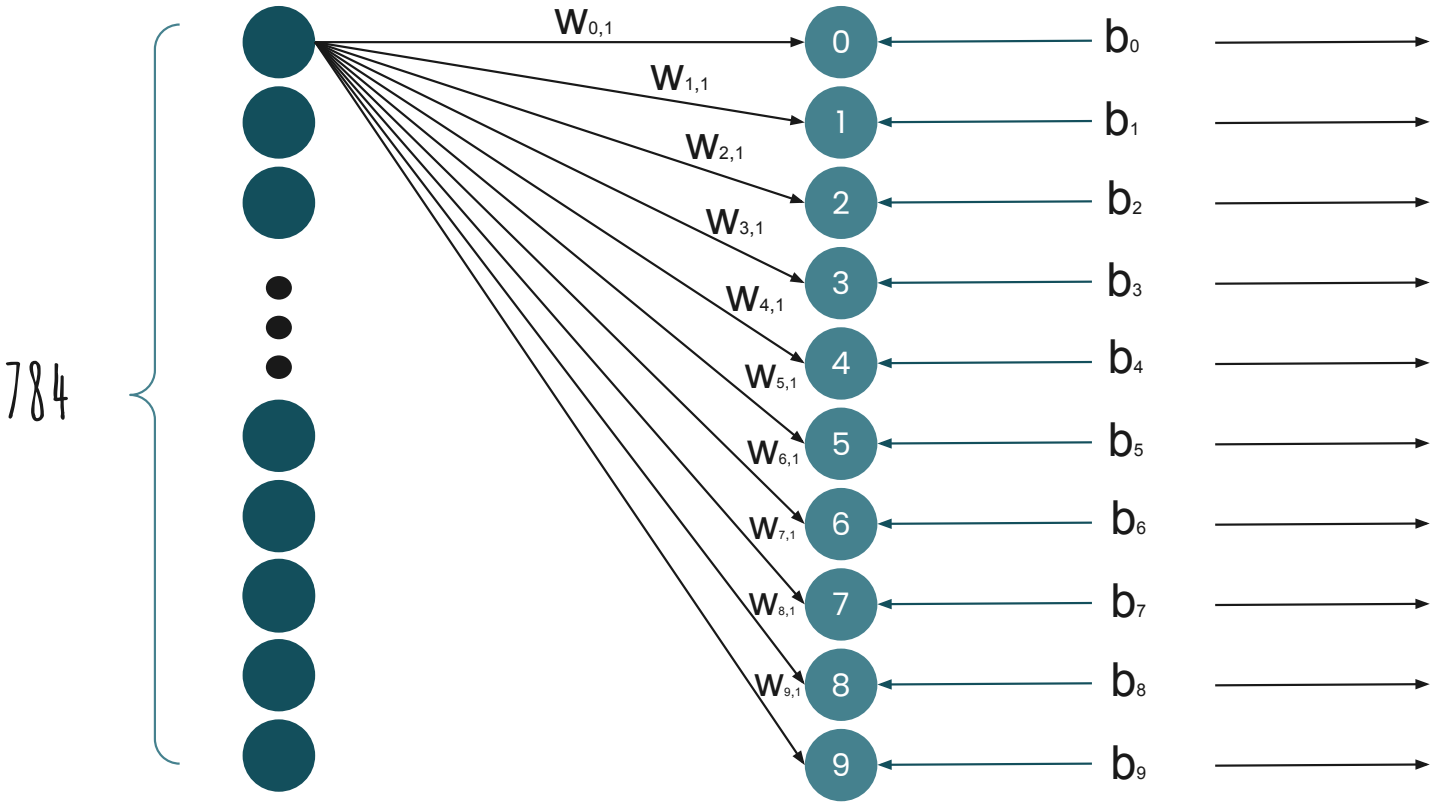


II. Implémentation avec des images -



$$I = \begin{pmatrix} i_1 & i_2 & \cdots & i_{784} \end{pmatrix}$$

II. Implémentation avec des images -



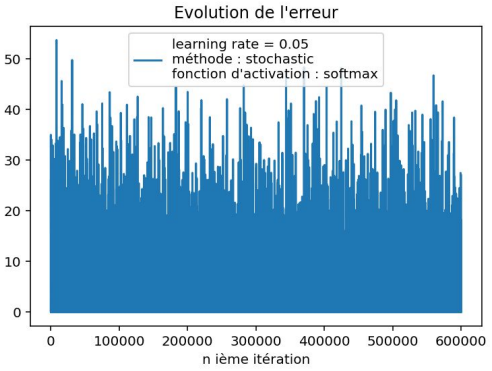
II. Implémentation avec des images -

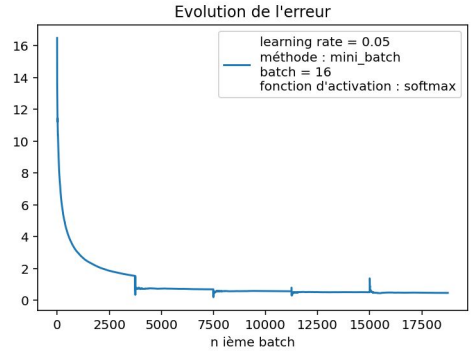
Pour un modèle de N classes, $N \in \mathbb{N}$:

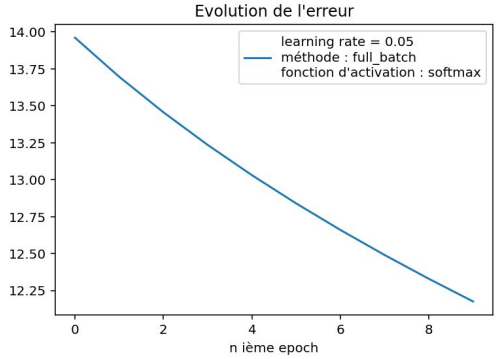
$$S : \mathbb{R}^N \to \mathbb{R}^N$$

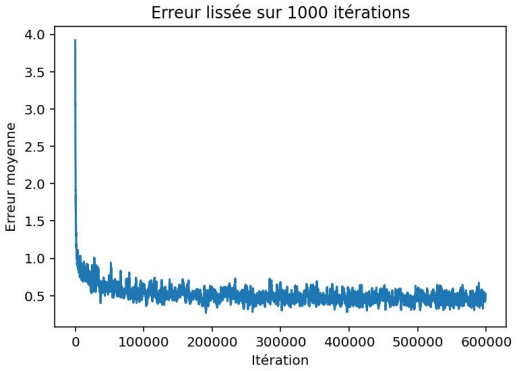
$$Z = (Z_1, \ldots, Z_N) \longmapsto \left(\frac{e^{Z_1}}{\sum_{k=1}^{N} e^{Z_k}}, \ldots, \frac{e^{Z_N}}{\sum_{k=1}^{N} e^{Z_k}} \right)$$

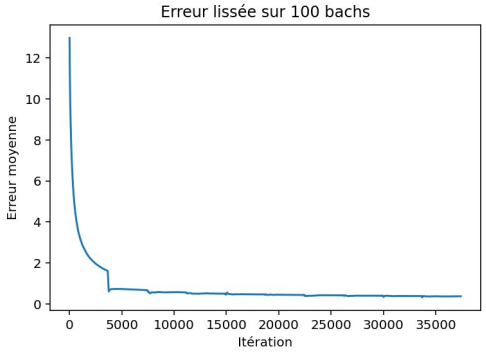
Evolution de l'erreur au cours de l'apprentissage avec la fonction d'activation softmax

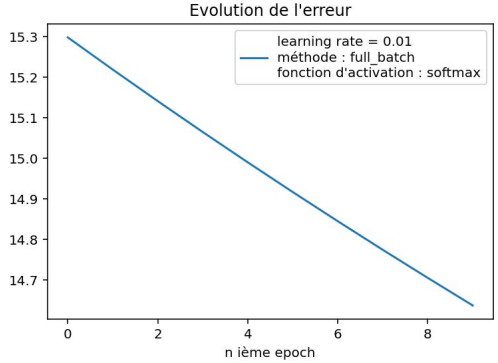












stochastic *Mini_batch_16* *Full_batch*

II. Implémentation avec des images -

*Précision obtenue en 10 epochs et un learning rate de **0.05***

	sigmoïde	softmax
Stochastic	88.64 %	89.52 %
Mini_batch_16	90.91 %	90.33 %
Mini_batch_32	89.65 %	89.58 %
Mini_batch_128	86.71 %	85.6 %
Full_batch	8.52 %	7.87 %

II. Implémentation avec des images -

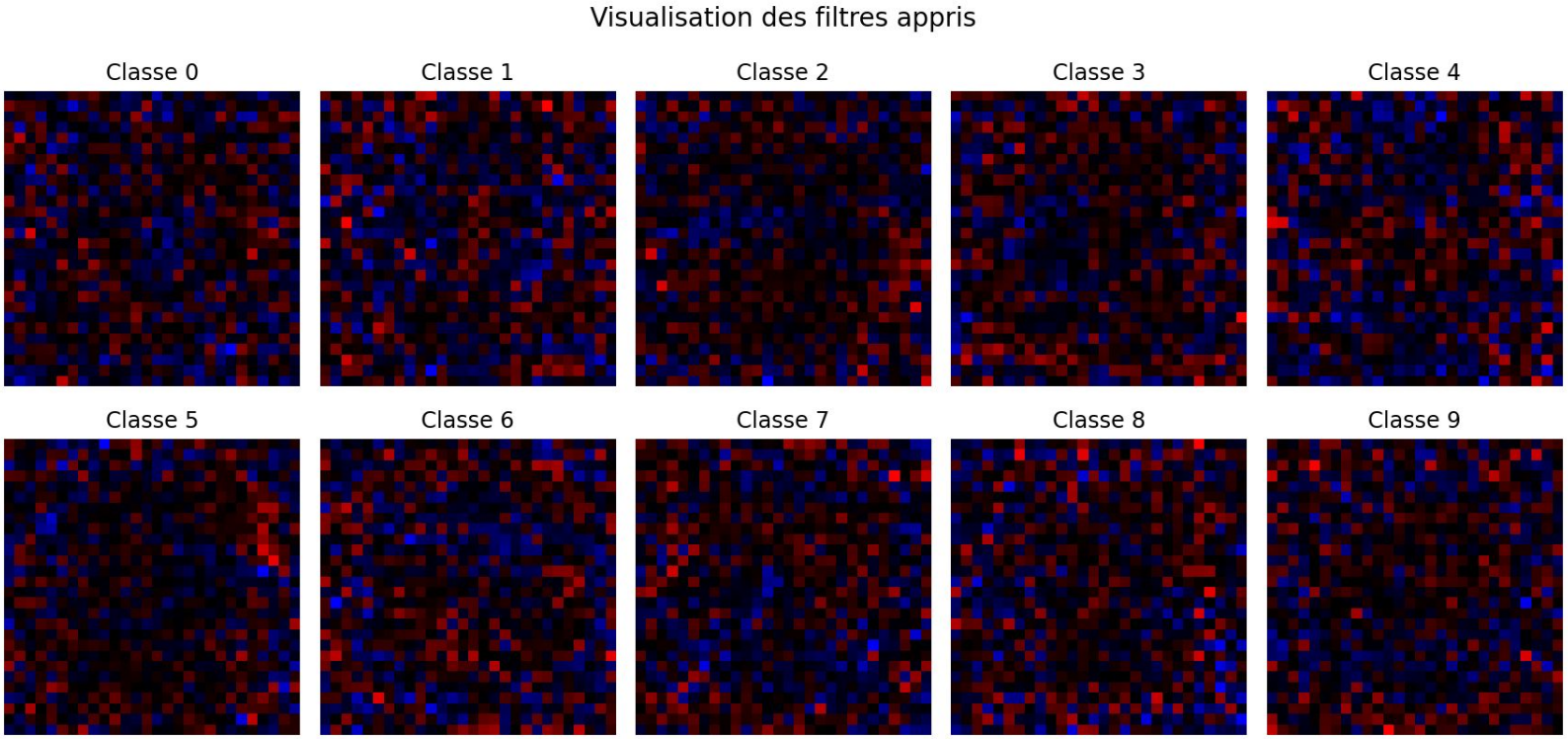
*Précision obtenue avec la fonction d'activation **softmax** et un learning rate de **0.05***

	epoch = 5	epoch = 10	epoch = 15
Stochastic	90.12 %	89.52 %	86.65 %
Mini_batch_16	89.29 %	90.33 %	90.88 %
Mini_batch_32	87.57 %	89.58 %	89.58 %
Mini_batch_128	80.68 %	85.6 %	87.02 %
Full_batch	15.92 %	7.87 %	9.47 %

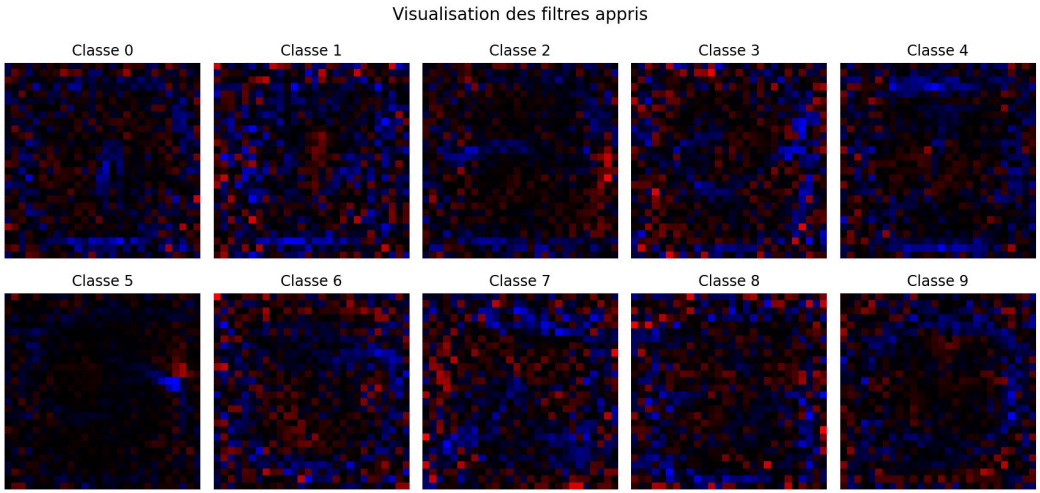
II. Implémentation avec des images -

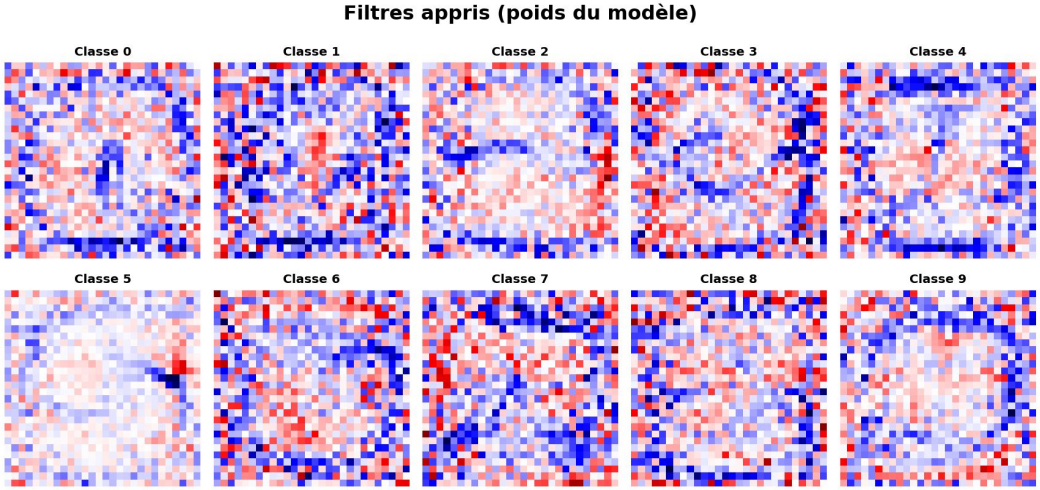
*Précision obtenue avec la fonction d'activation **softmax** et un learning rate de **0.01***

	epoch = 5	epoch = 10	epoch = 15
Stochastic	90.28 %	91.36 %	91.69 %
Mini_batch_16	84.19 %	87.01 %	87.93%
Mini_batch_32	80.21 %	84.73 %	85.82 %
Mini_batch_128	63.65 %	74.71 %	77.57 %
Full_batch	6.38 %	7.71 %	7.55 %

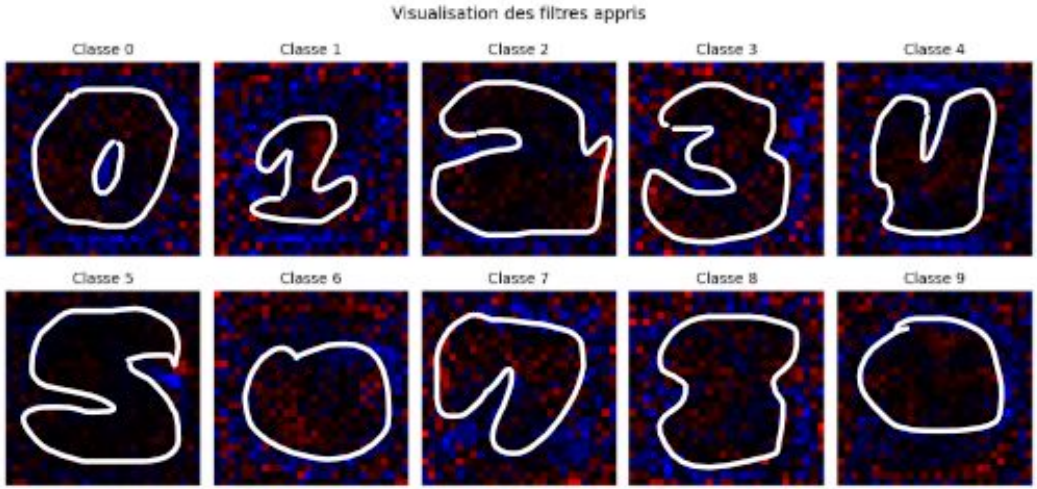


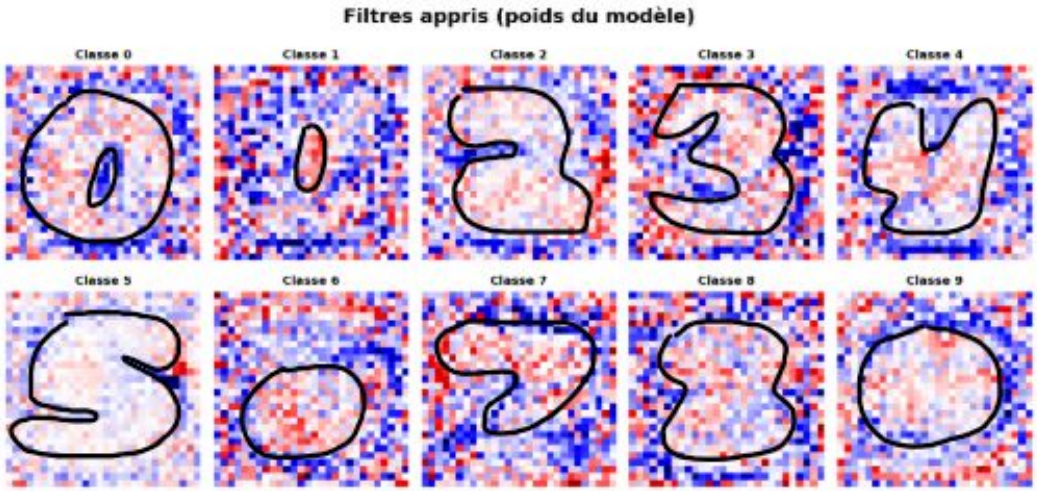
stochastic,lr=0.01, softmax, epoch 10





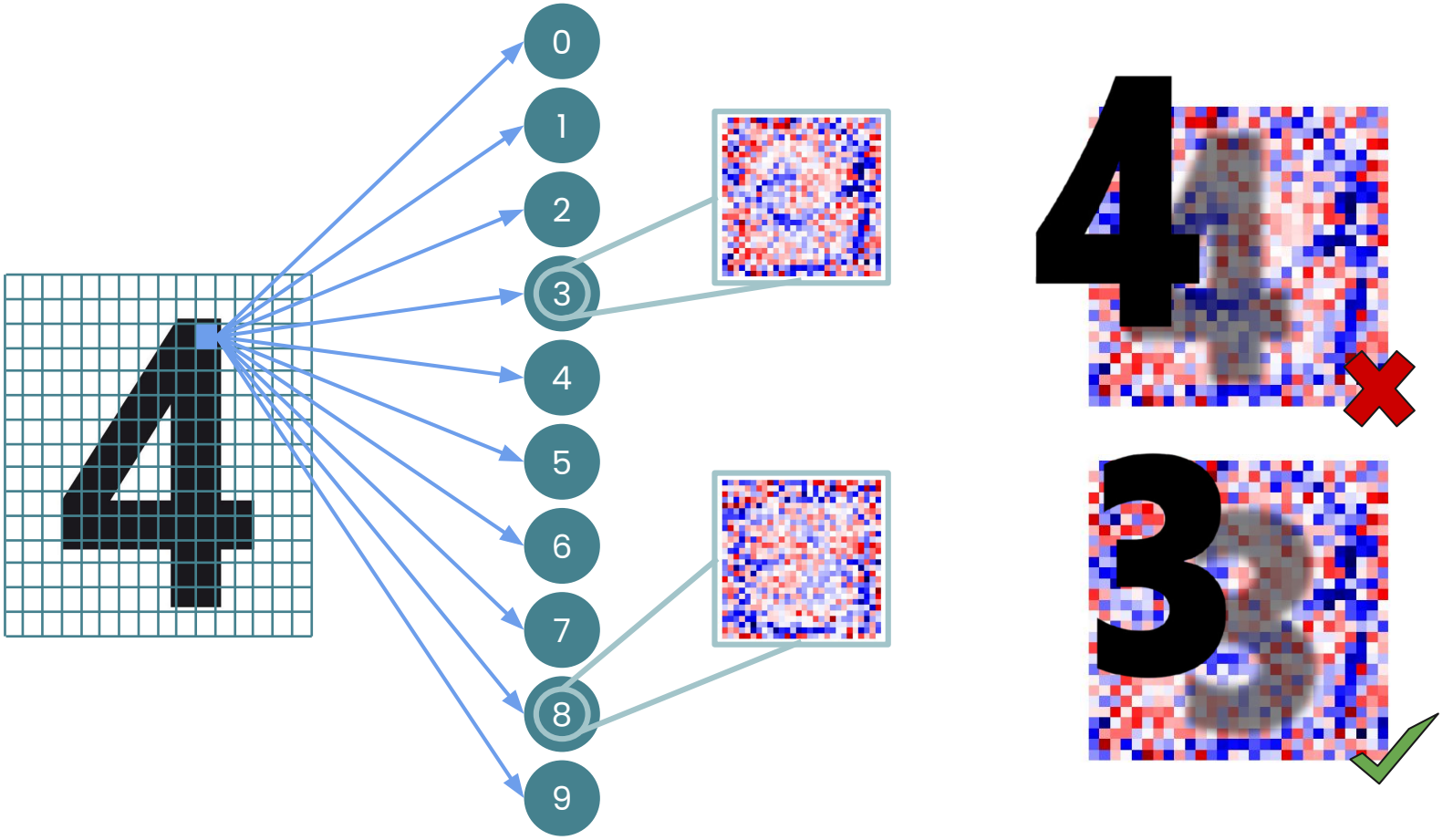
stochastic,lr=0.05, sigmoïde,epoch 10





stochastic,lr=0.05, sigmoïde,epoch 10

II. Implémentation avec des images -



```
In [7]: %runfile 'C:/Users/23mat/OneDrive/Bureau/TIPE/CONCOURS TIPE/
Programmes/Test_/Test_Test_ChestX_without_h.py' --wdir
✅ Données de test chargées
Nom du fichier des poids (sans le txt) :
weights_10_softmax_0.05_mini_batch_16
Nom du fichier des bias (sans le txt) : bias_10_softmax_0.05_mini_batch_16
✅ Poids et bias chargés
L'algorithme est précis à la hauteur de : 71.7948717948718 %
```



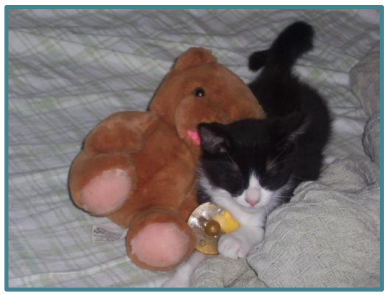


```
TIPE\data_set_dog_cat\archive\PetImages\Cat\666.jpg /!\
/!\ Problème avec C:\Users\23mat\OneDrive\Bureau\TIPE\CONCOURS
TIPE\data_set_dog_cat\archive\PetImages\Dog\11702.jpg /!\
✅ 4998 images test chargées
Nom du fichier des poids (sans le txt) :
weights_10_softmax_0.05_mini_batch_16
Nom du fichier des bias (sans le txt) : bias_10_softmax_0.05_mini_batch_16
✅ Poids et bias chargés
Précision sur le jeu de test : 59.16%
```

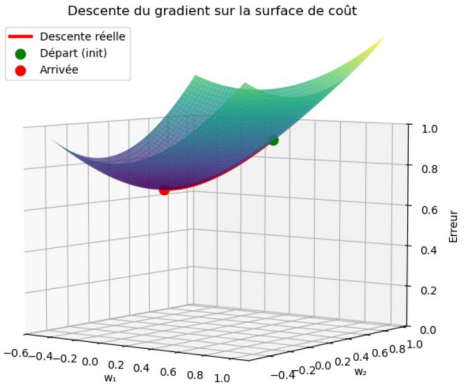


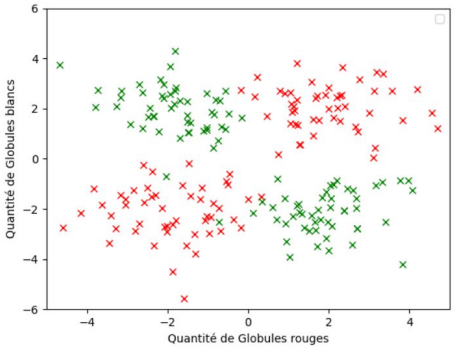


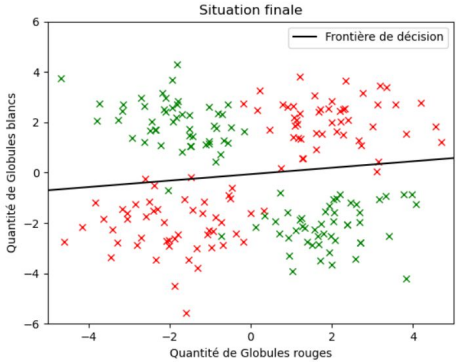




II. Couche intermédiaire -

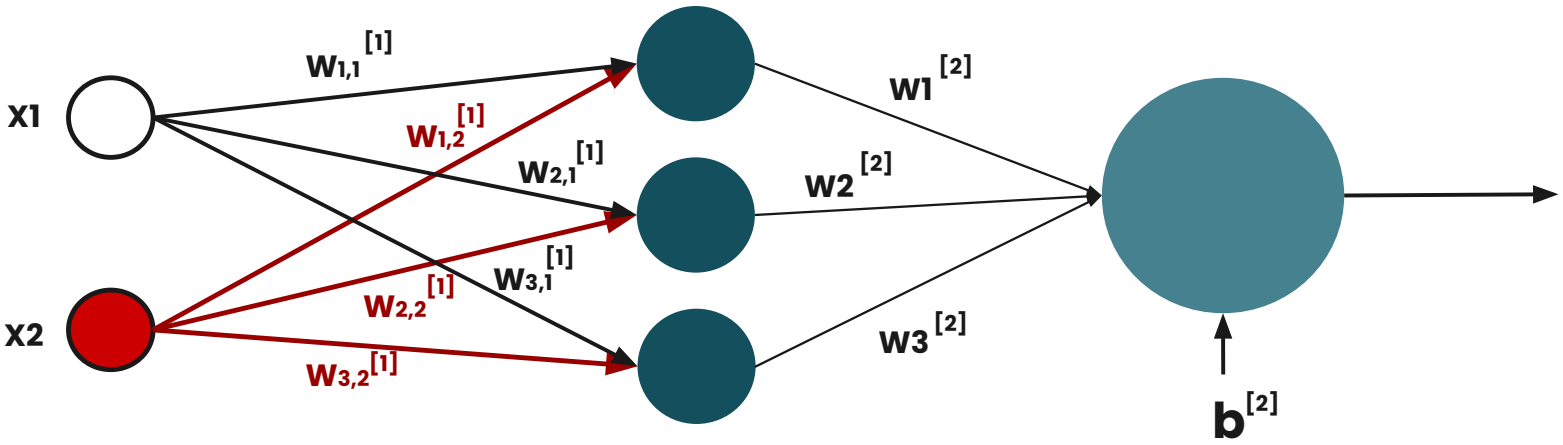




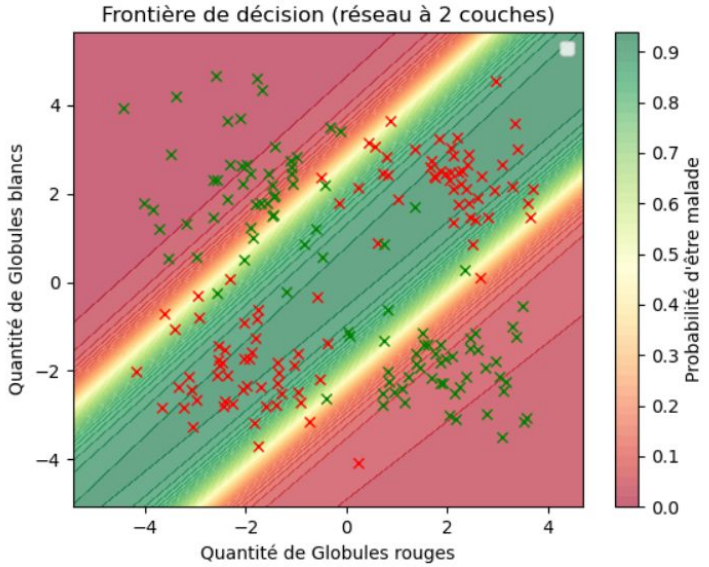



```
In [30]: %runfile 'C:/Users/23mat/OneDrive/Bureau/TIPE/CONCOURS TIPE/Programmes/
globule_rouge_blanc.py' --wdir
c:\users\23mat\onedrive\bureau\tipe\concours tipe\programmes\globule_rouge_blanc.py:73: UserWarning:
No artists with labels found to put in legend.  Note that artists whose label start with an underscore
are ignored when legend() is called with no argument.
  plt.legend()
c:\users\23mat\onedrive\bureau\tipe\concours tipe\programmes\globule_rouge_blanc.py:269: UserWarning:
No artists with labels found to put in legend.  Note that artists whose label start with an underscore
are ignored when legend() is called with no argument.
  plt.legend()
L'algorithme est fiable à 50.15500000000001%
```

II. Couche intermédiaire –

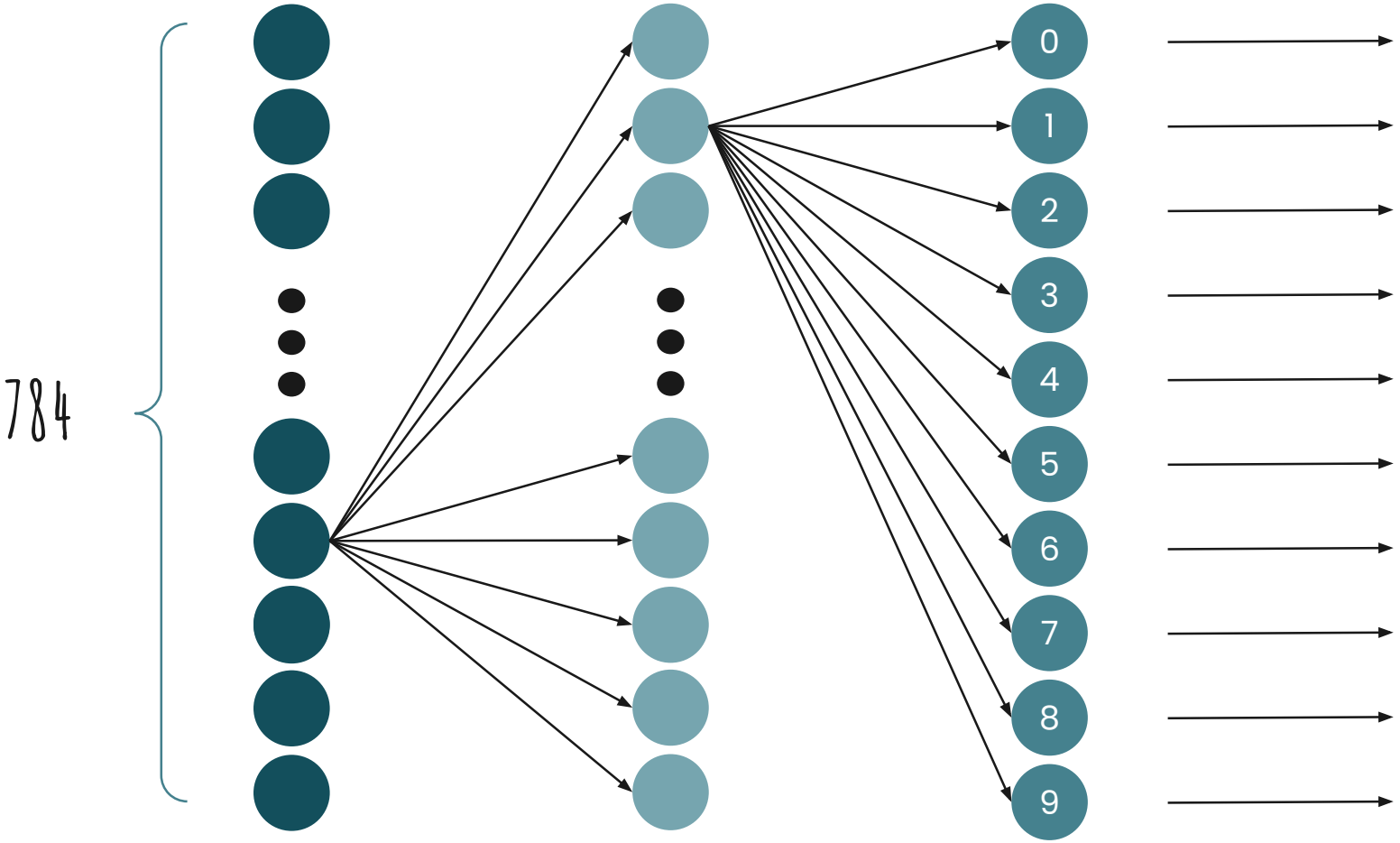


II. Couche intermédiaire -




```
In [34]: %runfile 'C:/Users/23mat/OneDrive/Bureau/TIPE/CONCOURS TIPE/Programmes/
globule_rouge_blanc_xor.py' --wdir
c:\users\23mat\onedrive\bureau\tipe\concours tipe\programmes\globule_rouge_blanc_xor.py:70:
UserWarning: No artists with labels found to put in legend.  Note that artists whose label start with
an underscore are ignored when legend() is called with no argument.
  plt.legend()
L'algorithme est fiable à 88.47%
```

II. Couche intermédiaire –



II. Couche intermédiaire –

Précision obtenue par mini batch 16 et une sous-couche de 64 neurones

	epoch = 10	epoch = 15
Sigmoïde (lr = 0.05)	92.79 %	92.95%
Sigmoïde (lr = 0.01)	86.72 %	89.16 %
Softmax (lr=0.05)	92.14 %	92.84 %
Softmax (lr=0.01)	87.03 %	88.54 %

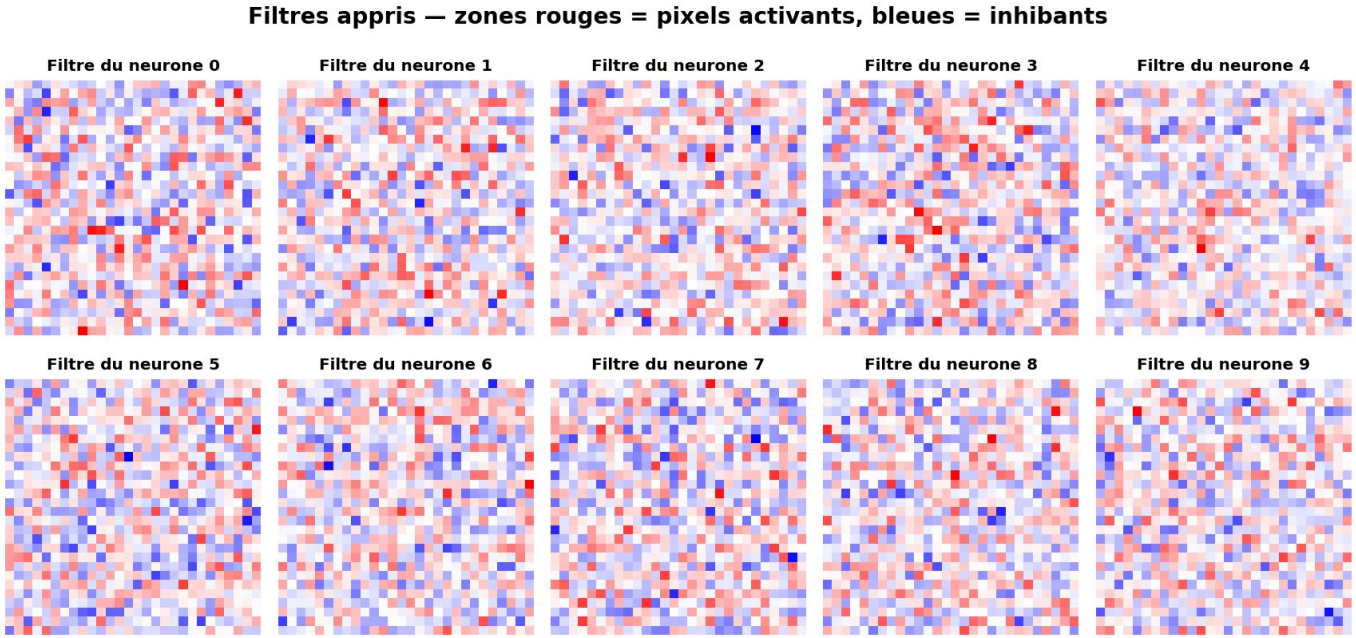
II. Couche intermédiaire –

Précision obtenue par mini batch 16 et l'activation softmax

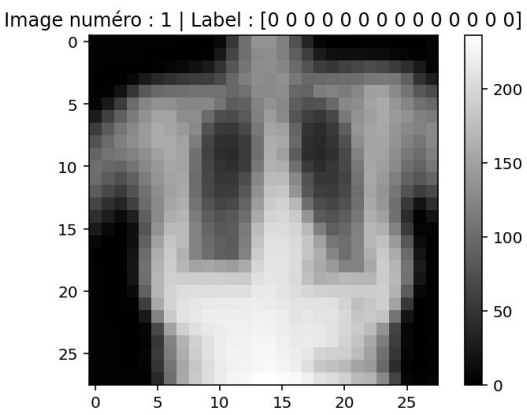
	epoch = 10	epoch = 15
lr=0.05, hs=64	92.14 %	92.84 %
lr=0.05, hs=128	92.93 %	93.71 %
lr=0.01, hs=64	87.03 %	88.54 %
lr=0.01, hs=128	87.72 %	89.35 %

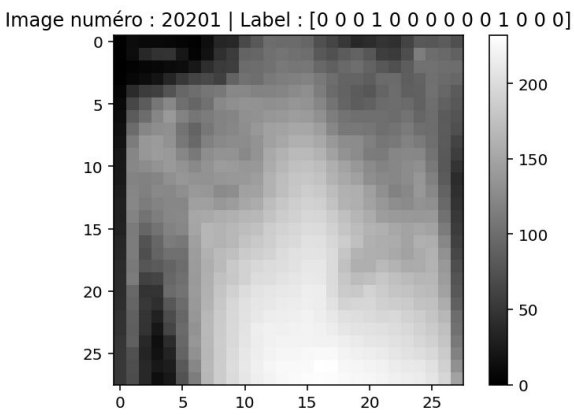
II. Couche intermédiaire –

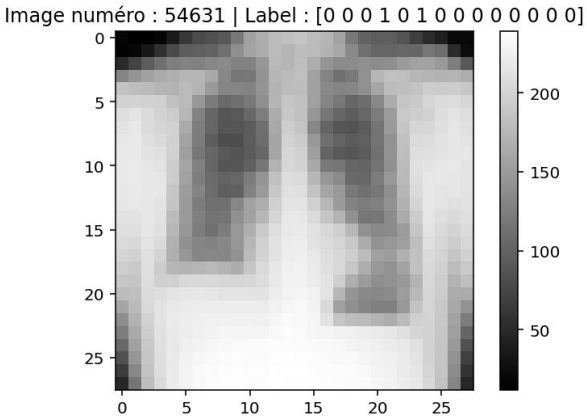
Exemple de filtres de la couche intermédiaires (lr=0.05, hs=128,softmax)



II. Couche intermédiaire -








```
In [8]: %runfile 'C:/Users/23mat/OneDrive/Bureau/TIPE/CONCOURS TIPE/Programmes/NO_CONVO_med/test_med-non-convo.py' --wdir
✅ Données flattenées et normalisées prêtes
Nom du fichier des poids 1 (sans le txt) : med-non-convo-weights1_10_sigmoide_0.05_mini_batch_32CC64
Nom du fichier des bias 1 (sans le txt) : med-non-convo-bias1_10_sigmoide_0.05_mini_batch_32CC64
Nom du fichier des poids 2 (sans le txt) : med-non-convo-weights2_10_sigmoide_0.05_mini_batch_32CC64
Nom du fichier des bias 2 (sans le txt) : med-non-convo-bias2_10_sigmoide_0.05_mini_batch_32CC64
Précision globale : 94.73893689780999 %
```

II. Couche intermédiaire -

```
✅ Importation réussie
Nom du fichier des poids 1 (sans le txt) : weights1_10_softmax_0.05_
Nom du fichier des bias 1 (sans le txt) : bias1_10_softmax_0.05_mini
Nom du fichier des poids 2 (sans le txt) : weights2_10_softmax_0.05_
Nom du fichier des bias 2 (sans le txt) : bias2_10_softmax_0.05_mini
L'algorithme est précis à la hauteur de : 74.67948717948718 %
```



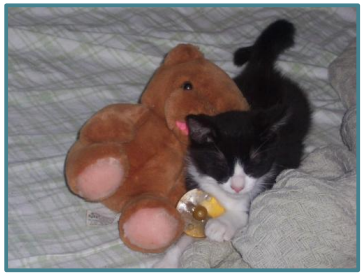


```
✅ 4998 images test chargées
Nom du fichier des poids 1 (sans le txt) : weights1_20_softmax_0.001_
Nom du fichier des bias 1 (sans le txt) : bias1_20_softmax_0.001_mini
Nom du fichier des poids 2 (sans le txt) : weights2_20_softmax_0.001_
Nom du fichier des bias 2 (sans le txt) : bias2_20_softmax_0.001_mini
L'algorithme est précis à la hauteur de : 36.014405762304925 %
```







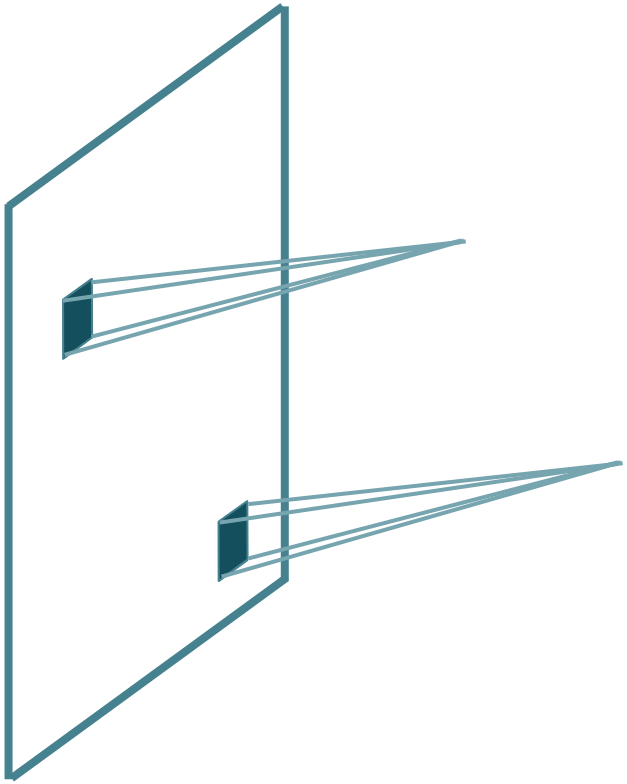


03

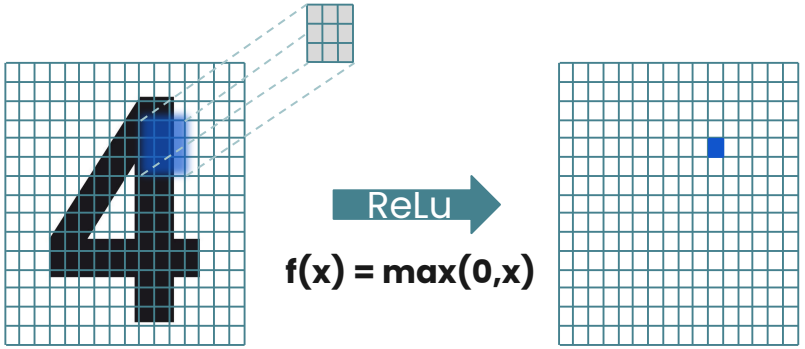
Convolution -

Bilan

III. Convolution –



F_1	F_2	F_3
F_4	F_5	F_6
F_7	F_8	F_9



III. Convolution -

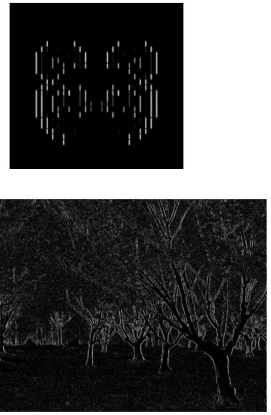
0.25	0.5	0.25
0	0	0
-0.25	-0.5	-0.25



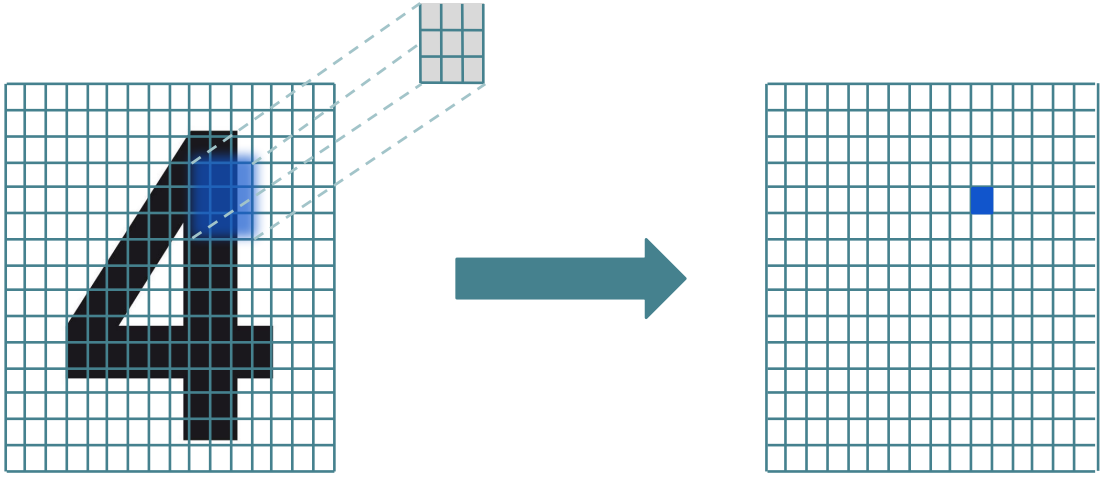


0.25	0	-0.25
0.5	0	-0.5
0.25	0	-0.25





III. Convolution –

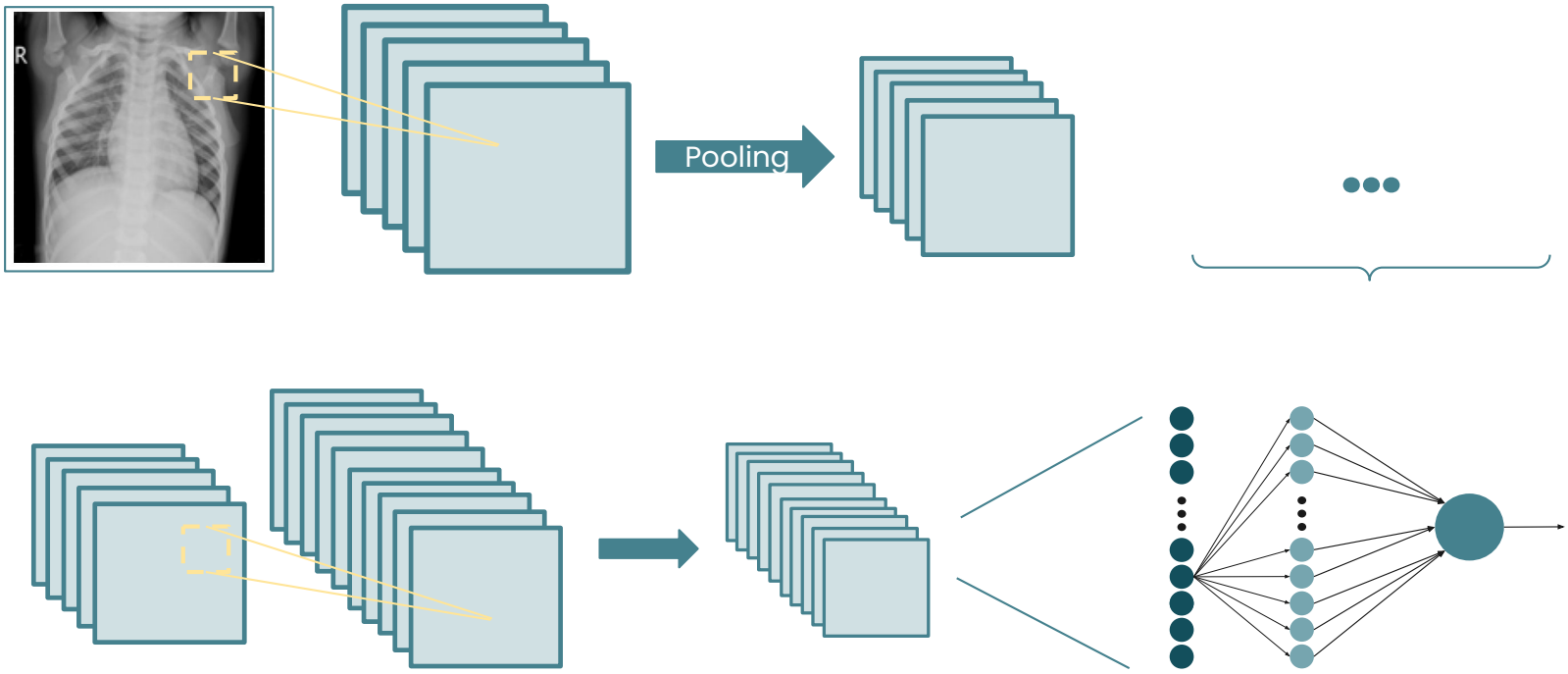


64	12	76	16
63	47	79	11
23	03	07	15
24	08	10	99

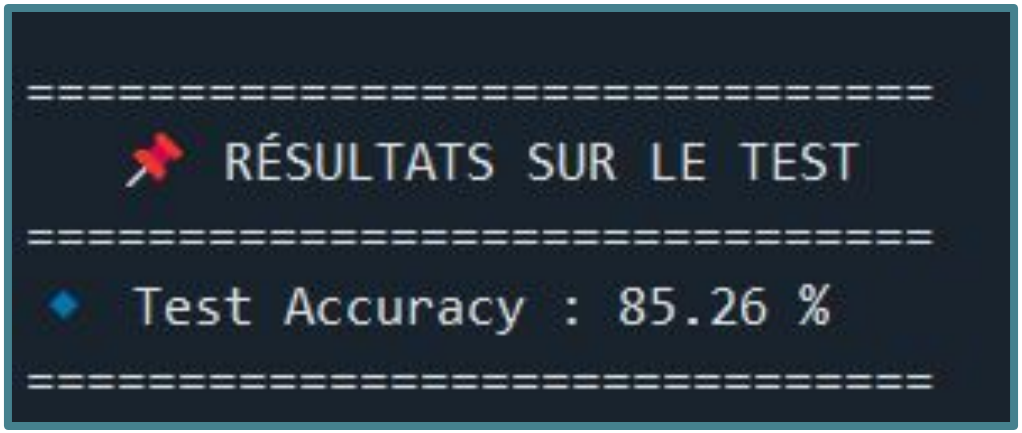


64	79
24	99

III. Convolution –



III. Convolution -



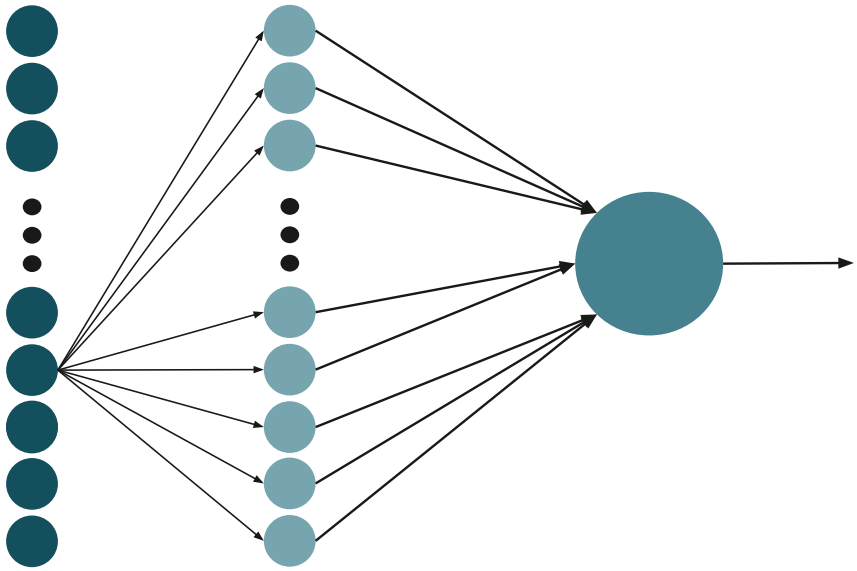
CONCLUSION –











Annexe

Les coulisses

Programme 1 (réseau linéaire sur Malade/sain)

```python
import numpy as np
import matplotlib.pyplot as plt

# ------------ Implémentation ------------


def dataset():
    """
        Définissons notre dataset composé de deux classes:
            -Malade 1
            -Sain 0
    """

    #Nombre de personnes dans une classe
    row_per_class = 100

    #Generate rows
    sick = np.random.randn(row_per_class,2) + np.array([-2,-2]) #Les personnes malades ont des taux faibles
    healthy = np.random.randn(row_per_class,2) + np.array([2,2]) #Les personnes en bonne santé ont des taux élevés

    features = np.vstack([sick, healthy]) #On met la matrice de l'une en dessous de l'autre, shape (10,2)
    tag = np.concatenate((np.ones(row_per_class), np.zeros(row_per_class))) #On étiquettes nos données/features

    return features, tag


def dataset_xor():
    """
        Dataset non linéairement séparable :
            - Sain (0) : globules rouges hauts / blancs bas OU rouges bas / blancs hauts
            - Malade (1) : globules rouges et blancs tous deux hauts OU tous deux bas
    """

    row_per_quadrant = 50

    # Groupe 1 : Malade (taux faibles)
    sick_low = np.random.randn(row_per_quadrant, 2) + np.array([-2, -2])

    # Groupe 2 : Malade (taux élevés)
    sick_high = np.random.randn(row_per_quadrant, 2) + np.array([2, 2])

    # Groupe 3 : Sain (rouges hauts / blancs bas)
    healthy_rb = np.random.randn(row_per_quadrant, 2) + np.array([2, -2])

    # Groupe 4 : Sain (rouges bas / blancs hauts)
    healthy_br = np.random.randn(row_per_quadrant, 2) + np.array([-2, 2])

    # Fusion des données
    features = np.vstack([sick_low, sick_high, healthy_rb, healthy_br])
    tag = np.concatenate((
        np.ones(2 * row_per_quadrant),      # Malades = 1
        np.zeros(2 * row_per_quadrant)      # Sains = 0
    ))

    return features, tag


def affichage_globule_blanc_rouge (features,tag):
    """
         schéma du dataset
    """
    plt.figure()

    for i, elt in enumerate(features):
        if tag[i] ==1:
            plt.plot(elt[0],elt[1],'rx')
        else:
            plt.plot(elt[0],elt[1],'gx')
    plt.xlabel('Quantité de Globules rouges')
    plt.ylabel('Quantité de Globules blancs')
    plt.legend()

def visualiser_descente(features, tag, w_hist, b_hist):
    """
    Visualise la surface de la fonction de coût et la descente du gradient.
    """
    # Trajectoire des poids
    traj_w1 = [w[0] for w in w_hist]
    traj_w2 = [w[1] for w in w_hist]

    # Plage de valeurs autour de la trajectoire
    pad = 0.5
    w1_min, w1_max = min(traj_w1)-pad, max(traj_w1)+pad
    w2_min, w2_max = min(traj_w2)-pad, max(traj_w2)+pad
    w1_vals = np.linspace(w1_min, w1_max, 50)
    w2_vals = np.linspace(w2_min, w2_max, 50)
    W1, W2 = np.meshgrid(w1_vals, w2_vals)
```

Programme 1 (réseau linéaire sur Malade/sain)

```
    # On fixe le biais au biais initial (coupe 3D)
    b_fix = b_hist[0]
    Z = np.zeros_like(W1)

    # Calcul du coût pour chaque (w1, w2)
    for i in range(W1.shape[0]):
        for j in range(W1.shape[1]):
            weights_tmp = np.array([W1[i, j], W2[i, j]])
            Z[i, j] = erreur(activation(pre_activation(features, weights_tmp, b_fix)),
tag)

    # Calcul du coût réel à chaque étape (pour la trajectoire)
    traj_z = [
        erreur(activation(pre_activation(features, np.array([w[0], w[1]]), b)), tag)
        for w, b in zip(w_hist, b_hist)
    ]

    # --- Affichage ---
    fig = plt.figure(figsize=(10,7))
    ax = fig.add_subplot(111, projection='3d')
    ax.plot_surface(W1, W2, Z, cmap='viridis', alpha=0.8)

    # Trajectoire
    ax.plot(traj_w1, traj_w2, traj_z, 'r-', linewidth=3, label='Descente réelle')
    ax.scatter(traj_w1[0], traj_w2[0], traj_z[0], c='g', s=80, label='Départ (init)')
    ax.scatter(traj_w1[-1], traj_w2[-1], traj_z[-1], c='r', s=80, label='Arrivée')

    # Réglages
    ax.set_xlabel('w₁')
    ax.set_ylabel('w₂')
    ax.set_zlabel('Erreur')
    ax.set_zlim(0, 1)
    ax.set_title("Descente du gradient sur la surface de coût")
    ax.legend()
    plt.show()


def affichage_fontière(features, weights, bias):

    if weights is not None and bias is not None:
        x_vals = np.linspace(features[:, 0].min() - 1, features[:, 0].max() + 1, 200)
        y_vals = -(weights[0] / weights[1]) * x_vals - bias / weights[1]
        plt.plot(x_vals, y_vals, 'k-', label='Frontière de décision')
        plt.legend()

def init_variables():
    """
        On initialise le model de variables (weights, bias)
    """
    weights = np.random.normal(size=2)
    bias = 0
    return weights, bias

def pre_activation(features,weights,bias):
    return np.dot(features,weights)+bias

def activation(z):
    """
        Définition de la fonction sigmoïde
    """

    return 1/(1+np.exp(-z))

def predict(features,weights, bias):
    """
        Calcul de la probabilité d'appartenir à une classe ou une autre
    """
    z = pre_activation(features, weights, bias)
    a = activation(z)
    return(np.round(a))

def erreur(predictions, tag):
    epsilon = 1e-15  # petite constante pour éviter log(0)
    LL = - (1/len(tag)) *
np.sum(tag*np.log(predictions+epsilon)+(1-tag)*np.log(1-predictions+epsilon))
    return LL

def train(features,tag,weights,bias):
    """
        Descente du gradient
    """
    epochs = 100
    learning_rate = 0.1
    pred = [] #évolution du diagnostique pour chaque patient
    cout = [] #évolution de l'erreur
    just_diag = [] #justesse du diagnostique en %
    w = [] #poids sauvegardés
    b = [] #bias sauvegardés

    for epoch in range(epochs):

        w.append(weights)
        b.append(bias)

        predictions = predict(features, weights, bias)
        just_diag.append(np.mean(predictions==tag))
        pred.append(activation(pre_activation(features, weights, bias)))
        cout.append(erreur(activation(pre_activation(features, weights, bias)),tag))
```

Programme 1 (réseau linéaire sur Malade/sain)

```
 # Init gradients

        weights_gradient = np.zeros(weights.shape)
        bias_gradient = 0

        for feature, target in zip(features,tag):
            z = pre_activation(feature, weights, bias)
            a = activation(z)
            weights_gradient += (a - target) * feature
            bias_gradient += (a - target)

        weights_gradient = weights_gradient * (1/len(tag))
        bias_gradient = bias_gradient * (1/len(tag))

        weights = weights - learning_rate*weights_gradient
        bias = bias - learning_rate*bias_gradient


    return cout, pred, just_diag, w, b


# ------------ Entrainement ------------


# Initialistation de l'algorithme

features, tag = dataset_xor()
weights, bias = init_variables()

# Récupération des données après algorithme

cout, pred, just_diag, w, b = train(features, tag, weights, bias)


# ------------ Interface graphique ------------


affichage_globule_blanc_rouge(features, tag)
plt.xlim(-5,5)
plt.ylim(-6,6)

affichage_globule_blanc_rouge(features, tag)
affichage_fontière(features, weights, bias)
plt.title("Situation à l'initialisation")
plt.xlim(-5,5)
plt.ylim(-6,6)
plt.legend()

affichage_globule_blanc_rouge(features, tag)
affichage_fontière(features, w[49], b[49])
plt.title("Situation à epoch=50")
plt.xlim(-5,5)
plt.ylim(-6,6)
plt.legend()

affichage_globule_blanc_rouge(features, tag)
affichage_fontière(features, w[99], b[99])
plt.title("Situation finale")
plt.xlim(-5,5)
plt.ylim(-6,6)
plt.legend()

epoc = [i + 1 for i in range(100)]
plt.figure()
plt.plot(epoc,cout,'-')
plt.plot(0,cout[0],'rx', label='erreur epoch 1 = ' + str(cout[0]))
plt.plot(9,cout[9],'gx', label='erreur epoch 10 = ' + str(cout[9]))
plt.plot(49,cout[49],'yx', label='erreur epoch 50 = ' + str(cout[49]))
plt.plot(99,cout[99],'mx', label='erreur epoch 100 = ' + str(cout[99]))
plt.title("évolution de l'erreur")
plt.xlabel('epoch')
plt.ylabel('erreur')
plt.legend()
plt.show()

plt.figure()
plt.plot(epoc,pred,'-')
plt.title("évolution du diagnostique pour chaque patient")
plt.xlabel('epoch')
plt.ylabel('%chance malade/sain')
plt.legend()
plt.show()

plt.figure()
plt.plot(epoc,just_diag,'-')
plt.plot(0,just_diag[0],'rx', label='justesse epoch 1 = ' + str(just_diag[0]))
plt.plot(9,just_diag[9],'gx', label='justesse epoch 10 = ' + str(just_diag[9]))
plt.plot(49,just_diag[49],'yx', label='justesse epoch 50 = ' + str(just_diag[49]))
plt.plot(99,just_diag[99],'mx', label='justesse epoch 100 = ' + str(just_diag[99]))
plt.title("évolution de la justesse du diagnostique")
plt.xlabel('epoch')
plt.ylabel('précision (%)')
plt.legend()
plt.show()

visualiser_descente(features, tag, w, b)
```

Programme 1 (réseau linéaire sur Malade/sain)

```python
# ------------ Test des poids obtenues ------------

nb = 100  # nombre de tests
precisions = []

for i in range(nb):
    features_test, tag_test = dataset_xor()
    predictions = predict(features_test, w[-1], b[-1])  # utiliser les poids finaux
    precision = np.mean(predictions == tag_test)
    precisions.append(precision)

print("L'algorithme est fiable à " + str(np.mean(precisions)*100) + "%")
```

Programme 2 (réseau linéaire sur MNIST)

```python
# -*- coding: utf-8 -*-

import numpy as np
import matplotlib.pyplot as plt

# ------------ Paramètres ------------

epochs = 15
learning_rate = 0.01

#choix de la méthode utilisée selon stochastic, full_batch, mini_batch pour la descente

methode = "full_batch"
batch = 128 # généralement batch = 16, 32 ou 128

#choix de la fonction d'activation parmis sigmoide et softmax
fonction_activation = "softmax"

# ------------ Vérification des paramètres ------------

assert type(epochs) == int, "epochs doit être un entier"
assert type(learning_rate) == float or type(learning_rate) == int, "le learning rate doit être un nombre"
assert methode in ["stochastic","full_batch","mini_batch"], "erreur sur la methode"
assert type(batch) == int, "batch doit être un entier"
assert fonction_activation in ["sigmoide","softmax"]

print("✅ Aucun problème pour les entrées")


# ------------ Téléchargement des données  ------------

from tensorflow.keras.datasets import mnist
(I_train_data, Y_train_data), (I_test_data, Y_test_data) = mnist.load_data()  #I_train_data sont les matrices des nombres et Y_train_data les numéro


# ------------ Affichage de nombres pour l'exemple ------------

def affichage_MNIST(nb):
    """
        affiche l'image de l'item numéro nb de la banque d'image MNIST
    """
    plt.imshow(I_train_data[nb], cmap='gray')
    plt.colorbar()  #ajoute une échelle de couleurs
    plt.title('Image numéro : '+ str(nb))
    plt.show()
```

```python
# ------------ Vectorisation ------------

#On transforme les matrices de MNIST en lignes
Data_set = []

for i in range(len(I_train_data)):
    N_image = []
    N_sortie = [0 for i in range(10)]
    for j in range(28):
        for k in range(28):
            N_image.append((I_train_data[i][j][k])/255) #on divise par 255 pour normalisée

    N_sortie[Y_train_data[i]] = 1

    Data_set.append((np.array(N_image), np.array(N_sortie)))

print("✅ Vectorisation effectuée")


# ------------ Implémentation ------------

def initialisation():
    """
        On initialise le model de variables (weights, bias)
    """
    W = np.random.randn(784, 10)
    bias = np.zeros(10)
    return W, bias

def pre_activation(I,W,bias):
    return I.dot(W)+bias

def activation(Z, fonction_activation):
    """
        Définition de la fonction sigmoïde
    """

    if fonction_activation == "sigmoide":

        #----------- Sigmoïde -----------

        #La fonction np.clip permet de limiter les valeurs prisent dans l'exp
        #ainsi on évite l'erreur d'overflow.
        #Par ailleurs, cela ne change rien car sigmoïde aurait renvoyé 0 ou 1

        Z = np.clip(Z, -500, 500)
        return 1 / (1 + np.exp(-Z))

    elif fonction_activation == "softmax":
```

Programme 2 (réseau linéaire sur MNIST)

```
        #---------- Softmax ----------

        exp_Z = np.exp(Z - np.max(Z))  # pour stabilité numérique et éviter l'overflow
(astuce de calucul)
        return exp_Z / np.sum(exp_Z)

def model(I,W,bias,fonction_activation):
    Z = pre_activation(I, W, bias)
    return activation(Z,fonction_activation)

def Loss(A, S, fonction_activation):
    """
        Définition de la Log_Loss
    """
    epsilon = 1e-15  # petite constante pour éviter log(0)

    if fonction_activation == "sigmoide":

        # Cross-entropy binaire
        return np.mean(- (1/len(S)) *
np.sum(S*np.log(A+epsilon)+(1-S)*np.log(1-A+epsilon)))

    elif fonction_activation == "softmax":
        return -np.sum(S * np.log(A))  # produit de Hadamard

def gradient(W, I, S, bias, fonction_activation):

    A = model(I, W, bias, fonction_activation)

    dZ = A - S
    dW = np.outer(I, dZ)  # ecriture python de I^T.dZ
    db = dZ
    return dW, db

def update(dW, W, bias, db, learning_rate):
    W = W - learning_rate * dW
    bias = bias - learning_rate * db
    return W , bias

def train(features,tag,W,bias,epochs,learning_rate,methode,batch,fonction_activation):
    """
        descente du gradient
    """
    n = len(features)
    evo_cout = [] #évolution de l'erreur

    if methode == "full_batch":

        for epoch in range(epochs):

            print("Epoch ", epoch, "/", epochs, " starting -" )
            cout = []

            weights_gradient = np.zeros(W.shape)
            bias_gradient = np.zeros(bias.shape)

            for feature, target in zip(features,tag):
                A = model(feature, W, bias,fonction_activation)
                cout.append(Loss(A,target,fonction_activation))
                dW , db = gradient(W,feature,target,bias,fonction_activation)
                weights_gradient += dW
                bias_gradient += db

            W , bias = update(weights_gradient/n, W, bias, bias_gradient/n, learning_rate)

            evo_cout.append(np.mean(cout))

        return W , bias , evo_cout

    elif methode == "stochastic":

        for epoch in range(epochs):

            print("Epoch ", epoch, "/", epochs, " starting -" )
            cout = []

            # On mélange les données pour ne pas faire systématiquement la même descente
            indices = np.random.permutation(n)
            features = features[indices]
            tag = tag[indices]

            for feature, target in zip(features,tag):
                A = model(feature, W, bias,fonction_activation)
                evo_cout.append(Loss(A,target,fonction_activation))
                dW , db = gradient(W,feature,target,bias,fonction_activation)
                W , bias = update(dW, W, bias, db, learning_rate)

        return W , bias , evo_cout

    elif methode == "mini_batch":

        for epoch in range(epochs):

            print("Epoch ", epoch, "/", epochs, " starting -" )
            cout = []

            # On mélange les données pour ne pas faire systématiquement la même descent
            indices = np.random.permutation(n)
            features = features[indices]
```

Programme 2 (réseau linéaire sur MNIST)

```
            tag = tag[indices]


            for i in range(0, n, batch):
                batch_features = features[i:i + batch]
                batch_tags = tag[i:i + batch]

                weights_gradient = np.zeros(W.shape)
                bias_gradient = np.zeros(bias.shape)

                for feature, target in zip(batch_features,batch_tags):
                    A = model(feature, W, bias,fonction_activation)
                    cout.append(Loss(A,target,fonction_activation))
                    dW , db = gradient(W,feature,target,bias,fonction_activation)
                    weights_gradient += dW
                    bias_gradient += db

                W , bias = update(weights_gradient / len(batch_features), W, bias,
bias_gradient / len(batch_tags), learning_rate)

                evo_cout.append(np.mean(cout))

        return W , bias , evo_cout

# ------------ Entrainement ------------

#Initialisation de l'algorithme :

features = np.array([x[0] for x in Data_set])
tag = np.array([x[1] for x in Data_set])

W , bias = initialisation()

# Récupération des données après algorithme

W , bias , evo_cout = train(features, tag, W, bias, epochs, learning_rate, methode, batch,
fonction_activation)

np.savetxt("weights_" + str(epochs) + "_" + fonction_activation + "_" + str(learning_rate)
+ "_" + methode + "_" + str(batch) + ".txt", W)
np.savetxt("bias_" + str(epochs) + "_" + fonction_activation + "_" + str(learning_rate) +
"_" + methode + "_" + str(batch) + ".txt", bias)

print("✅ Poids et bias sauvegardés")


# ------------ Interface graphique ------------

def courbe_de_erreur(methode,fonction_activation, learning_rate, batch):

    if methode == "full_batch":

        info = "learning rate = " + str(learning_rate) + "\n" + \
       "méthode : " + methode + "\n" + \
       "fonction d'activation : " + fonction_activation

        plt.plot(range(len(evo_cout)), evo_cout, label=info)
        plt.title("Evolution de l'erreur")
        plt.xlabel("n ième epoch")
        plt.legend()
        plt.show()

    elif methode == "mini_batch":

        info = "learning rate = " + str(learning_rate) + "\n" + \
            "méthode : " + methode + "\n" + \
            "batch = " + str(batch) + "\n" + \
            "fonction d'activation : " + fonction_activation

        plt.plot(range(len(evo_cout)), evo_cout, label=info)
        plt.title("Evolution de l'erreur")
        plt.xlabel("n ième batch")
        plt.legend()
        plt.show()

        plt.figure()
        window_size = 100
        smooth_error = np.convolve(evo_cout, np.ones(window_size)/window_size, mode='valid')

        plt.plot(range(len(smooth_error)), smooth_error)
        plt.xlabel("Itération")
        plt.ylabel("Erreur moyenne")
        plt.title("Erreur lissée sur "+str(window_size)+" bachs")
        plt.show()

    elif methode == "stochastic":

        info = "learning rate = " + str(learning_rate) + "\n" + \
            "méthode : " + methode + "\n" + \
            "fonction d'activation : " + fonction_activation

        plt.plot(range(len(evo_cout)), evo_cout, label=info)
        plt.title("Evolution de l'erreur")
        plt.xlabel("n ième itération")
        plt.legend()
        plt.show()

        plt.figure()
        window_size = 1000
        smooth_error = np.convolve(evo_cout, np.ones(window_size)/window_size, mode='valid')

        plt.plot(range(len(smooth_error)), smooth_error)
        plt.xlabel("Itération")
        plt.ylabel("Erreur moyenne")
        plt.title("Erreur lissée sur "+str(window_size)+" itérations")
        plt.show()

courbe_de_erreur(methode, fonction_activation, learning_rate, batch)
```

Programme 2 (module de test)

```python
# -*- coding: utf-8 -*-

from PIL import Image
import numpy as np
import matplotlib.pyplot as plt
from tensorflow.keras.datasets import mnist

# ------------ Téléchargement des données ------------

(I_train_data, Y_train_data), (I_test_data, Y_test_data) = mnist.load_data()

Data_set = []

for i in range(len(I_test_data)):
    N_image = []
    for j in range(28):
        for k in range(28):
            N_image.append((I_test_data[i][j][k])/255)

    Data_set.append(np.array(N_image))

# ------------ Chargement des poids et bias ------------

nompoids = input('Nom du fichier des poids (sans le txt) : ')
W = np.loadtxt(nompoids + '.txt')  # W.shape = (784, 10)

nombias = input('Nom du fichier des bias (sans le txt) : ')
bias = np.loadtxt(nombias + '.txt')

# ------------ Fonction pour afficher un filtre ------------

def afficher_filtre(poids, taille=(28, 28), titre=""):
    poids = np.array(poids).reshape(taille)
    max_abs = np.max(np.abs(poids))
    if max_abs == 0:
        max_abs = 1
    poids_norm = poids / max_abs

    img = np.zeros((*taille, 3))
    img[:, :, 0] = np.clip(poids_norm, 0, 1)   # Rouge
    img[:, :, 2] = np.clip(-poids_norm, 0, 1)  # Bleu
    img[np.isclose(poids_norm, 0)] = [1, 1, 1]  # Blanc

    plt.imshow(img)
    plt.title(titre)
    plt.axis('off')

# ------------ Affichage de tous les filtres ------------

plt.figure(figsize=(12, 6))
for i in range(10):
    plt.subplot(2, 5, i + 1)
    afficher_filtre(W[:, i], (28, 28), titre=f"Classe {i}")
plt.suptitle("Visualisation des filtres appris", fontsize=14)
plt.tight_layout()
plt.show()
```

```python
def afficher_filtre_didactique(filtre, shape=(28,28), titre=None):
    """
    Affiche un filtre (vecteur de poids) sous forme d'image :
    - rouge vif : active la classe
    - bleu foncé : inhibe la classe
    - blanc : neutre
    """
    filtre_2D = filtre.reshape(shape)
    vabs = np.max(np.abs(filtre_2D))

    plt.imshow(filtre_2D, cmap='bwr', vmin=-vabs, vmax=vabs)
    plt.axis("off")
    if titre:
        plt.title(titre, fontsize=10, fontweight="bold")

def visualiser_filtres_didactique(W, shape=(28,28)):
    """
    Affiche les 10 filtres appris (un par chiffre)
    """
    plt.figure(figsize=(12, 6))
    for i in range(10):
        plt.subplot(2, 5, i + 1)
        afficher_filtre_didactique(W[:, i], shape, titre=f"Classe {i}")
    plt.suptitle(
        "Filtres appris -- zones rouges = pixels activants, bleues = inhibants",
        fontsize=14, fontweight="bold"
    )
    plt.tight_layout()
    plt.show()


visualiser_filtres_didactique(W)


# ------------ test image en particulier ------------

def calque(I):

    C = I.dot(W)+bias

    nb = 0
    for j in range (len(C)):
        if C[j]>C[nb]:
            nb = j

    return nb

def bonformat (img):
    """
        Entrée : Image

        Objectif : Mettre l'image dans un format adaptée aux tests

        Réalise : -Redimensionne l'image
                  -La convertit en un tableau avec des entiers compris entre 0 et 255

        Sortie : Tableau
    """
```

Programme 2 (module de test)

```
    img = redim(img, 28)
    img = convertgris(img)

    return (img)

def redim(img, f):
    """
        Entrée : img = Image et f = entier positif

        Objectif : Redimensionner une image dans le format "fxf"

        Sortie : Image
    """

    img_resize = img.resize((f, f))
    return (img_resize)

def convertgris (img):
    """
        Entrée : Image

        Objectif : Permet d'obtenir un tableau contenant le niveau de gris des pixels.

        Info : Le tableau renvoie ligne par ligne

        Sortie : Tableau
    """

    # Convertit en tableau NumPy (avec des nombres compris entre 0 et 255)
    #(0 correspond à noir, 255 à blanc):
    image = img.convert("L")


    #cette fonction sert à pouvoir ajuster les entiers dans la matrice
    #et pas uniquement avoir des entiers entre 0 et 255
    array = np.array(image).astype(int)

    return (array)

def ajustpixels(array):
    """
        Entrée : Tableau

        Objectif : Changer les valeurs des pixels

        Realise : -Les nombres proches de 255 correspondant au blanc deviennent proche de 0
                  -Les nombres proches de 0 correspondant au noir deviennent proche de 1

        Sortie : Tableau
    """

    return (255 - array)/255

def test():
    nom2 = input("nom de l'image:") + "." + input("format de l'image:")
    try:
        inter = Image.open(nom2)
        inter = list(ajustpixels(bonformat(inter)))
        img = []
        for l in range(28):
            for c in range(28):
                img.append(inter[l][c])
        print(calque(np.array(img)))

    except:
        print("/!\ Erreur : Le nom ou le format de l'image est incorrect")


# ------------ Efficacité ------------

Nb_Erreur = 0

for i in range(len(I_test_data)):

    if calque(Data_set[i]) != Y_test_data[i]:
        Nb_Erreur+=1

print("L'algorithme est précis à la hauteur de :", 100*(len(I_test_data)-Nb_Erreur)/len(I_test_data),"%")
```

Programme 3 (réseau linéaire sur Chest-Xray)

```python
# -*- coding: utf-8 -*-

import numpy as np
import os
import glob
from PIL import Image
import matplotlib.pyplot as plt

# ------------ Paramètres ------------

nbpix = 28

epochs = 10
learning_rate = 0.05

#choix de la méthode utilisée selon stochastic, full_batch, mini_batch pour la descente

methode = "mini_batch"
batch = 16 # généralement batch = 16, 32 ou 128

#choix de la fonction d'activation parmis sigmoide et softmax
fonction_activation = "softmax"

# ------------ Vérification des paramètres ------------

assert type(epochs) == int, "epochs doit être un entier"
assert type(learning_rate) == float or type(learning_rate) == int, "le learning rate doit être un nombre"
assert methode in ["stochastic","full_batch","mini_batch"], "erreur sur la methode"
assert type(batch) == int, "batch doit être un entier"
assert fonction_activation in ["sigmoide","softmax"]

print("✅ Aucun problème pour les entrées")


# ------------ Téléchargement des données  ------------

train_dir = r"C:\Users\23mat\.kaggle\chest_xray_data\archive\chest_xray\train" #chemin du dossier
categories = ["NORMAL", "PNEUMONIA"]

features = []
targets = []

for label, category in enumerate(categories):

    path = os.path.join(train_dir, category, "*.jpeg")  # ouvre toutes les images .jpeg de la category

    for file in glob.glob(path):
        try:
            img = Image.open(file).convert('L')  # converti en niveaux de gris
            img = img.resize((nbpix,nbpix))      # redimensionne
            features.append(np.array(img).flatten())  # flatten indispensable (28x28 -> 784)
            if label == 0:
                targets.append(np.array([1,0])) #cas NORMAL
            else:
                targets.append(np.array([0,1])) #cas PNEUMONIA

        except:
            print("""/!\\ Problème dans l'importation des image /!\\""")


features_ = np.array(features) / 255.0 # Normalisation
targets_ = np.array(targets)

indices = np.random.permutation(len(features_)) # On mélange
features_ = features_[indices]
targets_ = targets_[indices]

Data_set = [(x,y) for x,y in zip (features_,targets_)]
print("✅ Téléchargement effectué")


# ------------ Implémentation ------------

def initialisation():
    """
        On initialise le model de variables (weights, bias)
    """
    W = np.random.randn(784, 2)
    bias = np.zeros(2)
    return W, bias

def pre_activation(I,W,bias):
    return I.dot(W)+bias

def activation(Z, fonction_activation):
    """
        Définition de la fonction sigmoïde ou softmax
    """

    if fonction_activation == "sigmoide":

        #----------- Sigmoïde -----------

        #La fonction np.clip permet de limiter les valeurs prisent dans l'exp
        #ainsi on évite l'erreur d'overflow.
        #Par ailleurs, cela ne change rien car sigmoïde aurait renvoyé 0 ou 1

        Z = np.clip(Z, -500, 500)
        return 1 / (1 + np.exp(-Z))

    elif fonction_activation == "softmax":

        #----------- Softmax -----------

        exp_Z = np.exp(Z - np.max(Z))  # stabilité numérique
        return exp_Z / np.sum(exp_Z)

def model(I,W,bias,fonction_activation):
    Z = pre_activation(I, W, bias)
    return activation(Z,fonction_activation)

def Loss(A, S, fonction_activation):
    """
        Définition de la Log_Loss
    """
```

Programme 3 (réseau linéaire sur Chest-Xray)

```
    epsilon = 1e-15  # petite constante pour éviter log(0)

    if fonction_activation == "sigmoide":

        # Cross-entropy binaire (forme correcte)
        return -np.sum(S*np.log(A+epsilon)+(1-S)*np.log(1-A+epsilon))

    elif fonction_activation == "softmax":
        return -np.sum(S * np.log(A + epsilon))  # cross-entropy

def gradient(W, I, S, bias, fonction_activation):

    A = model(I, W, bias, fonction_activation)

    dZ = A - S
    dW = np.outer(I, dZ)  # ecriture python de I^T.dZ
    db = dZ
    return dW, db

def update(dW, W, bias, db, learning_rate):
    W = W - learning_rate * dW
    bias = bias - learning_rate * db
    return W , bias

def train(features,tag,W,bias,epochs,learning_rate,methode,batch,fonction_activation):
    """
        descente du gradient
    """
    n = len(features)
    evo_cout = [] #évolution de l'erreur

    if methode == "full_batch":

        for epoch in range(epochs):

            print("Epoch ", epoch, "/", epochs, " starting -" )
            cout = []

            weights_gradient = np.zeros(W.shape)
            bias_gradient = np.zeros(bias.shape)

            for feature, target in zip(features,tag):
                A = model(feature, W, bias,fonction_activation)
                cout.append(Loss(A,target,fonction_activation))
                dW , db = gradient(W,feature,target,bias,fonction_activation)
                weights_gradient += dW
                bias_gradient += db

            W , bias = update(weights_gradient/n, W, bias, bias_gradient/n, learning_rate)
            evo_cout.append(np.mean(cout))

        return W , bias , evo_cout

    elif methode == "stochastic":

        for epoch in range(epochs):

            print("Epoch ", epoch, "/", epochs, " starting -" )
            cout = []

            indices = np.random.permutation(n)
            features = features[indices]
            tag = tag[indices]

            for feature, target in zip(features,tag):
                A = model(feature, W, bias,fonction_activation)
                evo_cout.append(Loss(A,target,fonction_activation))
                dW , db = gradient(W,feature,target,bias,fonction_activation)
                W , bias = update(dW, W, bias, db, learning_rate)

        return W , bias , evo_cout

    elif methode == "mini_batch":

        for epoch in range(epochs):

            print("Epoch ", epoch, "/", epochs, " starting -" )
            cout = []

            indices = np.random.permutation(n)
            features = features[indices]
            tag = tag[indices]

            for i in range(0, n, batch):
                batch_features = features[i:i + batch]
                batch_tags = tag[i:i + batch]

                weights_gradient = np.zeros(W.shape)
                bias_gradient = np.zeros(bias.shape)

                for feature, target in zip(batch_features,batch_tags):
                    A = model(feature, W, bias,fonction_activation)
                    cout.append(Loss(A,target,fonction_activation))
                    dW , db = gradient(W,feature,target,bias,fonction_activation)
                    weights_gradient += dW
                    bias_gradient += db

                W , bias = update(weights_gradient / len(batch_features), W, bias, bias_gradient / len(batch
                evo_cout.append(np.mean(cout))

        return W , bias , evo_cout

# ------------ Entrainement ------------

features = np.array([x[0] for x in Data_set])
tag = np.array([x[1] for x in Data_set])

W , bias = initialisation()

W , bias , evo_cout = train(features, tag, W, bias, epochs, learning_rate, methode, batch, fonction_activati

np.savetxt("weights_" + str(epochs) + "_" + fonction_activation + "_" + str(learning_rate) + "_" + methode +
".txt", W)
np.savetxt("bias_" + str(epochs) + "_" + fonction_activation + "_" + str(learning_rate) + "_" + methode + "_
bias)

print("✅ Poids et bias sauvegardés")
```

Programme 3 (réseau linéaire sur Chest-Xray)

```
# ------------ Interface graphique ------------

def augment_np(img_np):
    """
        Augmentation simple et compatible avec la vectorisation
        img_np : numpy array (nbpix*nbpix,) ou (nbpix,nbpix)
    """

    # Remise en forme si image flatten
    if img_np.ndim == 1:
        img_np = img_np.reshape(nbpix, nbpix)

    # Flip horizontal
    if np.random.rand() < 0.5:
        img_np = np.fliplr(img_np)

    # Variation de luminosité
    img_np = img_np + np.random.uniform(-0.1, 0.1)
    img_np = np.clip(img_np, 0.0, 1.0)

    # Bruit gaussien
    img_np = img_np + np.random.normal(0, 0.02, img_np.shape)
    img_np = np.clip(img_np, 0.0, 1.0)

    # Rotation légère (avec PIL)
    pil = Image.fromarray((img_np * 255).astype("uint8"))
    angle = np.random.uniform(-10, 10)
    pil = pil.rotate(angle, resample=Image.BILINEAR)

    # Toujours revenir à nbpix × nbpix
    pil = pil.resize((nbpix, nbpix))

    # Retour numpy normalisé entre 0 et 1 + flatten
    img_np = np.array(pil).astype("float32") / 255.0

    return img_np.flatten()


# ------------ Interface graphique ------------

def courbe_de_erreur(methode,fonction_activation, learning_rate, batch):

    if methode == "full_batch":

        info = "learning rate = " + str(learning_rate) + "\n" + \
       "méthode : " + methode + "\n" + \
       "fonction d'activation : " + fonction_activation

        plt.plot(range(len(evo_cout)), evo_cout, label=info)
        plt.title("Evolution de l'erreur")
        plt.xlabel("n ième epoch")
        plt.legend()
        plt.show()

    elif methode == "mini_batch":

        info = "learning rate = " + str(learning_rate) + "\n" + \
            "méthode : " + methode + "\n" + \
            "batch = " + str(batch) + "\n" + \
            "fonction d'activation : " + fonction_activation

        plt.plot(range(len(evo_cout)), evo_cout, label=info)
        plt.title("Evolution de l'erreur")
        plt.xlabel("n ième batch")
        plt.legend()
        plt.show()

        plt.figure()
        window_size = 100
        smooth_error = np.convolve(evo_cout, np.ones(window_size)/window_size, mode='valid')

        plt.plot(range(len(smooth_error)), smooth_error)
        plt.xlabel("Itération")
        plt.ylabel("Erreur moyenne")
        plt.title("Erreur lissée sur "+str(window_size)+" bachs")
        plt.show()

    elif methode == "stochastic":

        info = "learning rate = " + str(learning_rate) + "\n" + \
            "méthode : " + methode + "\n" + \
            "fonction d'activation : " + fonction_activation

        plt.plot(range(len(evo_cout)), evo_cout, label=info)
        plt.title("Evolution de l'erreur")
        plt.xlabel("n ième itération")
        plt.legend()
        plt.show()

        plt.figure()
        window_size = 1000
        smooth_error = np.convolve(evo_cout, np.ones(window_size)/window_size, mode='valid')

        plt.plot(range(len(smooth_error)), smooth_error)
        plt.xlabel("Itération")
        plt.ylabel("Erreur moyenne")
        plt.title("Erreur lissée sur "+str(window_size)+" itérations")
        plt.show()

courbe_de_erreur(methode, fonction_activation, learning_rate, batch)
```

Programme 3 (module de test)

```python
# -*- coding: utf-8 -*-

from PIL import Image
import glob
import os
import numpy as np
import matplotlib.pyplot as plt

nbpix = 28

# ------------ Téléchargement des données ------------

train_dir = r"C:\Users\23mat\.kaggle\chest_xray_data\archive\chest_xray\test" #chemin du dossier
categories = ["NORMAL", "PNEUMONIA"]

features = []
targets = []

for label, category in enumerate(categories):

    path = os.path.join(train_dir, category, "*.jpeg")  # ouvre toutes les images .jpeg de la category

    for file in glob.glob(path):
        try:
            img = Image.open(file).convert('L')  # converti en niveaux de gris
            img = img.resize((nbpix,nbpix))      # redimensionne
            features.append(np.array(img).flatten())  # flatten indispensable (28x28 -> 784)
            if label == 0:
                targets.append(np.array([1,0])) # cas NORMAL
            else:
                targets.append(np.array([0,1])) # cas PNEUMONIA

        except:
             print("""/!\\ Problème dans l'importation des image /!\\""")


features_ = np.array(features) / 255.0 # Normalisation
targets_ = np.array(targets)

indices = np.random.permutation(len(features_)) # On mélange
I_test_data = features_[indices]
Y_test_data = targets_[indices]

Data_set = [(x,y) for x,y in zip(I_test_data, Y_test_data)]

print("✅ Données de test chargées")

# ------------ Chargement des poids et bias ------------

nompoids = input('Nom du fichier des poids (sans le txt) : ')
W = np.loadtxt(nompoids + '.txt')  # W.shape = (784, 2)

nombias = input('Nom du fichier des bias (sans le txt) : ')
bias = np.loadtxt(nombias + '.txt')

nb_classes = W.shape[1]

print("✅ Poids et bias chargés")
```

```python
# ------------ Fonction pour afficher un filtre ------------

def afficher_filtre(poids, taille=(28, 28), titre=""):
    poids = np.array(poids).reshape(taille)
    max_abs = np.max(np.abs(poids))
    if max_abs == 0:
        max_abs = 1
    poids_norm = poids / max_abs

    img = np.zeros((*taille, 3))
    img[:, :, 0] = np.clip(poids_norm, 0, 1)   # Rouge
    img[:, :, 2] = np.clip(-poids_norm, 0, 1)  # Bleu
    img[np.isclose(poids_norm, 0)] = [1, 1, 1]  # Blanc

    plt.imshow(img)
    plt.title(titre)
    plt.axis('off')

# ------------ Affichage de tous les filtres ------------

plt.figure(figsize=(12, 6))
for i in range(nb_classes):
    plt.subplot(1, nb_classes, i + 1)
    afficher_filtre(W[:, i], (28, 28), titre=categories[i])
plt.suptitle("Visualisation des filtres appris", fontsize=14)
plt.tight_layout()
plt.show()


# ------------ Version didactique (bwr) ------------

def afficher_filtre_didactique(filtre, shape=(28,28), titre=None):
    """
    Affiche un filtre (vecteur de poids) sous forme d'image :
    - rouge : pixels activants
    - bleu : pixels inhibants
    """
    filtre_2D = filtre.reshape(shape)
    vabs = np.max(np.abs(filtre_2D))
    if vabs == 0:
        vabs = 1

    plt.imshow(filtre_2D, cmap='bwr', vmin=-vabs, vmax=vabs)
    plt.axis("off")
    if titre:
        plt.title(titre, fontsize=10, fontweight="bold")

def visualiser_filtres_didactique(W, shape=(28,28)):
    plt.figure(figsize=(6, 3))
    for i in range(W.shape[1]):
        plt.subplot(1, W.shape[1], i + 1)
        afficher_filtre_didactique(W[:, i], shape, titre=categories[i])
    plt.suptitle(
        "Filtres appris -- zones rouges = pixels activants, bleues = inhibants",
        fontsize=14, fontweight="bold"
    )
    plt.tight_layout()
    plt.show()
```

Programme 3 (module de test)

```
visualiser_filtres_didactique(W)

# ------------ Test / classification ------------

def calque(I):
    """
    Classification par produit scalaire + argmax
    """
    C = I.dot(W) + bias
    return np.argmax(C)


# ------------ Efficacité ------------

Nb_Erreur = 0

for I, Y in Data_set:
    if calque(I) != np.argmax(Y):
        Nb_Erreur += 1

print("L'algorithme est précis à la hauteur de :",
      100 * (len(I_test_data) - Nb_Erreur) / len(I_test_data), "%")
```

Programme 4 (réseau linéaire sur Cat vs Dog)

```python
# -*- coding: utf-8 -*-

import numpy as np
import os
import glob
from PIL import Image
import matplotlib.pyplot as plt

# ------------ Paramètres ------------

nbpix = 28

epochs = 10
learning_rate = 0.05

#choix de la méthode utilisée selon stochastic, full_batch, mini_batch pour la descente

methode = "mini_batch"
batch = 16 # généralement batch = 16, 32 ou 128

#choix de la fonction d'activation parmis sigmoide et softmax
fonction_activation = "softmax"

# ------------ Vérification des paramètres ------------

assert type(epochs) == int, "epochs doit être un entier"
assert type(learning_rate) == float or type(learning_rate) == int, "le learning rate doit être un nombre"
assert methode in ["stochastic","full_batch","mini_batch"], "erreur sur la methode"
assert type(batch) == int, "batch doit être un entier"
assert fonction_activation in ["sigmoide","softmax"]

print("✅ Aucun problème pour les entrées")


# ------------ Téléchargement des données  ------------

train_dir = r"C:\Users\23mat\OneDrive\Bureau\TIPE\CONCOURS TIPE\data_set_dog_cat\archive\PetImages"
#chemin du dossier
categories = ["Cat", "Dog"]

features = []
targets = []

for label, category in enumerate(categories):

    path = os.path.join(train_dir, category, "*.jpg")  # ouvre toutes les images .jpeg de la category

    for file in glob.glob(path):
        try:
            img = Image.open(file).resize((nbpix,nbpix))
            img = img.convert('L')
            features.append(np.array(img).flatten())  # flatten indispensable (28x28 -> 784)
            if label == 0:
                targets.append(np.array([1,0])) #cas Cat
            else:
                targets.append(np.array([0,1])) #cas Dog

        except:
            print("""/!\\ Problème dans l'importation des image /!\\""")


features_ = np.array(features[:20000]) / 255.0 # Normalisation
print(len(features_))
targets_ = np.array(targets[:20000])

indices = np.random.permutation(len(features_)) # On mélange
features_ = features_[indices]
targets_ = targets_[indices]

Data_set = [(x,y) for x,y in zip (features_,targets_)]
print("✅ Téléchargement effectué")


# ------------ Implémentation ------------

def initialisation():
    """
        On initialise le model de variables (weights, bias)
    """
    W = np.random.randn(784, 2)
    bias = np.zeros(2)
    return W, bias

def pre_activation(I,W,bias):
    return I.dot(W)+bias

def activation(Z, fonction_activation):
    """
        Définition de la fonction sigmoïde ou softmax
    """

    if fonction_activation == "sigmoide":

        #---------- Sigmoïde ----------

        #La fonction np.clip permet de limiter les valeurs prisent dans l'exp
        #ainsi on évite l'erreur d'overflow.
        #Par ailleurs, cela ne change rien car sigmoïde aurait renvoyé 0 ou 1

        Z = np.clip(Z, -500, 500)
        return 1 / (1 + np.exp(-Z))

    elif fonction_activation == "softmax":

        #---------- Softmax ----------

        exp_Z = np.exp(Z - np.max(Z))  # stabilité numérique
        return exp_Z / np.sum(exp_Z)

def model(I,W,bias,fonction_activation):
    Z = pre_activation(I, W, bias)
    return activation(Z,fonction_activation)

def Loss(A, S, fonction_activation):
    """
        Définition de la Log_Loss
```

Programme 4 (réseau linéaire sur Cat vs Dog)

```
    epsilon = 1e-15  # petite constante pour éviter log(0)

    if fonction_activation == "sigmoide":

        # Cross-entropy binaire (forme correcte)
        return -np.sum(S*np.log(A+epsilon)+(1-S)*np.log(1-A+epsilon))

    elif fonction_activation == "softmax":
        return -np.sum(S * np.log(A + epsilon))  # cross-entropy

def gradient(W, I, S, bias, fonction_activation):

    A = model(I, W, bias, fonction_activation)

    dZ = A - S
    dW = np.outer(I, dZ)  # ecriture python de I^T.dZ
    db = dZ
    return dW, db

def update(dW, W, bias, db, learning_rate):
    W = W - learning_rate * dW
    bias = bias - learning_rate * db
    return W , bias

def train(features,tag,W,bias,epochs,learning_rate,methode,batch,fonction_activation):
    """
        descente du gradient
    """
    n = len(features)
    evo_cout = [] #évolution de l'erreur

    if methode == "full_batch":

        for epoch in range(epochs):

            print("Epoch ", epoch, "/", epochs, " starting -" )
            cout = []

            weights_gradient = np.zeros(W.shape)
            bias_gradient = np.zeros(bias.shape)

            for feature, target in zip(features,tag):
                A = model(feature, W, bias,fonction_activation)
                cout.append(Loss(A,target,fonction_activation))
                dW , db = gradient(W,feature,target,bias,fonction_activation)
                weights_gradient += dW
                bias_gradient += db

            W , bias = update(weights_gradient/n, W, bias, bias_gradient/n, learning_rate)
            evo_cout.append(np.mean(cout))

        return W , bias , evo_cout

    elif methode == "stochastic":

        for epoch in range(epochs):

            print("Epoch ", epoch, "/", epochs, " starting -" )
            cout = []

            indices = np.random.permutation(n)
            features = features[indices]
            tag = tag[indices]

            for feature, target in zip(features,tag):
                A = model(feature, W, bias,fonction_activation)
                evo_cout.append(Loss(A,target,fonction_activation))
                dW , db = gradient(W,feature,target,bias,fonction_activation)
                W , bias = update(dW, W, bias, db, learning_rate)

        return W , bias , evo_cout

    elif methode == "mini_batch":

        for epoch in range(epochs):

            print("Epoch ", epoch, "/", epochs, " starting -" )
            cout = []

            indices = np.random.permutation(n)
            features = features[indices]
            tag = tag[indices]

            for i in range(0, n, batch):
                batch_features = features[i:i + batch]
                batch_tags = tag[i:i + batch]

                weights_gradient = np.zeros(W.shape)
                bias_gradient = np.zeros(bias.shape)

                for feature, target in zip(batch_features,batch_tags):
                    A = model(feature, W, bias,fonction_activation)
                    cout.append(Loss(A,target,fonction_activation))
                    dW , db = gradient(W,feature,target,bias,fonction_activation)
                    weights_gradient += dW
                    bias_gradient += db

                W , bias = update(weights_gradient / len(batch_features), W, bias, bias_gradient /
len(batch_tags), learning_rate)
                evo_cout.append(np.mean(cout))

        return W , bias , evo_cout

# ------------ Entrainement ------------

features = np.array([x[0] for x in Data_set])
tag = np.array([x[1] for x in Data_set])

W , bias = initialisation()

W , bias , evo_cout = train(features, tag, W, bias, epochs, learning_rate, methode, batch,
fonction_activation)

np.savetxt("weights_" + str(epochs) + "_" + fonction_activation + "_" + str(learning_rate) + "_" +
methode + "_" + str(batch) + ".txt", W)
np.savetxt("bias_" + str(epochs) + "_" + fonction_activation + "_" + str(learning_rate) + "_" + methode +
"_" + str(batch) + ".txt", bias)
```

Programme 4 (réseau linéaire sur Cat vs Dog)

```
print("✅ Poids et bias sauvegardés")


# ------------ Interface graphique ------------

def augment_np(img_np):
    """
        Augmentation simple et compatible avec la vectorisation
        img_np : numpy array (nbpix*nbpix,) ou (nbpix,nbpix)
    """

    # Remise en forme si image flatten
    if img_np.ndim == 1:
        img_np = img_np.reshape(nbpix, nbpix)

    # Flip horizontal
    if np.random.rand() < 0.5:
        img_np = np.fliplr(img_np)

    # Variation de luminosité
    img_np = img_np + np.random.uniform(-0.1, 0.1)
    img_np = np.clip(img_np, 0.0, 1.0)

    # Bruit gaussien
    img_np = img_np + np.random.normal(0, 0.02, img_np.shape)
    img_np = np.clip(img_np, 0.0, 1.0)

    # Rotation légère (avec PIL)
    pil = Image.fromarray((img_np * 255).astype("uint8"))
    angle = np.random.uniform(-10, 10)
    pil = pil.rotate(angle, resample=Image.BILINEAR)

    # Toujours revenir à nbpix × nbpix
    pil = pil.resize((nbpix, nbpix))

    # Retour numpy normalisé entre 0 et 1 + flatten
    img_np = np.array(pil).astype("float32") / 255.0

    return img_np.flatten()


# ------------ Interface graphique ------------

def courbe_de_erreur(methode,fonction_activation, learning_rate, batch):

    if methode == "full_batch":

        info = "learning rate = " + str(learning_rate) + "\n" + \
       "méthode : " + methode + "\n" + \
       "fonction d'activation : " + fonction_activation

        plt.plot(range(len(evo_cout)), evo_cout, label=info)
        plt.title("Evolution de l'erreur")
        plt.xlabel("n ième epoch")
        plt.legend()
        plt.show()

    elif methode == "mini_batch":

        info = "learning rate = " + str(learning_rate) + "\n" + \
            "méthode : " + methode + "\n" + \
            "batch = " + str(batch) + "\n" + \
            "fonction d'activation : " + fonction_activation

        plt.plot(range(len(evo_cout)), evo_cout, label=info)
        plt.title("Evolution de l'erreur")
        plt.xlabel("n ième batch")
        plt.legend()
        plt.show()

        plt.figure()
        window_size = 100
        smooth_error = np.convolve(evo_cout, np.ones(window_size)/window_size, mode='valid')

        plt.plot(range(len(smooth_error)), smooth_error)
        plt.xlabel("Itération")
        plt.ylabel("Erreur moyenne")
        plt.title("Erreur lissée sur "+str(window_size)+" bachs")
        plt.show()

    elif methode == "stochastic":

        info = "learning rate = " + str(learning_rate) + "\n" + \
            "méthode : " + methode + "\n" + \
            "fonction d'activation : " + fonction_activation

        plt.plot(range(len(evo_cout)), evo_cout, label=info)
        plt.title("Evolution de l'erreur")
        plt.xlabel("n ième itération")
        plt.legend()
        plt.show()

        plt.figure()
        window_size = 1000
        smooth_error = np.convolve(evo_cout, np.ones(window_size)/window_size, mode='valid')

        plt.plot(range(len(smooth_error)), smooth_error)
        plt.xlabel("Itération")
        plt.ylabel("Erreur moyenne")
        plt.title("Erreur lissée sur "+str(window_size)+" itérations")
        plt.show()

courbe_de_erreur(methode, fonction_activation, learning_rate, batch)
```

Programme 4 (module de test)

```python
# -*- coding: utf-8 -*-

from PIL import Image
import glob
import os
import numpy as np

nbpix = 28

# ------------ Téléchargement des données ------------

train_dir = r"C:\Users\23mat\OneDrive\Bureau\TIPE\CONCOURS TIPE\data_set_dog_cat\archive\PetImages"
categories = ["Cat", "Dog"]

features = []
targets = []

for label, category in enumerate(categories):
    path = os.path.join(train_dir, category, "*.jpg")
    for file in glob.glob(path):
        try:
            img = Image.open(file).resize((nbpix, nbpix)).convert('L')
            features.append(np.array(img).flatten() / 255.0)  # Normalisation
            if label == 0:
                targets.append(np.array([1, 0]))
            else:
                targets.append(np.array([0, 1]))
        except:
            print(f"/!\\ Problème avec {file} /!\\")

# Prendre un subset si nécessaire
features_ = np.array(features[20000:])
targets_ = np.array(targets[20000:])
indices = np.random.permutation(len(features_))
I_test_data = features_[indices]
Y_test_data = targets_[indices]

Data_set = [(I_test_data[i], Y_test_data[i]) for i in range(len(I_test_data))]
print(f"✅ {len(Data_set)} images test chargées")

# ------------ Chargement des poids et bias ------------

nompoids = input('Nom du fichier des poids (sans le txt) : ')
W = np.loadtxt(nompoids + '.txt')  # W.shape = (784, 2)
nombias = input('Nom du fichier des bias (sans le txt) : ')
bias = np.loadtxt(nombias + '.txt')

print("✅ Poids et bias chargés")

# ------------ Fonction de classification ------------

def calque(I):
    """Classification par produit scalaire + argmax"""
    C = I.dot(W) + bias
    # Pour softmax, argmax suffit
    return np.argmax(C)
```

```python
# ------------ Calcul de la précision ------------

Nb_Erreur = 0
for I, Y in Data_set:
    if calque(I) != np.argmax(Y):
        Nb_Erreur += 1

precision = 100 * (len(Data_set) - Nb_Erreur) / len(Data_set)
print(f"Précision sur le jeu de test : {precision:.2f}%")
```

Programme 5 (réseau non linéaire malade/sain)

```python
# -*- coding: utf-8 -*-

import numpy as np
import matplotlib.pyplot as plt

# ------------ Implémentation ------------


def dataset():
    """
        Définissons notre dataset composé de deux classes:
            -Malade 1
            -Sain 0
    """

    #Nombre de personnes dans une classe
    row_per_class = 100

    #Generate rows
    sick = np.random.randn(row_per_class,2) + np.array([-2,-2]) #Les personnes malades ont des taux faibles
    healthy = np.random.randn(row_per_class,2) + np.array([2,2]) #Les personnes en bonne santé ont des taux élevés

    features = np.vstack([sick, healthy]) #On met la matrice de l'une en dessous de l'autre, shape (10,2)
    tag = np.concatenate((np.ones(row_per_class), np.zeros(row_per_class))) #On étiquettes nos données/features

    return features, tag

def dataset_xor():
    """
        Dataset non linéairement séparable :
            - Sain (0) : globules rouges hauts / blancs bas OU rouges bas / blancs hauts
            - Malade (1) : globules rouges et blancs tous deux hauts OU tous deux bas
    """

    row_per_quadrant = 50

    # Groupe 1 : Malade (taux faibles)
    sick_low = np.random.randn(row_per_quadrant, 2) + np.array([-2, -2])

    # Groupe 2 : Malade (taux élevés)
    sick_high = np.random.randn(row_per_quadrant, 2) + np.array([2, 2])

    # Groupe 3 : Sain (rouges hauts / blancs bas)
    healthy_rb = np.random.randn(row_per_quadrant, 2) + np.array([2, -2])

    # Groupe 4 : Sain (rouges bas / blancs hauts)
    healthy_br = np.random.randn(row_per_quadrant, 2) + np.array([-2, 2])

    # Fusion des données
    features = np.vstack([sick_low, sick_high, healthy_rb, healthy_br])
    tag = np.concatenate((
        np.ones(2 * row_per_quadrant),      # Malades = 1
        np.zeros(2 * row_per_quadrant)      # Sains = 0
    ))

    return features, tag

def affichage_globule_blanc_rouge (features,tag):
    """
         schéma du dataset
    """

    for i, elt in enumerate(features):
        if tag[i] ==1:
            plt.plot(elt[0],elt[1],'rx')
        else:
            plt.plot(elt[0],elt[1],'gx')
    plt.xlabel('Quantité de Globules rouges')
    plt.ylabel('Quantité de Globules blancs')
    plt.legend()

def afficher_frontiere(features, weights1, bias1, weights2, bias2, resolution=200):
    # Grille de points

    plt.figure()
    x_min, x_max = features[:,0].min() - 1, features[:,0].max() + 1
    y_min, y_max = features[:,1].min() - 1, features[:,1].max() + 1
    xx, yy = np.meshgrid(np.linspace(x_min, x_max, resolution),
                         np.linspace(y_min, y_max, resolution))
    grid = np.c_[xx.ravel(), yy.ravel()]

    # Passage dans le réseau
    Z1 = np.dot(grid, weights1) + bias1
    A1 = activation(Z1)
    Z2 = np.dot(A1, weights2) + bias2
    A2 = activation(Z2).reshape(xx.shape)

    # Tracé
    plt.contourf(xx, yy, A2, levels=50, cmap="RdYlGn", alpha=0.6)
    plt.colorbar(label="Probabilité d'être malade")
    affichage_globule_blanc_rouge(features, tag)
    plt.title("Frontière de décision (réseau à 2 couches)")
    plt.show()

def init_variables():
    """
        On initialise le model de variables (weights, bias)
    """

    # 1ère couche
    weights1 = np.random.normal(size=(2,3))
    bias1 = np.zeros(3)

    # 2ème couche
    weights2 = np.random.normal(size=(3,1))
    bias2 = np.zeros(1)

    return weights1, bias1, weights2, bias2

def pre_activation(features,weights,bias):
    return np.dot(features,weights)+bias

def activation(z):
    """
        Définition de la fonction sigmoïde
    """
```

Programme 5 (réseau non linéaire malade/sain)

```
    return 1/(1+np.exp(-z))

def predict(features,w1,b1,w2,b2):
    """
        Calcul de la probabilité d'appartenir à une classe ou une autre
    """
    A1 = activation(np.dot(features, w1) + b1)
    A2 = activation(np.dot(A1, w2) + b2)
    return (A2 > 0.5).astype(int)


def erreur(predictions, tag):
    predictions = predictions.reshape(-1) #on s'assure de la bonne dimension
    epsilon = 1e-15  # petite constante pour éviter log(0)
    LL = - (1/len(tag)) * np.sum(tag*np.log(predictions+epsilon)+(1-tag)*np.log(1-predictions+epsilon))
    return LL

def train(features,tag,weights1,bias1,weights2,bias2,epochs):
    """
        Descente du gradient
    """
    learning_rate = 0.1
    pred = [] #évolution du diagnostique pour chaque patient
    cout = [] #évolution de l'erreur
    just_diag = [] #justesse du diagnostique en %
    w2 = [] #poids sauvegardés de la couche 2
    b2 = [] #bias sauvegardés de la couche 2
    w1 = [] #poids sauvegardés de la couche 1
    b1 = [] #bias sauvegardés de la couche 1

    for epoch in range(epochs):

        # Récupération de données pour les graphs

        w1.append(weights1)
        b1.append(bias1)
        w2.append(weights2)
        b2.append(bias2)

        Z1 = np.dot(features, weights1) + bias1
        A1 = activation(Z1)
        Z2 = np.dot(A1, weights2) + bias2
        A2 = activation(Z2)

        predictions = (A2 > 0.5).astype(int)
        just_diag.append(np.mean(predictions.reshape(-1) == tag.reshape(-1))) #on s'assure des bonnes
dimensions)
        cout.append(erreur(A2,tag))
        pred.append(A2.mean())


        # Init. gradients et bias

        weights_gradient1 = np.zeros(weights1.shape)
        weights_gradient2 = np.zeros(weights2.shape)
        bias_gradient1 = np.zeros(bias1.shape)
        bias_gradient2 = np.zeros(bias2.shape)

        # Descente

        for feature, target in zip(features,tag):

            Z1 = np.dot(feature, weights1) + bias1 #pre_activation
            A1 = activation(Z1)
            Z2 = np.dot(A1, weights2) + bias2 #pre_activation
            A2 = activation(Z2)

            # Descente sur la seconde couche
            dZ2 = A2 - target
            weights_gradient2 += np.outer(A1,dZ2)
            bias_gradient2 += dZ2

            # Descente sur la première couche
            dZ1 = np.dot(weights2, dZ2) * A1 * (1 - A1)  # produit matriciel
            weights_gradient1 += np.outer(feature, dZ1)
            bias_gradient1 += dZ1

        # Normalisation

        weights_gradient2 = weights_gradient2 * (1/len(tag))
        bias_gradient2 = bias_gradient2 * (1/len(tag))
        weights_gradient1 = weights_gradient1 * (1/len(tag))
        bias_gradient1 = bias_gradient1 * (1/len(tag))

        weights2 = weights2 - learning_rate*weights_gradient2
        bias2 = bias2 - learning_rate*bias_gradient2
        weights1 = weights1 - learning_rate*weights_gradient1
        bias1 = bias1 - learning_rate*bias_gradient1


    w1.append(weights1)
    b1.append(bias1)
    w2.append(weights2)
    b2.append(bias2)

    return cout, pred, just_diag, w1, b1, w2, b2


# ------------ Entrainement ------------

epochs = 10000

# Initialistation de l'algorithme

features, tag = dataset_xor()
weights1_i, bias1_i, weights2_i, bias2_i = init_variables()

# Récupération des données après algorithme

cout, pred, just_diag, w1, b1, w2, b2 = train(features, tag, weights1_i, bias1_i, weights2_i, bias2_i,
epochs)
```

Programme 5 (réseau non linéaire malade/sain)

```
# ------------ Interface graphique ------------


affichage_globule_blanc_rouge(features, tag)
plt.xlim(-5,5)
plt.ylim(-6,6)
plt.show()

afficher_frontiere(features, weights1_i, bias1_i, weights2_i, bias2_i)

afficher_frontiere(features, w1[epochs//2], b1[epochs//2], w2[epochs//2], b2[epochs//2])

afficher_frontiere(features, w1[-1], b1[-1], w2[-1], b2[-1])

epoc = [i + 1 for i in range(epochs)]
plt.plot(epoc, cout, '-')
plt.title("Évolution de l'erreur")
plt.xlabel("Époch")
plt.ylabel("Erreur (log loss)")
plt.show()


plt.figure()
plt.plot(just_diag)
plt.title("Évolution de la justesse du diagnostic")
plt.xlabel("Époch")
plt.ylabel("Précision (%)")
plt.show()


# ------------ Test des poids obtenues ------------

nb = 1000  # nombre de tests
precisions = []

for i in range(nb):
    features_test, tag_test = dataset_xor()
    predictions = predict(features_test, w1[-1], b1[-1], w2[-1], b2[-1])  # utiliser les poids finaux
    precision = np.mean(predictions.reshape(-1) == tag.reshape(-1)) #on s'assure des bonnes dimensions
    precisions.append(precision)

print("L'algorithme est fiable à " + str(np.mean(precisions)*100) + "%")
```

Programme 5 (réseau non linéaire malade/sain)

```
# ------------ Interface graphique ------------


affichage_globule_blanc_rouge(features, tag)
plt.xlim(-5,5)
plt.ylim(-6,6)
plt.show()

afficher_frontiere(features, weights1_i, bias1_i, weights2_i, bias2_i)

afficher_frontiere(features, w1[epochs//2], b1[epochs//2], w2[epochs//2], b2[epochs//2])

afficher_frontiere(features, w1[-1], b1[-1], w2[-1], b2[-1])

epoc = [i + 1 for i in range(epochs)]
plt.plot(epoc, cout, '-')
plt.title("Évolution de l'erreur")
plt.xlabel("Époch")
plt.ylabel("Erreur (log loss)")
plt.show()


plt.figure()
plt.plot(just_diag)
plt.title("Évolution de la justesse du diagnostic")
plt.xlabel("Époch")
plt.ylabel("Précision (%)")
plt.show()


# ------------ Test des poids obtenues ------------

nb = 1000  # nombre de tests
precisions = []

for i in range(nb):
    features_test, tag_test = dataset_xor()
    predictions = predict(features_test, w1[-1], b1[-1], w2[-1], b2[-1])  # utiliser les poids finaux
    precision = np.mean(predictions.reshape(-1) == tag.reshape(-1)) #on s'assure des bonnes dimensions
    precisions.append(precision)

print("L'algorithme est fiable à " + str(np.mean(precisions)*100) + "%")
```

Programme 6 (réseau non linéaire MNIST)

```python
# -*- coding: utf-8 -*-

import tensorflow as tf
from tensorflow import keras
import numpy as np
import matplotlib.pyplot as plt

# ------------ Paramètres ------------

epochs = 15
learning_rate = 0.01

#choix de la méthode utilisée selon stochastic, full_batch, mini_batch pour la descente

methode = "mini_batch"
batch = 16 # généralement batch = 16, 32 ou 128

#choix de la fonction d'activation parmis sigmoide et softmax pour la dernière couche
#(la fonction d'activation pour la première couche est toujours sigmoïde)
fonction_activation = "sigmoide"

#choix du nombre de neuronnes dans la couche cachée
hidden_size = 64

# ------------ Vérification des paramètres ------------

assert type(epochs) == int, "epochs doit être un entier"
assert type(learning_rate) == float or type(learning_rate) == int, "le learning rate doit être un nombre"
assert methode in ["stochastic","full_batch","mini_batch"], "erreur sur la methode"
assert type(batch) == int, "batch doit être un entier"
assert fonction_activation in ["sigmoide","softmax"]
assert type(hidden_size) == int, "hidden_size doit être un entier"

print("✅ Aucun problème pour les entrées")


# ------------ Téléchargement des données  ------------

from tensorflow.keras.datasets import mnist
(I_train_data, Y_train_data), (I_test_data, Y_test_data) = mnist.load_data()  #I_train_data sont les
matrices des nombres et Y_train_data les numéro


# ------------ Affichage de nombres pour l'exemple ------------

def affichage_MNIST(nb):
    """
        affiche l'image de l'item numéro nb de la banque d'image MNIST
    """
    plt.imshow(I_train_data[nb], cmap='gray')
    plt.colorbar()  #ajoute une échelle de couleurs
    plt.title('Image numéro : '+ str(nb))
    plt.show()
```

```python
# ------------ Vectorisation ------------

#On transforme les matrices de MNIST en lignes
Data_set = []

for i in range(len(I_train_data)):
    N_image = []
    N_sortie = [0 for i in range(10)]
    for j in range(28):
        for k in range(28):
            N_image.append((I_train_data[i][j][k])/255) #on divise par 255 pour normalisée

    N_sortie[Y_train_data[i]] = 1

    Data_set.append((np.array(N_image), np.array(N_sortie)))

print("✅ Vectorisation effectuée")


# ------------ Implémentation ------------

def initialisation(hidden_size):
    """
        On initialise le model de variables (weights, bias)
    """

    #Initialisation de la première couche
    W1 = np.random.randn(784, hidden_size)
    bias1 = np.zeros(hidden_size)

    #Initialisation de la seconde couche
    W2 = np.random.randn(hidden_size, 10)
    bias2 = np.zeros(10)

    return W1, bias1, W2, bias2

def pre_activation(I,W,bias):
    return I.dot(W)+bias

def activation(Z, fonction_activation):
    """
        Définition de la fonction sigmoïde / softmax
    """

    if fonction_activation == "sigmoide":

        #---------- Sigmoïde ----------

        #La fonction np.clip permet de limiter les valeurs prisent dans l'exp
        #ainsi on évite l'erreur d'overflow.
        #Par ailleurs, cela ne change rien car sigmoïde aurait renvoyée 0 ou 1

        Z = np.clip(Z, -500, 500)
        return 1 / (1 + np.exp(-Z))

    elif fonction_activation == "softmax":

        #---------- Softmax ----------
```

Programme 6 (réseau non linéaire MNIST)

```
                exp_Z = np.exp(Z - np.max(Z))  # pour stabilité numérique et éviter l'overflow (astuce
de calcul)
        return exp_Z / np.sum(exp_Z)

def model(I,W,bias,fonction_activation):
    Z = pre_activation(I, W, bias)
    return activation(Z,fonction_activation)

def Loss(A, S, fonction_activation):
    """
        Définition de la Log_Loss
    """
    epsilon = 1e-15  # petite constante pour éviter log(0)

    if fonction_activation == "sigmoide":

        # Cross-entropy binaire
        return -np.mean(S*np.log(A+epsilon)+(1-S)*np.log(1-A+epsilon))


    elif fonction_activation == "softmax":

        #Cross-entropy multi-classes
        return -np.mean(np.sum(S * np.log(A)))  # produit de Hadamard

def gradient(W1, bias1, W2, bias2, I, S, fonction_activation):

    A1 = model(I, W1, bias1, "sigmoide")
    A2 = model(A1, W2, bias2, fonction_activation)

    # Back-propagation

    # Descente sur la seconde couche
    dZ2 = A2 - S
    dW2 = np.outer(A1,dZ2)
    db2 = dZ2

    # Descente sur la première couche
    dZ1 = np.dot(W2, dZ2) * A1 * (1 - A1)  # produit matriciel
    dW1 = np.outer(I, dZ1) # ecriture python de I^T.dZ1
    db1 = dZ1

    return dW1, db1, dW2, db2

def update(dW1, W1, db1, bias1, dW2, W2, db2, bias2, learning_rate):

    W1 = W1 - learning_rate * dW1
    bias1 = bias1 - learning_rate * db1
    W2 = W2 - learning_rate * dW2
    bias2 = bias2 - learning_rate * db2

    return W1 , bias1, W2, bias2

def train(features,tag,W1,bias1,W2,bias2,epochs,learning_rate,methode,batch,fonction_activation):
    """
        descente du gradient
    """
    n = len(features)
    evo_cout = [] #évolution de l'erreur

    if methode == "full_batch":

        for epoch in range(epochs):

            print("Epoch ", epoch, "/", epochs, " starting -" )
            cout = []

            weights_gradient1 = np.zeros(W1.shape)
            bias_gradient1 = np.zeros(bias1.shape)
            weights_gradient2 = np.zeros(W2.shape)
            bias_gradient2 = np.zeros(bias2.shape)

            for feature, target in zip(features,tag):

                A1 = model(feature, W1, bias1, "sigmoide")
                A2 = model(A1, W2, bias2, fonction_activation)
                cout.append(Loss(A2,target,fonction_activation))

                dW1, db1, dW2, db2 = gradient(W1, bias1, W2, bias2, feature, target,
fonction_activation)
                weights_gradient1 += dW1
                bias_gradient1 += db1
                weights_gradient2 += dW2
                bias_gradient2 += db2

            W1 , bias1, W2, bias2 = update(weights_gradient1/n, W1, bias_gradient1/n, bias1,
weights_gradient2/n, W2, bias_gradient2/n, bias2, learning_rate)

            evo_cout.append(np.mean(cout))

        return W1, bias1, W2, bias2, evo_cout

    elif methode == "stochastic":

        for epoch in range(epochs):

            print("Epoch ", epoch, "/", epochs, " starting -" )
            cout = []

            # On mélange les données pour ne pas faire systématiquement la même descente
            indices = np.random.permutation(n)
            features = features[indices]
            tag = tag[indices]

            for feature, target in zip(features,tag):

                A1 = model(feature, W1, bias1, "sigmoide")
                A2 = model(A1, W2, bias2, fonction_activation)
                cout.append(Loss(A2,target,fonction_activation))

                dW1, db1, dW2, db2 = gradient(W1, bias1, W2, bias2, feature, target,
fonction_activation)
                W1 , bias1, W2, bias2 = update(weights_gradient1/n, W1, bias_gradient1/n, bias1,
weights_gradient2/n, W2, bias_gradient2/n, bias2, learning_rate)

        return W1, bias1, W2, bias2, evo_cout
```

Programme 6 (réseau non linéaire MNIST)

```
    elif methode == "mini_batch":

     for epoch in range(epochs):

         print("Epoch ", epoch, "/", epochs, " starting -" )
         cout = []

         # On mélange les données pour ne pas faire systématiquement la même descente
         indices = np.random.permutation(n)
         features = features[indices]
         tag = tag[indices]


         for i in range(0, n, batch):
             batch_features = features[i:i + batch]
             batch_tags = tag[i:i + batch]

             weights_gradient1 = np.zeros(W1.shape)
             bias_gradient1 = np.zeros(bias1.shape)
             weights_gradient2 = np.zeros(W2.shape)
             bias_gradient2 = np.zeros(bias2.shape)

             for feature, target in zip(batch_features,batch_tags):

                 A1 = model(feature, W1, bias1, "sigmoide")
                 A2 = model(A1, W2, bias2, fonction_activation)
                 cout.append(Loss(A2,target,fonction_activation))

                 dW1, db1, dW2, db2 = gradient(W1, bias1, W2, bias2, feature, target,
fonction_activation)
                 weights_gradient1 += dW1
                 bias_gradient1 += db1
                 weights_gradient2 += dW2
                 bias_gradient2 += db2

             W1 , bias1, W2, bias2 = update(weights_gradient1/len(batch_features), W1,
bias_gradient1/len(batch_features), bias1, weights_gradient2/len(batch_features), W2,
bias_gradient2/len(batch_features), bias2, learning_rate)

             evo_cout.append(np.mean(cout))

     return W1, bias1, W2, bias2, evo_cout

# ------------ Entrainement ------------

#Initialisation de l'algorithme :

features = np.array([x[0] for x in Data_set])
tag = np.array([x[1] for x in Data_set])

W1, bias1, W2, bias2 = initialisation(hidden_size)

# Récupération des données après algorithme

W1, bias1, W2, bias2, evo_cout = train(features, tag, W1, bias1, W2, bias2, epochs, learning_rate,
methode, batch, fonction_activation)

np.savetxt("weights1_" + str(epochs) + "_" + fonction_activation + "_" + str(learning_rate) + "_" +
methode + "_" + str(batch) + "CC" + str(hidden_size) + ".txt", W1)
np.savetxt("bias1_" + str(epochs) + "_" + fonction_activation + "_" + str(learning_rate) + "_" +
methode + "_" + str(batch) + "CC" + str(hidden_size) + ".txt", bias1)
np.savetxt("weights2_" + str(epochs) + "_" + fonction_activation + "_" + str(learning_rate) + "_" +
methode + "_" + str(batch) + "CC" + str(hidden_size) + ".txt", W2)
np.savetxt("bias2_" + str(epochs) + "_" + fonction_activation + "_" + str(learning_rate) + "_" +
methode + "_" + str(batch) + "CC" + str(hidden_size) + ".txt", bias2)

print("✅ Poids et bias sauvegardés")


# ------------ Interface graphique ------------

def courbe_de_erreur(methode,fonction_activation, learning_rate, batch, hidden_size):

    if methode == "full_batch":

        info = "learning rate = " + str(learning_rate) + "\n" + \
       "méthode : " + methode + "\n" + \
       str(hidden_size) + " neuronnes dans la couche cachée" + "\n" + \
       "fonction d'activation : " + fonction_activation

        plt.plot(range(len(evo_cout)), evo_cout, label=info)
        plt.title("Evolution de l'erreur")
        plt.xlabel("n ième epoch")
        plt.legend()
        plt.show()

    elif methode == "mini_batch":

        info = "learning rate = " + str(learning_rate) + "\n" + \
            "méthode : " + methode + "\n" + \
            str(hidden_size) + " neuronnes dans la couche cachée" + "\n" + \
            "batch = " + str(batch) + "\n" + \
            "fonction d'activation : " + fonction_activation

        plt.plot(range(len(evo_cout)), evo_cout, label=info)
        plt.title("Evolution de l'erreur")
        plt.xlabel("n ième batch")
        plt.legend()
        plt.show()

        plt.figure()
        window_size = 100
        smooth_error = np.convolve(evo_cout, np.ones(window_size)/window_size, mode='valid')

        plt.plot(range(len(smooth_error)), smooth_error)
        plt.xlabel("Itération")
        plt.ylabel("Erreur moyenne")
        plt.title("Erreur lissée sur "+str(window_size)+" bachs")
        plt.show()

    elif methode == "stochastic":

        info = "learning rate = " + str(learning_rate) + "\n" + \
            "méthode : " + methode + "\n" + \
            str(hidden_size) + " neuronnes dans la couche cachée" + "\n" + \
            "fonction d'activation : " + fonction_activation

        plt.plot(range(len(evo_cout)), evo_cout, label=info)
```

Programme 6 (réseau non linéaire MNIST)

```
        plt.title("Evolution de l'erreur")
        plt.xlabel("n ième itération")
        plt.legend()
        plt.show()

        plt.figure()
        window_size = 1000
        smooth_error = np.convolve(evo_cout, np.ones(window_size)/window_size, mode='valid')

        plt.plot(range(len(smooth_error)), smooth_error)
        plt.xlabel("Itération")
        plt.ylabel("Erreur moyenne")
        plt.title("Erreur lissée sur "+str(window_size)+" itérations")
        plt.show()

courbe_de_erreur(methode, fonction_activation, learning_rate, batch, hidden_size)
```

Programme 6 (module de test)

```python
# -*- coding: utf-8 -*-

from PIL import Image
import numpy as np
import matplotlib.pyplot as plt
from tensorflow.keras.datasets import mnist

# ------------ Téléchargement des données ------------

(I_train_data, Y_train_data), (I_test_data, Y_test_data) = mnist.load_data()

Data_set = []

for i in range(len(I_test_data)):
    N_image = []
    for j in range(28):
        for k in range(28):
            N_image.append((I_test_data[i][j][k])/255)

    Data_set.append(np.array(N_image))

# ------------ Chargement des poids et bias ------------

nompoids1 = input('Nom du fichier des poids 1 (sans le txt) : ')
W1 = np.loadtxt(nompoids1 + '.txt')  # W1.shape = (784, hidden_size)

nombias1 = input('Nom du fichier des bias 1 (sans le txt) : ')
bias1 = np.loadtxt(nombias1 + '.txt')

nompoids2 = input('Nom du fichier des poids 2 (sans le txt) : ')
W2 = np.loadtxt(nompoids2 + '.txt')  # W2.shape = (hidden_size, 10)

nombias2 = input('Nom du fichier des bias 2 (sans le txt) : ')
bias2 = np.loadtxt(nombias2 + '.txt')

# ------------ Fonction pour afficher un filtre ------------

def afficher_filtre(poids, taille=(28, 28), titre=""):
    poids = np.array(poids).reshape(taille)
    max_abs = np.max(np.abs(poids))
    if max_abs == 0:
        max_abs = 1
    poids_norm = poids / max_abs

    img = np.zeros((*taille, 3))
    img[:, :, 0] = np.clip(poids_norm, 0, 1)   # Rouge
    img[:, :, 2] = np.clip(-poids_norm, 0, 1)  # Bleu
    img[np.isclose(poids_norm, 0)] = [1, 1, 1]  # Blanc

    plt.imshow(img)
    plt.title(titre)
    plt.axis('off')

# ------------ Affichage de tous les filtres ------------


def afficher_filtre_didactique(filtre, shape=(28,28), titre=None):
    """
    Affiche un filtre (vecteur de poids) sous forme d'image :
    - rouge vif : active la classe
    - bleu foncé : inhibe la classe
    - blanc : neutre
    """
    filtre_2D = filtre.reshape(shape)
    vabs = np.max(np.abs(filtre_2D))

    plt.imshow(filtre_2D, cmap='bwr', vmin=-vabs, vmax=vabs)
    plt.axis("off")
    if titre:
        plt.title(titre, fontsize=10, fontweight="bold")

def visualiser_filtres_didactique(W, shape=(28,28)):
    """
    Affiche 10 filtres appris de la couche cachée
    """
    plt.figure(figsize=(12, 6))
    for i in range(10):
        plt.subplot(2, 5, i + 1)
        afficher_filtre_didactique(W[:, i], shape, titre=f"Filtre du neurone {i}")
    plt.suptitle(
        "Filtres appris -- zones rouges = pixels activants, bleues = inhibants",
        fontsize=14, fontweight="bold"
    )
    plt.tight_layout()
    plt.show()


visualiser_filtres_didactique(W1[:, :10])


def visualiser_W2(W2):
    plt.figure(figsize=(10, 4))
    plt.imshow(W2, cmap="bwr", aspect="auto")
    plt.colorbar()
    plt.title("Poids entre la couche cachée (64 neurones) et les 10 sorties")
    plt.xlabel("Classe de sortie (0-9)")
    plt.ylabel("Neurones cachés (1-64)")
    plt.show()

visualiser_W2(W2)


# ------------ test image en particulier ------------

# def calque(I, W1, b1, W2, b2):
#     # Première couche (activation sigmoïde)
#     Z1 = I.dot(W1) + b1
#     A1 = 1 / (1 + np.exp(-Z1))  # sigmoïde

#     # Deuxième couche (sortie softmax)
#     Z2 = A1.dot(W2) + b2
#     expZ = np.exp(Z2 - np.max(Z2))
#     A2 = expZ / np.sum(expZ)
#     return np.argmax(A2)
```

Programme 6 (module de test)

```
def calque(I, W1, b1, W2, b2, activation="softmax"):
    """
    Calcule la sortie du réseau à deux couches :
    - Première couche : sigmoïde
    - Deuxième couche : sigmoïde
    """
    # /!\ Il s'agit de la version sigmoïde

    # Aplatit l'image si elle est 2D (28x28)
    if I.ndim == 2:
       I = I.flatten()

    # --- Couche 1 ---
    Z1 = I.dot(W1) + b1
    A1 = 1 / (1 + np.exp(-Z1))  # sigmoïde

    # --- Couche 2 ---
    Z2 = A1.dot(W2) + b2

    A2 = 1 / (1 + np.exp(-Z2))
    return np.argmax(A2)  # pour rester comparable


def bonformat (img):
    """
        Entrée : Image

        Objectif : Mettre l'image dans un format adaptée aux tests

        Réalise : -Redimensionne l'image
                  -La convertit en un tableau avec des entiers compris entre 0 et 255

        Sortie : Tableau
    """

    img = redim(img, 28)
    img = convertgris(img)

    return (img)

def redim(img, f):
    """
        Entrée : img = Image et f = entier positif

        Objectif : Redimensionner une image dans le format "fxf"

        Sortie : Image
    """

    img_resize = img.resize((f, f))
    return (img_resize)

def convertgris (img):
    """
        Entrée : Image

        Objectif : Permet d'obtenir un tableau contenant le niveau de gris des pixels.

        Info : Le tableau renvoie ligne par ligne
        Sortie : Tableau
    """

    # Convertit en tableau NumPy (avec des nombres compris entre 0 et 255)
    #(0 correspond à noir, 255 à blanc):
    image = img.convert("L")


    #cette fonction sert à pouvoir ajuster les entiers dans la matrice
    #et pas uniquement avoir des entiers entre 0 et 255
    array = np.array(image).astype(int)

    return (array)

def ajustpixels(array):
    """
        Entrée : Tableau

        Objectif : Changer les valeurs des pixels

        Realise : -Les nombres proches de 255 correspondant au blanc deviennent proche de 0
                  -Les nombres proches de 0 correspondant au noir deviennent proche de 1

        Sortie : Tableau
    """

    return (255 - array)/255

def test():
    nom2 = input("nom de l'image:") + "." + input("format de l'image:")
    try:
        inter = Image.open(nom2)
        inter = list(ajustpixels(bonformat(inter)))
        img = []
        for l in range(28):
            for c in range(28):
                img.append(inter[l][c])
        print(calque(np.array(img)))

    except:
        print("/!\ Erreur : Le nom ou le format de l'image est incorrect")


# ------------ Efficacité ------------

Nb_Erreur = 0

for i in range(len(I_test_data)):

    if calque(Data_set[i], W1, bias1, W2, bias2) != Y_test_data[i]:
        Nb_Erreur+=1

print("L'algorithme est précis à la hauteur de :", 100*(len(I_test_data)-Nb_Erreur)/len(I_test_data),"%")
```

Programme 6 (module de test)

```
# def precision(I_test_data, Y_test_data, W1, b1, W2, b2, activation="sigmoide"):
#     bonnes = 0
#     for i in range(len(I_test_data)):
#         pred = calque(I_test_data[i], W1, b1, W2, b2, activation)
#         if pred == np.argmax(Y_test_data[i]):
#             bonnes += 1
#     taux = 100 * bonnes / len(I_test_data)
#     print(f"Précision avec {activation} : {taux:.2f}%")
#     return taux

# # Exemple d'appel :
# precision(I_test_data, Y_test_data, W1, bias1, W2, bias2, activation="sigmoide")
```

Programme 7 (réseau non linéaire Chest Xray MNIST)

```python
# -*- coding: utf-8 -*-

import numpy as np
import matplotlib.pyplot as plt
import medmnist
from medmnist import INFO

# ------------ Paramètres ------------

epochs = 10
learning_rate = 0.05
methode = "mini_batch"  # "stochastic", "full_batch", "mini_batch"
batch = 32  # batch size
hidden_size = 64
fonction_activation = "sigmoide"  #idéal pour le multi-label

# ------------ Vérification des paramètres ------------

assert type(epochs) == int
assert type(learning_rate) in [float,int]
assert methode in ["stochastic","full_batch","mini_batch"]
assert type(batch) == int
assert fonction_activation in ["sigmoide","softmax"]
assert type(hidden_size) == int
print("✅ Aucun problème pour les entrées")

# ------------ Téléchargement des données ChestMNIST ------------

data_flag = "chestmnist"
info = INFO[data_flag]
DataClass = getattr(medmnist, info['python_class'])

train_dataset = DataClass(split='train', download=True)
test_dataset = DataClass(split='test', download=True)

# ------------ Vectorisation ------------

Data_set = []
for i in range(len(train_dataset)):
    N_image = train_dataset.imgs[i].flatten().astype("float32") / 255.0
    N_sortie = train_dataset.labels[i].flatten()  # 14 labels multi-label
    Data_set.append((N_image, N_sortie))

print("✅ Vectorisation effectuée")

# ------------ Fonctions du modèle ------------

def initialisation(hidden_size, input_size=784, output_size=14):

    #Initialisation de la première couche
    W1 = np.random.randn(input_size, hidden_size)
    bias1 = np.zeros(hidden_size)

    #Initialisation de la deuxième couche
    W2 = np.random.randn(hidden_size, output_size)
    bias2 = np.zeros(output_size)

    return W1, bias1, W2, bias2

def pre_activation(I, W, bias):

    return I.dot(W) + bias

def activation(Z, fonction_activation):

    if fonction_activation == "sigmoide":
        Z = np.clip(Z, -500, 500)
        return 1 / (1 + np.exp(-Z))
    elif fonction_activation == "softmax":
        exp_Z = np.exp(Z - np.max(Z))
        return exp_Z / np.sum(exp_Z)

def model(I, W, bias, fonction_activation):
    Z = pre_activation(I, W, bias)
    return activation(Z, fonction_activation)

def Loss(A, S, fonction_activation):
    epsilon = 1e-15
    if fonction_activation == "sigmoide":
        # binary cross-entropy multi-label
        return -np.mean(S*np.log(A+epsilon) + (1-S)*np.log(1-A+epsilon))
    elif fonction_activation == "softmax":
        return -np.mean(np.sum(S * np.log(A + epsilon)))

def gradient(W1, bias1, W2, bias2, I, S, fonction_activation):
    A1 = model(I, W1, bias1, "sigmoide")
    A2 = model(A1, W2, bias2, fonction_activation)

    dZ2 = A2 - S
    dW2 = np.outer(A1, dZ2)
    db2 = dZ2

    dZ1 = np.dot(W2, dZ2) * A1 * (1 - A1)
    dW1 = np.outer(I, dZ1)
    db1 = dZ1

    return dW1, db1, dW2, db2

def update(dW1, W1, db1, bias1, dW2, W2, db2, bias2, learning_rate):
    W1 = W1 - learning_rate * dW1
    bias1 = bias1 - learning_rate * db1
    W2 = W2 - learning_rate * dW2
    bias2 = bias2 - learning_rate * db2
    return W1, bias1, W2, bias2

# ------------ Entrainement ------------

features = np.array([x[0] for x in Data_set])
tag = np.array([x[1] for x in Data_set])
W1, bias1, W2, bias2 = initialisation(hidden_size)

def train(features, tag, W1, bias1, W2, bias2, epochs, learning_rate, methode, batch,
fonction_activation):
    n = len(features)
    evo_cout = []

    for epoch in range(epochs):
        print("Epoch ", epoch+1, "/", epochs, " starting -")
```

Programme 7 (réseau non linéaire Chest Xray MNIST)

```
        cout = []
        indices = np.random.permutation(n)
        features_shuffled = features[indices]
        tag_shuffled = tag[indices]

        for i in range(0, n, batch):
            batch_features = features_shuffled[i:i+batch]
            batch_tags = tag_shuffled[i:i+batch]

            weights_gradient1 = np.zeros(W1.shape)
            bias_gradient1 = np.zeros(bias1.shape)
            weights_gradient2 = np.zeros(W2.shape)
            bias_gradient2 = np.zeros(bias2.shape)

            for feature, target in zip(batch_features, batch_tags):
                A1 = model(feature, W1, bias1, "sigmoide")
                A2 = model(A1, W2, bias2, fonction_activation)
                cout.append(Loss(A2, target, fonction_activation))
                dW1, db1, dW2, db2 = gradient(W1, bias1, W2, bias2, feature, target,
fonction_activation)
                weights_gradient1 += dW1
                bias_gradient1 += db1
                weights_gradient2 += dW2
                bias_gradient2 += db2

            W1, bias1, W2, bias2 = update(weights_gradient1/len(batch_features),
                                          W1,
                                          bias_gradient1/len(batch_features),
                                          bias1,
                                          weights_gradient2/len(batch_features),
                                          W2,
                                          bias_gradient2/len(batch_features),
                                          bias2,
                                          learning_rate)
        evo_cout.append(np.mean(cout))
        print("  Erreur moyenne :", np.mean(cout))
    return W1, bias1, W2, bias2, evo_cout

W1, bias1, W2, bias2, evo_cout = train(features, tag, W1, bias1, W2, bias2,
                                       epochs, learning_rate, methode, batch, fonction_activation)

np.savetxt("med-non-convo-weights1_" + str(epochs) + "_" + fonction_activation + "_" +
str(learning_rate) + "_" + methode + "_" + str(batch) + "CC" + str(hidden_size) + ".txt", W1)
np.savetxt("med-non-convo-bias1_" + str(epochs) + "_" + fonction_activation + "_" + str(learning_rate)
+ "_" + methode + "_" + str(batch) + "CC" + str(hidden_size) + ".txt", bias1)
np.savetxt("med-non-convo-weights2_" + str(epochs) + "_" + fonction_activation + "_" +
str(learning_rate) + "_" + methode + "_" + str(batch) + "CC" + str(hidden_size) + ".txt", W2)
np.savetxt("med-non-convo-bias2_" + str(epochs) + "_" + fonction_activation + "_" + str(learning_rate)
+ "_" + methode + "_" + str(batch) + "CC" + str(hidden_size) + ".txt", bias2)

print("✅ Poids et bias sauvegardés")

# ------------ Courbe de l'erreur ------------
plt.plot(range(len(evo_cout)), evo_cout, label="Erreur")
plt.xlabel("Epochs")
plt.ylabel("Erreur moyenne")
plt.title("Évolution de l'erreur (ChestMNIST)")
plt.legend()
plt.show()
```

Programme 7 (module de test)

```python
# -*- coding: utf-8 -*-

from PIL import Image
import numpy as np
import matplotlib.pyplot as plt
import medmnist
from medmnist import INFO

# ------------ Téléchargement des données ChestMNIST ------------
data_flag = "chestmnist"
info = INFO[data_flag]
DataClass = getattr(medmnist, info['python_class'])

train_dataset = DataClass(split='train', download=True)
test_dataset = DataClass(split='test', download=True)

# Flatten et normalisation
Data_set = []
for i in range(len(test_dataset)):
    N_image = test_dataset.imgs[i].flatten().astype("float32") / 255.0
    Data_set.append(np.array(N_image))

print("✅ Données flattenées et normalisées prêtes")

# ------------ Chargement des poids et biais ------------
nompoids1 = input('Nom du fichier des poids 1 (sans le txt) : ')
W1 = np.loadtxt(nompoids1 + '.txt')

nombias1 = input('Nom du fichier des bias 1 (sans le txt) : ')
bias1 = np.loadtxt(nombias1 + '.txt')

nompoids2 = input('Nom du fichier des poids 2 (sans le txt) : ')
W2 = np.loadtxt(nompoids2 + '.txt')

nombias2 = input('Nom du fichier des bias 2 (sans le txt) : ')
bias2 = np.loadtxt(nombias2 + '.txt')

# ------------ Visualisation des filtres ------------

def afficher_filtre_didactique(filtre, shape=(28,28), titre=None):
    filtre_2D = filtre.reshape(shape)
    vabs = np.max(np.abs(filtre_2D))
    plt.imshow(filtre_2D, cmap='bwr', vmin=-vabs, vmax=vabs)
    plt.axis("off")
    if titre:
        plt.title(titre, fontsize=10, fontweight="bold")

def visualiser_filtres_didactique(W, shape=(28,28), n_affiche=10):
    plt.figure(figsize=(12, 6))
    for i in range(n_affiche):
        plt.subplot(2, n_affiche//2, i + 1)
        afficher_filtre_didactique(W[:, i], shape, titre=f"Filtre du neurone {i}")
    plt.suptitle(
        "Filtres appris -- zones rouges = pixels activants, bleues = inhibants",
        fontsize=14, fontweight="bold"
    )
    plt.tight_layout()
    plt.show()

visualiser_filtres_didactique(W1, n_affiche=min(W1.shape[1], 10))

def visualiser_W2(W2):
    plt.figure(figsize=(10, 4))
    plt.imshow(W2, cmap="bwr", aspect="auto")
    plt.colorbar()
    plt.title(f"Poids entre la couche cachée ({W2.shape[0]} neurones) et les {W2.shape[1]} sorties")
    plt.xlabel("Classe de sortie")
    plt.ylabel("Neurones cachés")
    plt.show()

visualiser_W2(W2)

# ------------ Fonction de prédiction ------------

def calque(I, W1, b1, W2, b2):
    Z1 = I.dot(W1) + b1
    A1 = 1 / (1 + np.exp(-Z1))  # sigmoïde couche cachée
    Z2 = A1.dot(W2) + b2
    A2 = 1 / (1 + np.exp(-Z2))  # sigmoïde multi-label
    return A2  # vecteur de probabilités pour chaque pathologie

# ------------ Prétraitement image individuelle ------------

def redim(img, f):
    return img.resize((f,f))

def convertgris(img):
    return np.array(img.convert("L")).astype(int)

def ajustpixels(array):
    return (255 - array)/255

def bonformat(img):
    img = redim(img, 28)
    img = convertgris(img)
    return img

def test_image():
    nom2 = input("Nom de l'image (sans extension) : ")
    ext = input("Format de l'image (ex: png, jpg) : ")
    try:
        inter = Image.open(f"{nom2}.{ext}")
        inter = ajustpixels(bonformat(inter))
        img = inter.flatten()
        pred = calque(img, W1, bias1, W2, bias2)
        print("Probabilités prédites :", pred)
        print("Labels prédits (seuil 0.5) :", (pred>0.5).astype(int))
    except:
        print("/!\\ Erreur : nom ou format de l'image incorrect")

# ------------ Test d'efficacité sur le dataset ------------

Nb_Erreur = 0
total_labels = len(Data_set) * 14

for i in range(len(Data_set)):
    pred = (calque(Data_set[i], W1, bias1, W2, bias2) > 0.5).astype(int)
    Nb_Erreur += np.sum(pred != test_dataset.labels[i].flatten())

precision = 100 * (total_labels - Nb_Erreur)/total_labels
print("Précision globale :", precision, "%")
```

Programme 8 (réseau non linéaire sur Chest-Xray)

```python
# -*- coding: utf-8 -*-
import numpy as np
import matplotlib.pyplot as plt

# ------------ Paramètres ------------

epochs = 10
learning_rate = 0.05

#choix de la méthode utilisée selon stochastic, full_batch, mini_batch pour la descente

methode = "mini_batch"
batch = 16 # généralement batch = 16, 32 ou 128

#choix de la fonction d'activation parmis sigmoide et softmax pour la dernière couche
#(la fonction d'activation pour la première couche est toujours sigmoïde)
fonction_activation = "softmax"

#choix du nombre de neuronnes dans la couche cachée
hidden_size = 64

#Redimensionnement des images :
nbpix = 150

# ------------ Vérification des paramètres ------------

assert type(epochs) == int, "epochs doit être un entier"
assert type(learning_rate) == float or type(learning_rate) == int, "le learning rate doit être un nombre"
assert methode in ["stochastic","full_batch","mini_batch"], "erreur sur la methode"
assert type(batch) == int, "batch doit être un entier"
assert fonction_activation in ["sigmoide","softmax"]
assert type(hidden_size) == int, "hidden_size doit être un entier"
assert type(nbpix) == int, "Nombre de pixels doit être un entier"


print("✅ Aucun problème pour les entrées")


# ------------ Téléchargement des données  ------------

import os
import glob
from PIL import Image


train_dir = r"C:\Users\23mat\.kaggle\chest_xray_data\archive\chest_xray\train" #chemin du dossier
categories = ["NORMAL", "PNEUMONIA"]

I_train_data = []
Y_train_data = []

for label, category in enumerate(categories):

    path = os.path.join(train_dir, category, "*.jpeg")  # ouvre toutes les images .jpeg de la category

    for file in glob.glob(path):
        try:
            img = Image.open(file).convert('L')
            img = img.resize((nbpix,nbpix))          # redimensionne
            I_train_data.append(list(np.array(img)))
            if label == 0:
                Y_train_data.append(np.array([1,0])) #cas NORMAL
            else:
                Y_train_data.append(np.array([0,1])) #cas PNEUMONIA

        except:
            print("""/!\\ Problème dans l'importantion des image /!\\""")

print("✅ Importation réussie")


# ------------ Vectorisation ------------

Data_set = []


for i in range (len(I_train_data)):
    N_image = []
    for j in range (nbpix):
        for k in range(nbpix):
            N_image.append((I_train_data[i][j][k]) /255)

    Data_set.append((np.array(N_image), Y_train_data[i]))

print("✅ Vectorisation effectuée")


# ------------ Implémentation ------------

def initialisation(hidden_size):
    """
        On initialise le model de variables (weights, bias)
    """

    #Initialisation de la première couche
    W1 = np.random.randn(nbpix**2, hidden_size)
    bias1 = np.zeros(hidden_size)

    #Initialisation de la seconde couche
    W2 = np.random.randn(hidden_size, 2)
    bias2 = np.zeros(2)

    return W1, bias1, W2, bias2

def pre_activation(I,W,bias):
    return I.dot(W)+bias

def activation(Z, fonction_activation):
    """
        Définition de la fonction sigmoïde / softmax
    """

    if fonction_activation == "sigmoide":

        #---------- Sigmoïde ----------
```

Programme 8 (réseau non linéaire sur Chest-Xray)

```
        #La fonction np.clip permet de limiter les valeurs prisent dans l'exp
        #ainsi on évite l'erreur d'overflow.
        #Par ailleurs, cela ne change rien car sigmoïde aurait renvoyée 0 ou 1

        Z = np.clip(Z, -500, 500)
        return 1 / (1 + np.exp(-Z))

    elif fonction_activation == "softmax":

        #---------- Softmax ----------

        exp_Z = np.exp(Z - np.max(Z))  # pour stabilité numérique et éviter l'overflow (astuce de calcul)
        return exp_Z / np.sum(exp_Z)

def model(I,W,bias,fonction_activation):
    Z = pre_activation(I, W, bias)
    return activation(Z,fonction_activation)

def Loss(A, S, fonction_activation):
    """
        Définition de la Log_Loss
    """
    epsilon = 1e-15  # petite constante pour éviter log(0)

    if fonction_activation == "sigmoide":

        # Cross-entropy binaire
        return -np.mean(S*np.log(A+epsilon)+(1-S)*np.log(1-A+epsilon))


    elif fonction_activation == "softmax":

        #Cross-entropy multi-classes
        return -np.mean(np.sum(S * np.log(A)))  # produit de Hadamard

def gradient(W1, bias1, W2, bias2, I, S, fonction_activation):

    A1 = model(I, W1, bias1, "sigmoide")
    A2 = model(A1, W2, bias2, fonction_activation)

    # Back-propagation

    # Descente sur la seconde couche
    dZ2 = A2 - S
    dW2 = np.outer(A1,dZ2)
    db2 = dZ2

    # Descente sur la première couche
    dZ1 = np.dot(W2, dZ2) * A1 * (1 - A1)  # produit matriciel
    dW1 = np.outer(I, dZ1) # ecriture python de I^T.dZ1
    db1 = dZ1

    return dW1, db1, dW2, db2

def update(dW1, W1, db1, bias1, dW2, W2, db2, bias2, learning_rate):

    W1 = W1 - learning_rate * dW1
    bias1 = bias1 - learning_rate * db1
    W2 = W2 - learning_rate * dW2
    bias2 = bias2 - learning_rate * db2

    return W1 , bias1, W2, bias2

def train(features,tag,W1,bias1,W2,bias2,epochs,learning_rate,methode,batch,fonction_activation):
    """
        descente du gradient
    """
    n = len(features)
    evo_cout = [] #évolution de l'erreur

    if methode == "full_batch":

        for epoch in range(epochs):

            print("Epoch ", epoch, "/", epochs, " starting -" )
            cout = []

            weights_gradient1 = np.zeros(W1.shape)
            bias_gradient1 = np.zeros(bias1.shape)
            weights_gradient2 = np.zeros(W2.shape)
            bias_gradient2 = np.zeros(bias2.shape)

            for feature, target in zip(features,tag):

                A1 = model(feature, W1, bias1, "sigmoide")
                A2 = model(A1, W2, bias2, fonction_activation)
                cout.append(Loss(A2,target,fonction_activation))

                dW1, db1, dW2, db2 = gradient(W1, bias1, W2, bias2, feature, target, fonction_activation)
                weights_gradient1 += dW1
                bias_gradient1 += db1
                weights_gradient2 += dW2
                bias_gradient2 += db2

            W1 , bias1, W2, bias2 = update(weights_gradient1/n, W1, bias_gradient1/n, bias1,
weights_gradient2/n, W2, bias_gradient2/n, bias2, learning_rate)

            evo_cout.append(np.mean(cout))

        return W1, bias1, W2, bias2, evo_cout

    elif methode == "stochastic":

        for epoch in range(epochs):

            print("Epoch ", epoch, "/", epochs, " starting -" )
            cout = []

            # On mélange les données pour ne pas faire systématiquement la même descente
            indices = np.random.permutation(n)
            features = features[indices]
            tag = tag[indices]

            for feature, target in zip(features,tag):

                A1 = model(feature, W1, bias1, "sigmoide")
                A2 = model(A1, W2, bias2, fonction_activation)
                cout.append(Loss(A2,target,fonction_activation))
```

Programme 8 (réseau non linéaire sur Chest-Xray)

```
            dW1, db1, dW2, db2 = gradient(W1, bias1, W2, bias2, feature, target, fonction_activation)
            W1 , bias1, W2, bias2 = update(weights_gradient1/n, W1, bias_gradient1/n, bias1,
weights_gradient2/n, W2, bias_gradient2/n, bias2, learning_rate)

        return W1, bias1, W2, bias2, evo_cout

    elif methode == "mini_batch":

        for epoch in range(epochs):

            print("Epoch ", epoch, "/", epochs, " starting -" )
            cout = []

            # On mélange les données pour ne pas faire systématiquement la même descente
            indices = np.random.permutation(n)
            features = features[indices]
            tag = tag[indices]


            for i in range(0, n, batch):
                batch_features = features[i:i + batch]
                batch_tags = tag[i:i + batch]

                weights_gradient1 = np.zeros(W1.shape)
                bias_gradient1 = np.zeros(bias1.shape)
                weights_gradient2 = np.zeros(W2.shape)
                bias_gradient2 = np.zeros(bias2.shape)

                for feature, target in zip(batch_features,batch_tags):

                    A1 = model(feature, W1, bias1, "sigmoide")
                    A2 = model(A1, W2, bias2, fonction_activation)
                    cout.append(Loss(A2,target,fonction_activation))

                    dW1, db1, dW2, db2 = gradient(W1, bias1, W2, bias2, feature, target,
fonction_activation)
                    weights_gradient1 += dW1
                    bias_gradient1 += db1
                    weights_gradient2 += dW2
                    bias_gradient2 += db2

                W1 , bias1, W2, bias2 = update(weights_gradient1/len(batch_features), W1,
bias_gradient1/len(batch_features), bias1, weights_gradient2/len(batch_features), W2,
bias_gradient2/len(batch_features), bias2, learning_rate)

                evo_cout.append(np.mean(cout))

        return W1, bias1, W2, bias2, evo_cout

# ------------ Entrainement ------------

#Initialisation de l'algorithme :

features = np.array([x[0] for x in Data_set])
tag = np.array([x[1] for x in Data_set])

W1, bias1, W2, bias2 = initialisation(hidden_size)

# Récupération des données après algorithme
W1, bias1, W2, bias2, evo_cout = train(features, tag, W1, bias1, W2, bias2, epochs, learning_rate,
methode, batch, fonction_activation)

np.savetxt("weights1_" + str(epochs) + "_" + fonction_activation + "_" + str(learning_rate) + "_" +
methode + "_" + str(batch) + "CC" + str(hidden_size) + ".txt", W1)
np.savetxt("bias1_" + str(epochs) + "_" + fonction_activation + "_" + str(learning_rate) + "_" + methode +
"_" + str(batch) + "CC" + str(hidden_size) + ".txt", bias1)
np.savetxt("weights2_" + str(epochs) + "_" + fonction_activation + "_" + str(learning_rate) + "_" +
methode + "_" + str(batch) + "CC" + str(hidden_size) + ".txt", W2)
np.savetxt("bias2_" + str(epochs) + "_" + fonction_activation + "_" + str(learning_rate) + "_" + methode +
"_" + str(batch) + "CC" + str(hidden_size) + ".txt", bias2)

print("✅ Poids et bias sauvegardés")


# ------------ Interface graphique ------------

def courbe_de_erreur(methode,fonction_activation, learning_rate, batch, hidden_size):

    if methode == "full_batch":

        info = "learning rate = " + str(learning_rate) + "\n" + \
       "méthode : " + methode + "\n" + \
       str(hidden_size) + " neuronnes dans la couche cachée" + "\n" + \
       "fonction d'activation : " + fonction_activation

        plt.plot(range(len(evo_cout)), evo_cout, label=info)
        plt.title("Evolution de l'erreur")
        plt.xlabel("n ième epoch")
        plt.legend()
        plt.show()

    elif methode == "mini_batch":

        info = "learning rate = " + str(learning_rate) + "\n" + \
            "méthode : " + methode + "\n" + \
            str(hidden_size) + " neuronnes dans la couche cachée" + "\n" + \
            "batch = " + str(batch) + "\n" + \
            "fonction d'activation : " + fonction_activation

        plt.plot(range(len(evo_cout)), evo_cout, label=info)
        plt.title("Evolution de l'erreur")
        plt.xlabel("n ième batch")
        plt.legend()
        plt.show()

        plt.figure()
        window_size = 100
        smooth_error = np.convolve(evo_cout, np.ones(window_size)/window_size, mode='valid')

        plt.plot(range(len(smooth_error)), smooth_error)
        plt.xlabel("Itération")
        plt.ylabel("Erreur moyenne")
        plt.title("Erreur lissée sur "+str(window_size)+" bachs")
        plt.show()

    elif methode == "stochastic":

        info = "learning rate = " + str(learning_rate) + "\n" + \
```

Programme 8 (réseau non linéaire sur Chest-Xray)

```
W1, bias1, W2, bias2, evo_cout = train(features, tag, W1, bias1, W2, bias2, epochs, learning_rate,
methode, batch, fonction_activation)

np.savetxt("weights1_" + str(epochs) + "_" + fonction_activation + "_" + str(learning_rate) + "_" +
methode + "_" + str(batch) + "CC" + str(hidden_size) + ".txt", W1)
np.savetxt("bias1_" + str(epochs) + "_" + fonction_activation + "_" + str(learning_rate) + "_" + methode +
"_" + str(batch) + "CC" + str(hidden_size) + ".txt", bias1)
np.savetxt("weights2_" + str(epochs) + "_" + fonction_activation + "_" + str(learning_rate) + "_" +
methode + "_" + str(batch) + "CC" + str(hidden_size) + ".txt", W2)
np.savetxt("bias2_" + str(epochs) + "_" + fonction_activation + "_" + str(learning_rate) + "_" + methode +
"_" + str(batch) + "CC" + str(hidden_size) + ".txt", bias2)

print("✅ Poids et bias sauvegardés")


# ------------ Interface graphique ------------

def courbe_de_erreur(methode,fonction_activation, learning_rate, batch, hidden_size):

    if methode == "full_batch":

        info = "learning rate = " + str(learning_rate) + "\n" + \
       "méthode : " + methode + "\n" + \
       str(hidden_size) + " neuronnes dans la couche cachée" + "\n" + \
       "fonction d'activation : " + fonction_activation

        plt.plot(range(len(evo_cout)), evo_cout, label=info)
        plt.title("Evolution de l'erreur")
        plt.xlabel("n ième epoch")
        plt.legend()
        plt.show()

    elif methode == "mini_batch":

        info = "learning rate = " + str(learning_rate) + "\n" + \
            "méthode : " + methode + "\n" + \
            str(hidden_size) + " neuronnes dans la couche cachée" + "\n" + \
            "batch = " + str(batch) + "\n" + \
            "fonction d'activation : " + fonction_activation

        plt.plot(range(len(evo_cout)), evo_cout, label=info)
        plt.title("Evolution de l'erreur")
        plt.xlabel("n ième batch")
        plt.legend()
        plt.show()

        plt.figure()
        window_size = 100
        smooth_error = np.convolve(evo_cout, np.ones(window_size)/window_size, mode='valid')

        plt.plot(range(len(smooth_error)), smooth_error)
        plt.xlabel("Itération")
        plt.ylabel("Erreur moyenne")
        plt.title("Erreur lissée sur "+str(window_size)+" bachs")
        plt.show()

    elif methode == "stochastic":

        info = "learning rate = " + str(learning_rate) + "\n" + \
```

Programme 9 (réseau non linéaire sur Cat vs Dog)

```python
# -*- coding: utf-8 -*-

import numpy as np
import os
import glob
from PIL import Image
import matplotlib.pyplot as plt

# ------------ Paramètres ------------

epochs = 20
learning_rate = 0.001

#choix de la méthode utilisée selon stochastic, full_batch, mini_batch pour la descente

methode = "mini_batch"
batch = 32 # généralement batch = 16, 32 ou 128

#choix de la fonction d'activation parmis sigmoide et softmax pour la dernière couche
#(la fonction d'activation pour la première couche est toujours sigmoïde)
fonction_activation = "softmax"

#choix du nombre de neuronnes dans la couche cachée
hidden_size = 128

#Redimensionnement des images :
nbpix = 32

# ------------ Vérification des paramètres ------------

assert type(epochs) == int, "epochs doit être un entier"
assert type(learning_rate) == float or type(learning_rate) == int, "le learning rate doit être un nombre"
assert methode in ["stochastic","full_batch","mini_batch"], "erreur sur la methode"
assert type(batch) == int, "batch doit être un entier"
assert fonction_activation in ["sigmoide","softmax"]
assert type(hidden_size) == int, "hidden_size doit être un entier"
assert type(nbpix) == int, "Nombre de pixels doit être un entier"


print("✅ Aucun problème pour les entrées")


# ------------ Téléchargement des données  ------------

train_dir = r"C:\Users\23mat\OneDrive\Bureau\TIPE\CONCOURS TIPE\data_set_dog_cat\archive\PetImages"
#chemin du dossier
categories = ["Cat", "Dog"]

features = []
targets = []

for label, category in enumerate(categories):

    path = os.path.join(train_dir, category, "*.jpg")  # ouvre toutes les images .jpeg de la category

    for file in glob.glob(path):
        try:
            img = Image.open(file).resize((nbpix,nbpix))
            img = img.convert('L')
            features.append(np.array(img).flatten())  # flatten indispensable (28x28 -> 784)
            if label == 0:
                targets.append(np.array([1,0])) #cas Cat
            else:
                targets.append(np.array([0,1])) #cas Dog

        except:
            print("""/!\\ Problème dans l'importation des image /!\\""")


features_ = np.array(features[:20000]) / 255.0 # Normalisation
print(len(features_))
targets_ = np.array(targets[:20000])

indices = np.random.permutation(len(features_)) # On mélange
features_ = features_[indices]
targets_ = targets_[indices]

Data_set = [(x,y) for x,y in zip (features_,targets_)]
print("✅ Téléchargement effectué")


# ------------ Implémentation ------------

def initialisation(hidden_size):
    """
        On initialise le model de variables (weights, bias)
    """

    #Initialisation de la première couche
    W1 = np.random.randn(nbpix**2, hidden_size)
    bias1 = np.zeros(hidden_size)

    #Initialisation de la seconde couche
    W2 = np.random.randn(hidden_size, 2)
    bias2 = np.zeros(2)

    return W1, bias1, W2, bias2

def pre_activation(I,W,bias):
    return I.dot(W)+bias

def activation(Z, fonction_activation):
    """
        Définition de la fonction sigmoïde / softmax
    """

    if fonction_activation == "sigmoide":

        #----------- Sigmoïde -----------

        #La fonction np.clip permet de limiter les valeurs prisent dans l'exp
        #ainsi on évite l'erreur d'overflow.
        #Par ailleurs, cela ne change rien car sigmoïde aurait renvoyée 0 ou 1

        Z = np.clip(Z, -500, 500)
        return 1 / (1 + np.exp(-Z))

    elif fonction_activation == "softmax":
```

Programme 9 (réseau non linéaire sur Cat vs Dog)

```
        #---------- Softmax ----------

        exp_Z = np.exp(Z - np.max(Z))  # pour stabilité numérique et éviter l'overflow (astuce de calcul)
        return exp_Z / np.sum(exp_Z)

def model(I,W,bias,fonction_activation):
    Z = pre_activation(I, W, bias)
    return activation(Z,fonction_activation)

def Loss(A, S, fonction_activation):
    """
        Définition de la Log_Loss
    """
    epsilon = 1e-15  # petite constante pour éviter log(0)

    if fonction_activation == "sigmoide":

        # Cross-entropy binaire
        return -np.mean(S*np.log(A+epsilon)+(1-S)*np.log(1-A+epsilon))


    elif fonction_activation == "softmax":

        #Cross-entropy multi-classes
        return -np.mean(np.sum(S * np.log(A)))  # produit de Hadamard

def gradient(W1, bias1, W2, bias2, I, S, fonction_activation):

    A1 = model(I, W1, bias1, "sigmoide")
    A2 = model(A1, W2, bias2, fonction_activation)

    # Back-propagation

    # Descente sur la seconde couche
    dZ2 = A2 - S
    dW2 = np.outer(A1,dZ2)
    db2 = dZ2

    # Descente sur la première couche
    dZ1 = np.dot(W2, dZ2) * A1 * (1 - A1)  # produit matriciel
    dW1 = np.outer(I, dZ1) # ecriture python de I^T.dZ1
    db1 = dZ1

    return dW1, db1, dW2, db2

def update(dW1, W1, db1, bias1, dW2, W2, db2, bias2, learning_rate):

    W1 = W1 - learning_rate * dW1
    bias1 = bias1 - learning_rate * db1
    W2 = W2 - learning_rate * dW2
    bias2 = bias2 - learning_rate * db2

    return W1 , bias1, W2, bias2

def train(features,tag,W1,bias1,W2,bias2,epochs,learning_rate,methode,batch,fonction_activation):
    """
        descente du gradient
    """
    n = len(features)
    evo_cout = [] #évolution de l'erreur

    if methode == "full_batch":

        for epoch in range(epochs):

            print("Epoch ", epoch, "/", epochs, " starting -" )
            cout = []

            weights_gradient1 = np.zeros(W1.shape)
            bias_gradient1 = np.zeros(bias1.shape)
            weights_gradient2 = np.zeros(W2.shape)
            bias_gradient2 = np.zeros(bias2.shape)

            for feature, target in zip(features,tag):

                A1 = model(feature, W1, bias1, "sigmoide")
                A2 = model(A1, W2, bias2, fonction_activation)
                cout.append(Loss(A2,target,fonction_activation))

                dW1, db1, dW2, db2 = gradient(W1, bias1, W2, bias2, feature, target, fonction_activation)
                weights_gradient1 += dW1
                bias_gradient1 += db1
                weights_gradient2 += dW2
                bias_gradient2 += db2

            W1 , bias1, W2, bias2 = update(weights_gradient1/n, W1, bias_gradient1/n, bias1,
weights_gradient2/n, W2, bias_gradient2/n, bias2, learning_rate)

            evo_cout.append(np.mean(cout))

        return W1, bias1, W2, bias2, evo_cout

    elif methode == "stochastic":

        for epoch in range(epochs):

            print("Epoch ", epoch, "/", epochs, " starting -" )
            cout = []

            # On mélange les données pour ne pas faire systématiquement la même descente
            indices = np.random.permutation(n)
            features = features[indices]
            tag = tag[indices]

            for feature, target in zip(features,tag):

                A1 = model(feature, W1, bias1, "sigmoide")
                A2 = model(A1, W2, bias2, fonction_activation)
                cout.append(Loss(A2,target,fonction_activation))

                dW1, db1, dW2, db2 = gradient(W1, bias1, W2, bias2, feature, target, fonction_activation)
                W1 , bias1, W2, bias2 = update(weights_gradient1/n, W1, bias_gradient1/n, bias1,
weights_gradient2/n, W2, bias_gradient2/n, bias2, learning_rate)

        return W1, bias1, W2, bias2, evo_cout

    elif methode == "mini_batch":
```

Programme 9 (réseau non linéaire sur Cat vs Dog)

```
        for epoch in range(epochs):

            print("Epoch ", epoch, "/", epochs, " starting -" )
            cout = []

            # On mélange les données pour ne pas faire systématiquement la même descente
            indices = np.random.permutation(n)
            features = features[indices]
            tag = tag[indices]


            for i in range(0, n, batch):
                batch_features = features[i:i + batch]
                batch_tags = tag[i:i + batch]

                weights_gradient1 = np.zeros(W1.shape)
                bias_gradient1 = np.zeros(bias1.shape)
                weights_gradient2 = np.zeros(W2.shape)
                bias_gradient2 = np.zeros(bias2.shape)

                for feature, target in zip(batch_features,batch_tags):

                    A1 = model(feature, W1, bias1, "sigmoide")
                    A2 = model(A1, W2, bias2, fonction_activation)
                    cout.append(Loss(A2,target,fonction_activation))

                    dW1, db1, dW2, db2 = gradient(W1, bias1, W2, bias2, feature, target,
fonction_activation)
                    weights_gradient1 += dW1
                    bias_gradient1 += db1
                    weights_gradient2 += dW2
                    bias_gradient2 += db2

                W1 , bias1, W2, bias2 = update(weights_gradient1/len(batch_features), W1,
bias_gradient1/len(batch_features), bias1, weights_gradient2/len(batch_features), W2,
bias_gradient2/len(batch_features), bias2, learning_rate)

                evo_cout.append(np.mean(cout))

        return W1, bias1, W2, bias2, evo_cout

# ------------ Entrainement ------------

#Initialisation de l'algorithme :

features = np.array([x[0] for x in Data_set])
tag = np.array([x[1] for x in Data_set])

W1, bias1, W2, bias2 = initialisation(hidden_size)

# Récupération des données après algorithme

W1, bias1, W2, bias2, evo_cout = train(features, tag, W1, bias1, W2, bias2, epochs, learning_rate,
methode, batch, fonction_activation)

np.savetxt("weights1_" + str(epochs) + "_" + fonction_activation + "_" + str(learning_rate) + "_" +
methode + "_" + str(batch) + "CC" + str(hidden_size) + ".txt", W1)
np.savetxt("bias1_" + str(epochs) + "_" + fonction_activation + "_" + str(learning_rate) + "_" + methode +
"_" + str(batch) + "CC" + str(hidden_size) + ".txt", bias1)
np.savetxt("weights2_" + str(epochs) + "_" + fonction_activation + "_" + str(learning_rate) + "_" +
methode + "_" + str(batch) + "CC" + str(hidden_size) + ".txt", W2)
np.savetxt("bias2_" + str(epochs) + "_" + fonction_activation + "_" + str(learning_rate) + "_" + methode +
"_" + str(batch) + "CC" + str(hidden_size) + ".txt", bias2)

print("✅ Poids et bias sauvegardés")



# ------------ Interface graphique ------------

def augment_np(img_np):
    """
        Augmentation simple et compatible avec la vectorisation
        img_np : numpy array (nbpix*nbpix,) ou (nbpix,nbpix)
    """

    # Remise en forme si image flatten
    if img_np.ndim == 1:
        img_np = img_np.reshape(nbpix, nbpix)

    # Flip horizontal
    if np.random.rand() < 0.5:
        img_np = np.fliplr(img_np)

    # Variation de luminosité
    img_np = img_np + np.random.uniform(-0.1, 0.1)
    img_np = np.clip(img_np, 0.0, 1.0)

    # Bruit gaussien
    img_np = img_np + np.random.normal(0, 0.02, img_np.shape)
    img_np = np.clip(img_np, 0.0, 1.0)

    # Rotation légère (avec PIL)
    pil = Image.fromarray((img_np * 255).astype("uint8"))
    angle = np.random.uniform(-10, 10)
    pil = pil.rotate(angle, resample=Image.BILINEAR)

    # Toujours revenir à nbpix × nbpix
    pil = pil.resize((nbpix, nbpix))

    # Retour numpy normalisé entre 0 et 1 + flatten
    img_np = np.array(pil).astype("float32") / 255.0

    return img_np.flatten()


# ------------ Interface graphique ------------

def courbe_de_erreur(methode,fonction_activation, learning_rate, batch, hidden_size):

    if methode == "full_batch":

        info = "learning rate = " + str(learning_rate) + "\n" + \
       "méthode : " + methode + "\n" + \
       str(hidden_size) + " neuronnes dans la couche cachée" + "\n" + \
       "fonction d'activation : " + fonction_activation
```

Programme 9 (réseau non linéaire sur Cat vs Dog)

```
        plt.plot(range(len(evo_cout)), evo_cout, label=info)
        plt.title("Evolution de l'erreur")
        plt.xlabel("n ième epoch")
        plt.legend()
        plt.show()

    elif methode == "mini_batch":

        info = "learning rate = " + str(learning_rate) + "\n" + \
            "méthode : " + methode + "\n" + \
            str(hidden_size) + " neuronnes dans la couche cachée" + "\n" + \
            "batch = " + str(batch) + "\n" + \
            "fonction d'activation : " + fonction_activation

        plt.plot(range(len(evo_cout)), evo_cout, label=info)
        plt.title("Evolution de l'erreur")
        plt.xlabel("n ième batch")
        plt.legend()
        plt.show()

        plt.figure()
        window_size = 100
        smooth_error = np.convolve(evo_cout, np.ones(window_size)/window_size, mode='valid')

        plt.plot(range(len(smooth_error)), smooth_error)
        plt.xlabel("Itération")
        plt.ylabel("Erreur moyenne")
        plt.title("Erreur lissée sur "+str(window_size)+" bachs")
        plt.show()

    elif methode == "stochastic":

        info = "learning rate = " + str(learning_rate) + "\n" + \
            "méthode : " + methode + "\n" + \
            str(hidden_size) + " neuronnes dans la couche cachée" + "\n" + \
            "fonction d'activation : " + fonction_activation

        plt.plot(range(len(evo_cout)), evo_cout, label=info)
        plt.title("Evolution de l'erreur")
        plt.xlabel("n ième itération")
        plt.legend()
        plt.show()

        plt.figure()
        window_size = 1000
        smooth_error = np.convolve(evo_cout, np.ones(window_size)/window_size, mode='valid')

        plt.plot(range(len(smooth_error)), smooth_error)
        plt.xlabel("Itération")
        plt.ylabel("Erreur moyenne")
        plt.title("Erreur lissée sur "+str(window_size)+" itérations")
        plt.show()

courbe_de_erreur(methode, fonction_activation, learning_rate, batch, hidden_size)
```

Programme (Convolution)

```
import glob
import numpy as np
from PIL import Image
import matplotlib.pyplot as plt
import os
import tensorflow.compat.v1 as tf
tf.disable_v2_behavior()
from scipy.ndimage import rotate, zoom


# ------------ Paramètres ------------

nbpix = 224 # dimension d'un côté d'une image
nbcol = 1 # Correspond au système de couleur (1 pour gris et 3 pour RGB)
nb_neuronne_couche_int = 128 #256
batch_size = 16
epochs = 45


# ------------ Vérification des paramètres ------------

assert type(nbpix) == int, "La dimension du côté d'une image doit être un entier"
assert type(nbcol) == int, "nbcol doit être un entier"
assert type(nb_neuronne_couche_int) == int, "Il doit y avoir un nombre entier de neurones sur la couche
intermédiaire"
assert type(batch_size) == int, "batch doit être un entier"
assert type(epochs) == int, "epochs doit être entier"


print("✅ Aucun problème pour les entrées")


# ------------ Téléchargement des données  ------------

train_dir = r"C:\Users\23mat\.kaggle\chest_xray_data\archive\chest_xray\train" #chemin du dossier
categories = ["NORMAL", "PNEUMONIA"]

features = []
targets = []

for label, category in enumerate(categories):

    path = os.path.join(train_dir, category, "*.jpeg")  # ouvre toutes les images .jpeg de la category

    for file in glob.glob(path):
        try:
            img = Image.open(file).convert('L')  # converti en niveaux de gris
            img = img.resize((nbpix,nbpix))          # redimensionne
            features.append(np.array(img))
            if label == 0:
                targets.append(np.array([1,0])) #cas NORMAL
            else:
                targets.append(np.array([0,1])) #cas PNEUMONIA

        except:
             print("""/!\\ Problème dans l'importantion des image /!\\""")


features = np.array(features) / 255.0 # Normalisation
targets = np.array(targets)

indices = np.random.permutation(len(features)) # On mélange
features = features[indices]
targets = targets[indices]

print("✅ Téléchargement effectué")


# ------------ Affichage d'images pour l'exemple ------------


def affichage(nb):

    plt.imshow(features[nb], cmap='gray')
    plt.title("NORMAL" if targets[nb][0]==1 else "PNEUMONIA")
    plt.axis('off')
    plt.show()

affichage(100) # NORMAL
affichage(103) # PNEUMONIA


# ------------ Implémentation de la structure et du modèle ------------

# Entrées du modèle

x = tf.placeholder(tf.float32,(None, nbpix, nbpix, nbcol), name = 'Image')
y = tf.placeholder(tf.float32, (None,2), name = 'Targets')


# Filtres pour la convolution

def create_conv(prev,dimfiltre,nbfiltre):
    """
        Permet de créer les filtres
    """

    W_filters =
tf.Variable(tf.truncated_normal(shape=(dimfiltre,dimfiltre,int(prev.get_shape()[-1]),nbfiltre)))
    b_filters = tf.Variable(tf.zeros(shape=(nbfiltre)))

    conv = tf.nn.conv2d(prev, W_filters, strides=[1,1,1,1], padding='SAME') + b_filters
    # Srides correspond à comment on se décale entre les filtres
    # et padding dit si l'on peut déborder de l'image ou non : SAME oui VALID non

    conv = tf.nn.relu(conv) #Relu est la fonction d'activation (permet d'avoir un réseau non linéaire
comme l'ajout de couches intermédiaires)
    conv = tf.nn.max_pool(conv, ksize=[1,2,2,1], strides=[1,2,2,1], padding='SAME') # Implémentation du
pooling

    return conv

conv = create_conv(x,8,32)
conv = create_conv(conv,5,64)
conv = create_conv(conv,5,64)
conv = create_conv(conv,5,128)
conv = create_conv(conv,5,256)
```

Programme (Convolution)

```
flat = tf.reshape(conv, [-1, np.prod(conv.get_shape().as_list()[1:])]) # On vectorise sur une ligne

# Implémentation du réseau de neuronne (couche 1)

w1 = tf.Variable(tf.truncated_normal(shape=(int(flat.get_shape()[1]),nb_neuronne_couche_int)))
b1 = tf.Variable(tf.zeros(shape=(nb_neuronne_couche_int)))

fc1 = tf.matmul(flat,w1) + b1 # correspond au produit matriciel qui renvoie les sorties de la première couche
fc1 = tf.nn.relu(fc1) #On active

# Implémentation du réseau de neuronne (couche 2)

w2 = tf.Variable(tf.truncated_normal(shape=(nb_neuronne_couche_int,2)))
b2 = tf.Variable(tf.zeros(shape=(2)))

logits = tf.matmul(fc1,w2) + b2

softmax = tf.nn.softmax(logits)


# ------------ Erreur ------------

cross_entropy = tf.nn.softmax_cross_entropy_with_logits(labels=y, logits=logits)
loss_operation = tf.reduce_mean(cross_entropy)

correct_prediction = tf.equal(tf.argmax(softmax, axis=1), tf.argmax(y,axis=1))
accuracy = tf.reduce_mean(tf.cast(correct_prediction, tf.float32))

optimizer = tf.train.AdamOptimizer(0.001)
train_op = optimizer.minimize(loss_operation)

# ------------ Ajout artificiel d'images par transformation ------------


def augment_np(img_np):
    # img_np : numpy array (150,150)

    # Flip horizontal
    if np.random.rand() < 0.5:
        img_np = np.fliplr(img_np)

    # Variation de luminosité
    img_np = img_np + np.random.uniform(-0.1, 0.1)
    img_np = np.clip(img_np, 0.0, 1.0)

    # Bruit gaussien
    img_np = img_np + np.random.normal(0, 0.02, img_np.shape)
    img_np = np.clip(img_np, 0.0, 1.0)

    # Rotation légère (avec PIL)
    pil = Image.fromarray((img_np * 255).astype("uint8"))
    angle = np.random.uniform(-10, 10)
    pil = pil.rotate(angle, resample=Image.BILINEAR)

    # Toujours revenir à 150×150
    pil = pil.resize((nbpix, nbpix))

    # Retour numpy normalisé entre 0 et 1
    img_np = np.array(pil).astype("float32") / 255.0

    return img_np.reshape(nbpix, nbpix, 1)



# ------------ Descente du gradient ------------

sess = tf.Session()
sess.run(tf.global_variables_initializer())

accs = []

for epoch in range(epochs):

    print ("Epoch ", epoch, "/", epochs, " starting...")

    # On mélange

    index = np.arange(len(features))
    np.random.shuffle(index)
    features = features[index]
    targets = targets[index]

    for b in range(0, len(features), batch_size):

        batch = features[b:b+batch_size]
        batch_aug = np.array([augment_np(imge) for imge in batch])
        acc = sess.run(train_op,feed_dict={x:batch_aug,y:targets[b:b+batch_size]})
        accs.append(acc)


# ------------ TEST ------------


test_dir = r"C:\Users\23mat\.kaggle\chest_xray_data\archive\chest_xray\test"
test_features = []
test_targets = []

for label, category in enumerate(categories):
    path = os.path.join(test_dir, category, "*.jpeg")
    for file in glob.glob(path):
        try:
            img = Image.open(file).convert("L")
            img = img.resize((nbpix, nbpix))
            arr = np.array(img).astype("float32") / 255.0
            test_features.append(arr.reshape(nbpix, nbpix, 1))
            test_targets.append(np.array([1,0]) if label == 0 else np.array([0,1]))
        except:
            print("Erreur test :", file)

test_features = np.array(test_features)
test_targets = np.array(test_targets)

print("📁 Test chargé :", len(test_features), "images")
```

Programme （ Convolution ）

```
# ----------------------------
#  Calcul de l'accuracy sur TEST
# ----------------------------

feed_test = {x: test_features, y: test_targets}
test_acc, test_loss = sess.run([accuracy, loss_operation], feed_dict=feed_test)

print("\n==============================")
print("   📌 RÉSULTATS SUR LE TEST   ")
print("==============================")
print(" 🔹 Test Accuracy :", round(test_acc*100, 2), "%")
print("==============================\n")
```

I. Réseau linéaire basique -

On suppose qu'il existe un espace probabilisé qui schématise la situation. On note X la variable aléatoire renvoyant le taux de globules blancs et rouges, Y la variable aléatoire renvoyant 1 si le patient est malade et 0 sinon.

On définit la loi conditionnelle de Y :

$$P_{\{X=x\}} : \{Y=0; Y=1\} \longrightarrow [0,1]$$

$$A \longmapsto \begin{cases} \mathsf{a}(z) & \text{si } A = \{Y=1\} \\ 1-\mathsf{a}(z) & \text{si } A = \{Y=0\} \\ 0 & \text{si } A = \varnothing \\ 1 & \text{si } A = \{Y=1; Y=0\} \end{cases}$$

I. Réseau linéaire basique -

Démonstration

- $\forall A \in \mathcal{P}(\{Y=1, Y=0\}), \quad P_{\{X=x\}}(A) \in [0,1]$
- $P_{\{X=x\}}\big(\{Y=1, Y=0\}\big) = 1$
- $\{Y=0\}$ et $\{Y=1\}$ sont les seuls événements incompatibles :

$$P_{\{X=x\}}\big(\{Y=1\} \cup \{Y=0\}\big) = P_{\{X=x\}}\big(\{Y=1; Y=0\}\big) = 1$$

$$P_{\{X=x\}}\big(\{Y=1\}\big) + P_{\{X=x\}}\big(\{Y=0\}\big) = \sigma(z) + 1 - \sigma(z) = 1$$

I. Réseau linéaire basique -

On suppose qu'il existe un espace probabilisé qui schématise la situation. On note X la variable aléatoire renvoyant les données, Y la variable aléatoire renvoyant le numéro de la classe.

On définit la loi conditionnelle de Y :

$$P_{\{X=x\}} : \bigcup_{i=1}^{N} \{Y=i\} \longrightarrow [0,1]$$

$$A \longmapsto \sum_{j \in J} S(Z)_j \quad \text{avec } J \subset [\![1, N]\!],\ A = \bigcup_{k \in J} \{Y=k\}$$

$P_{\{X=x\}}$ est bien définie car $\forall k$, q $\in [\![1, N]\!]$, k et q distincts, $\{Y =$k$\}$ et $\{Y =$q$\}$ sont incompatibles.

I. Réseau linéaire basique -

Démonstration

- $\forall A \in \mathcal{P}\left(\bigcup_{i=1}^{N}\{Y=i\}\right), \quad P_{\{X=x\}}(A) \in [0,1]$
- $P_{\{X=x\}}\left(\bigcup_{i=1}^{N}\{Y=i\}\right) = \sum_{k=1}^{N} \frac{e^{Z_k}}{\sum_{j=1}^{N} e^{Z_j}} = 1$
- Soit $k, \ell \in [\![1, N]\!]$ tels que $k \neq \ell$

$$
\begin{aligned}
P_{\{X=x\}}(\{Y=k\} \cup \{Y=\ell\}) &= P_{\{X=x\}}(\{Y=k \ , \ Y=\ell\}) \\
&= \sum_{j \in \{k,\ell\}} S(Z_j) \ = \ S(Z_k) + S(Z_\ell) \\
&= P_{\{X=x\}}(\{Y=k\}) + P_{\{X=x\}}(\{Y=\ell\})
\end{aligned}
$$

Sources :

Applications et sites permettant l'expérimentation des réseaux de neuronnes

- Pour visualiser et comprendre ce qu'il se passe lorsque l'on ajoute des couches de neurones : *Neural Network by Tensorflow* :
  https://playground.tensorflow.org/#activation=sigmoid&batchSize=21&dataset=xor&regDataset=reg-plane&learningRate=0.03&regularizationRate=0&noise=0&networkShape=2,2&seed=0.72040&showTestData=false&discretize=false&percTrainData=50&x=true&y=true&xTimesY=false&xSquared=false&ySquared=false&cosX=false&sinX=false&cosY=false&sinY=false&collectStats=false&problem=classification&initZero=false&hideText=false

Sources :

Vidéos Youtube expliquant les réseaux de neurones

- Coder des réseaux linéaires et non linéaires ainsi que comprendre leurs origines : Chaîne Youtube @MachineLearnia
- Ajouter la convolution et en apprendre plus sur le machine learning : Chaîne Youtube @ThibaultNeveu
- Comprendre la convolution (vidéo) : *But what is a convolution?*, *3Blue1Brown* : https://www.youtube.com/watch?v=KuXjwB4LzSA

Sources :

Informations

- Chiffres de l'OMS : https://www.who.int/fr/news/item/09-12-2020-who-reveals-leading-causes-of-death-and-disability-worldwide-2000-2019?utm_source=chatgpt.com

- SARVAMANGALA D., KULKARNI R. V. : NIH, Convolutional Neural Networks in Medical Image Understanding : PMCID: PMC7778711, PMID: 33425040, 2021

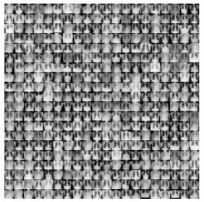
- IBOMOIYE DOMOR MIENYE, THEO G. SWART, GEORGE OBAIDO, MATT JORDAN, PHILIP ILONO : MDPI, Deep Learning-Based Image Classification : vol. 16, n°3, 2025, article 195

- RAVI AGGARWAL, VIKNESH SOUNDERAJAH, GUY MARTIN, DANIEL S. W. TING, ALAN KARTHIKESALINGAM, DOMINIC KING, HUTAN ASHRAFIAN, ARA DARZI : Nature, Diagnostic accuracy of deep learning in medical imaging: a systematic review and metaanalysis : Nature Digital Medicine, vol. 4, article 38, 2021

Sources :

Datasets

Facts of **ChestMNIST**



Data Modality: Chest X-Ray
Task: Multi-Label (14) Binary-Class (2)
Number of Samples: 112,120 (78,468 / 11,219 / 22,433)
Source Data:

Xiaosong Wang, Yifan Peng, et al., "Chest x-ray8: Hospital-scale chest x-ray database and benchmarks on weakly-supervised classification and localization of common thorax diseases," in CVPR, 2017, pp. 3462–3471.

License: CC BY 4.0

- Chest X-Ray :
  https://www.kaggle.com/datasets/paultimothymooney/chest-xray-pneumonia/data
- MNIST :
  https://www.kaggle.com/datasets/hojjatk/mnist-dataset
- Cats-vs-Dogs, Microsoft :
  https://www.kaggle.com/datasets/shaunthesheep/microsoft-catsvsdogs-dataset
- MNIST :
  https://www.kaggle.com/datasets/hojjatk/mnist-dataset